

Chapter 4

Machine Learning

Overview

4.1 Foundations and four pillars of machine learning

4.2 Review of mathematics for machine learning

4.3 Continuation optimization; Unconstraint optimization, constraint optimization, convex optimization

4.4 Review of machine learning pipeline

4.4.1 Model development: Generative versus discriminative

4.4.2 Learning algorithm

4.4.3 Capacity, overfitting and underfitting

4.4.4 Hyperparameters and validation sets

4.4.5 Estimators, bias and variance

4.4.6 Review of MLE and MAP and cross entropy

4.5 Supervised learning algorithm: Decision tree and support vector machine

4.6 Unsupervised learning algorithm: t-SNE

4.7 Semi supervised and reinforcement learning

4.8 Model evaluation: Energy-based indicators

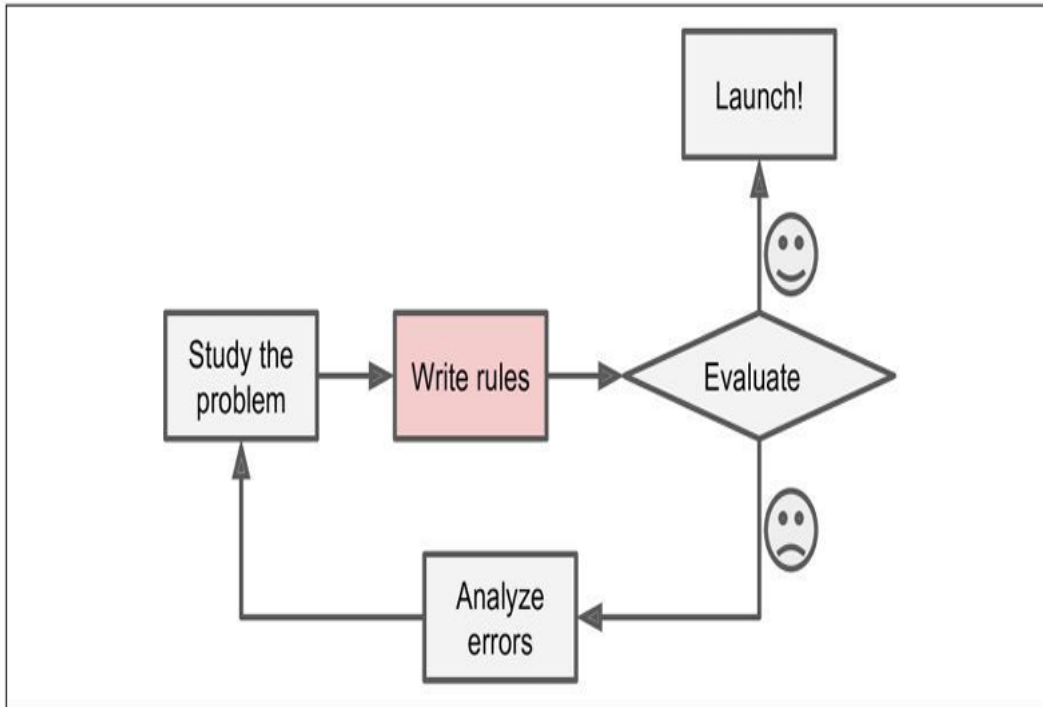
Machine Learning

- Machine Learning is the science (and art) of programming computers so they can learn from data.
- Machine learning is a branch of AI that uses algorithm to allow computer to evolve behaviors based on data collected from databases or gathered through sensors.
- **Arthur Samuel (1959)** defined Machine Learning as: “The field of study that gives computers the ability to learn without being explicitly programmed.”
- **Tom Mitchell** defined Machine Learning as: “A computer program is said to learn from experience E with respect to some task T and performance measure P if its performance at task T , as measured by P , improves with experience E .”

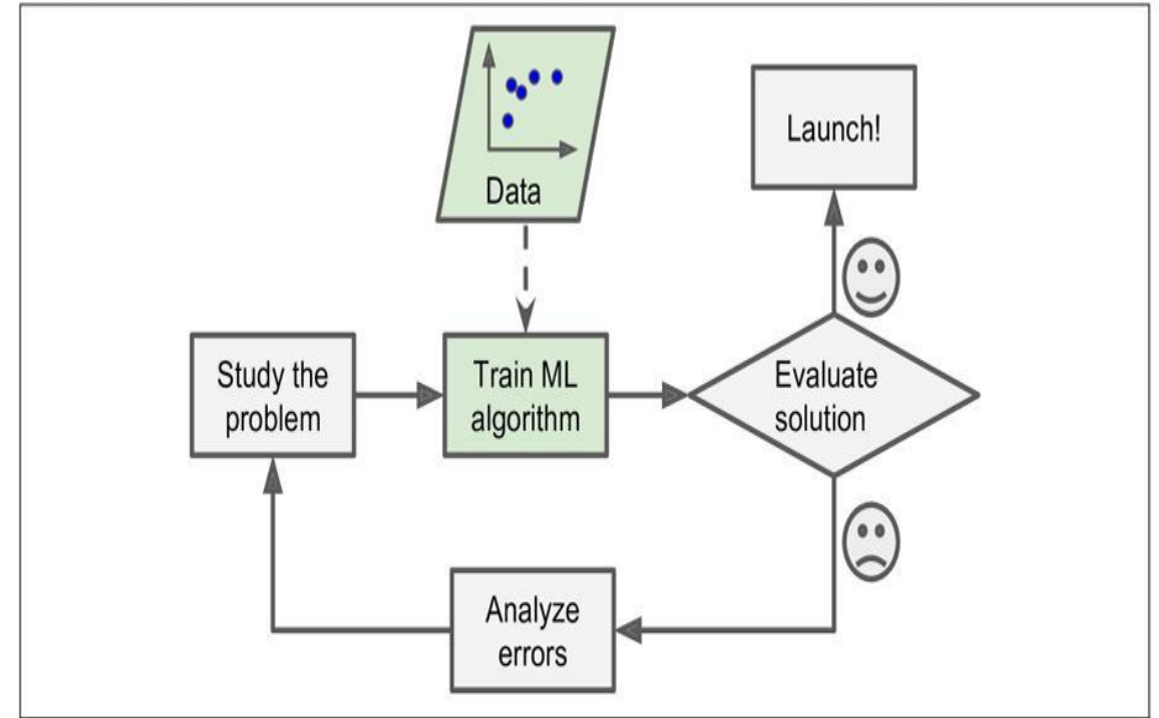
Machine Learning

- We can elaborate Tom Mitchell's definition of Machine learning as a concept that uses three components:
 1. Task (T): The objective the computer wants to perform. Examples: Predicting house prices , Detecting spam emails, Recognizing handwritten digits, etc.
 2. Experience (E): The data used for learning. Examples: Historical house prices, Thousands of spam emails and Millions of handwritten digits.
 3. Performance Measure (P): Metrics that measures how well the model performs. Examples: Accuracy, Precision, Recall, etc.
- Machine learning is about designing algorithms that automatically extract valuable information from data.
- The emphasis here is on “automatic”, i.e., machine learning is concerned about general-purpose methodologies that can be applied to many datasets, while producing something that is meaningful.

Why use Machine Learning?

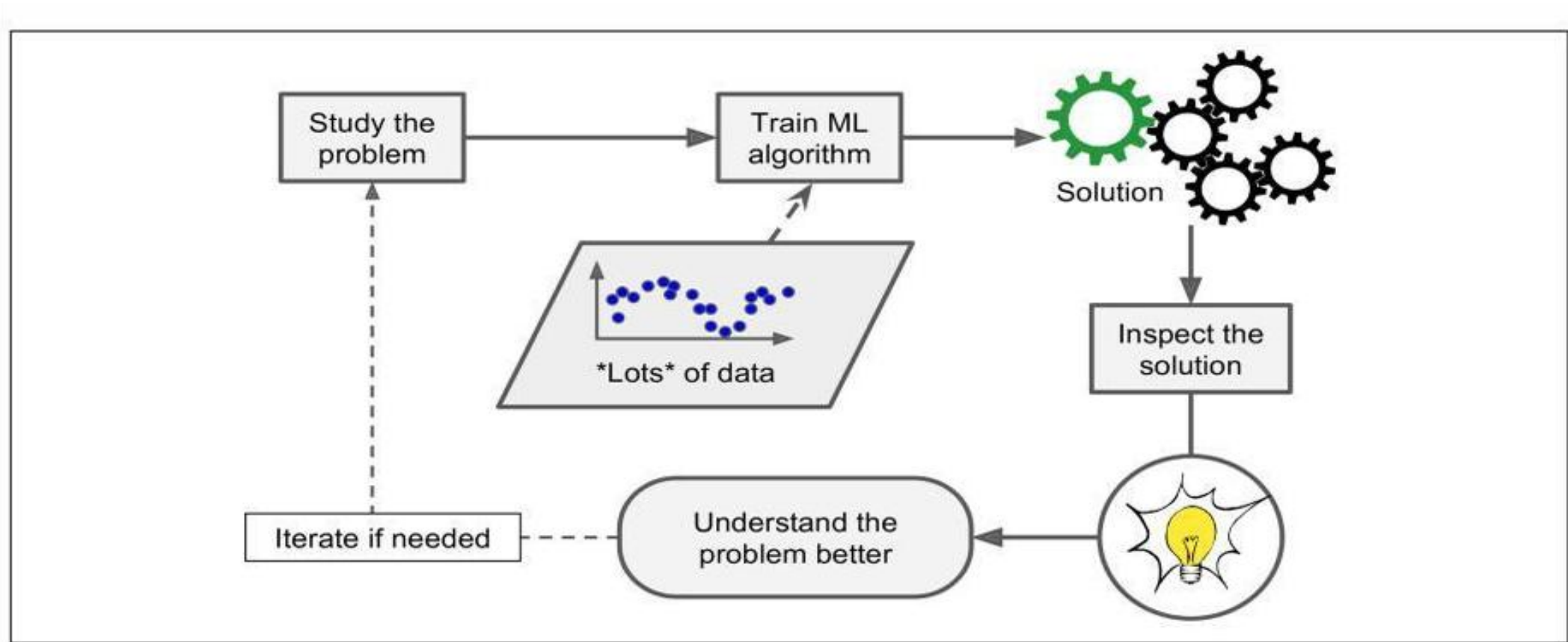


The traditional approach. If the problem is not trivial, your program will likely become a long list of complex rules pretty hard to maintain.



Machine Learning approach. The program is much shorter, easier to maintain, and most likely more accurate.

Why Machine Learning?

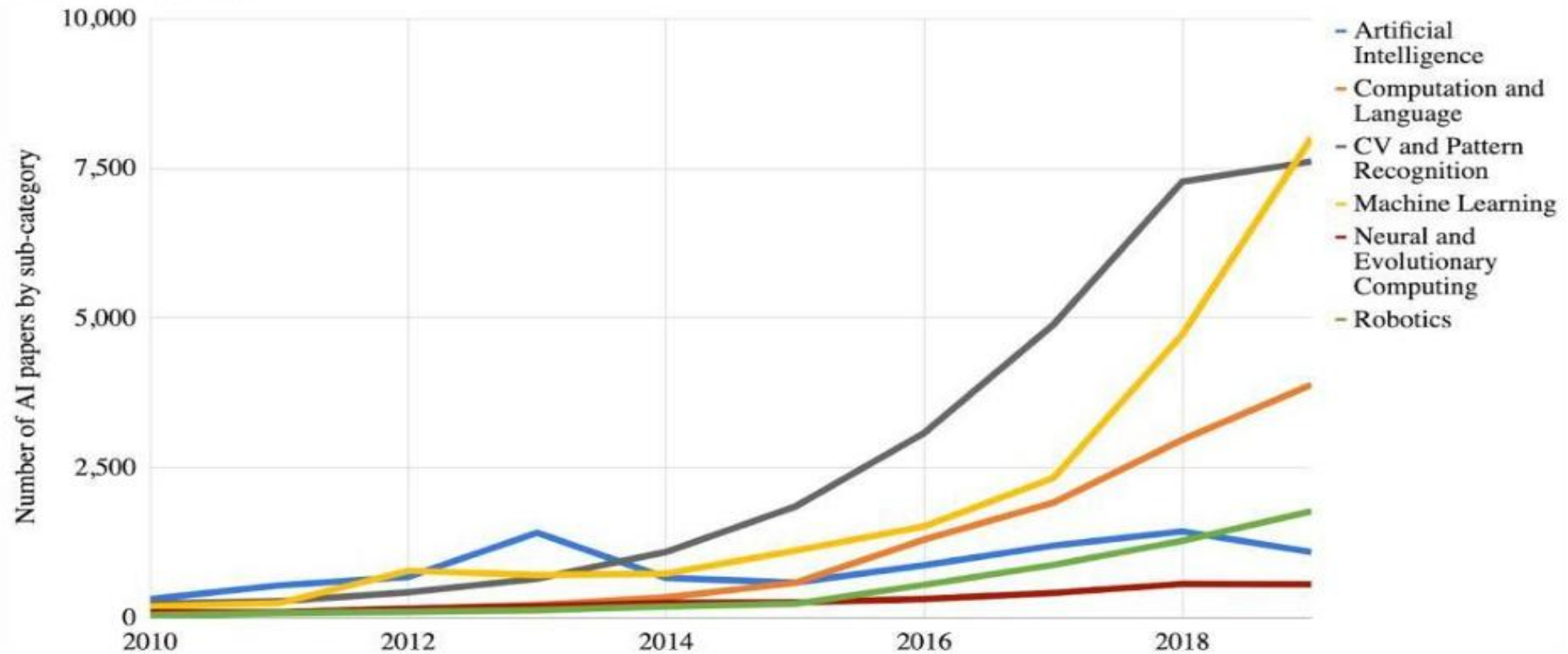


Machine Learning can help humans learn.

Why use Machine Learning?

Number of AI papers on arXiv, 2010-2019

Source: arXiv, 2019.



AI Index 2019 Annual Report.

Machine Learning Vocabulary

1) Examples

- Items or instances of data used for learning or evaluation. In spam detection problem, these examples correspond to the collection of email messages that will be used for learning and testing.

2) Training Sample

- Examples used to train a learning algorithm. In spam detection problem, the training sample consists of a set of email examples along with their associated labels.

3) Labels

- Values or categories assigned to examples. In classification problems, examples are assigned specific categories, for instance, the spam and non-spam categories in binary classification problem. In regression, items are assigned real-valued labels.

Machine Learning Vocabulary

4) Features

- The set of attributes, often represented as a vector, associated to an example. In the case of email messages, some relevant features may include the length of the message, the name of the sender, various characteristics of the header, the presence of certain keywords in the body of the message, and so on.

5) Test Sample

- Examples used to evaluate the performance of a learning algorithm. The test sample is separate from the training and validation data and is not made available in the learning stage. In the spam problem, the test sample consists of a collection of email examples for which the learning algorithm must predict labels based on features. These predictions are then compared with the labels of the test sample to measure the performance of the algorithm.

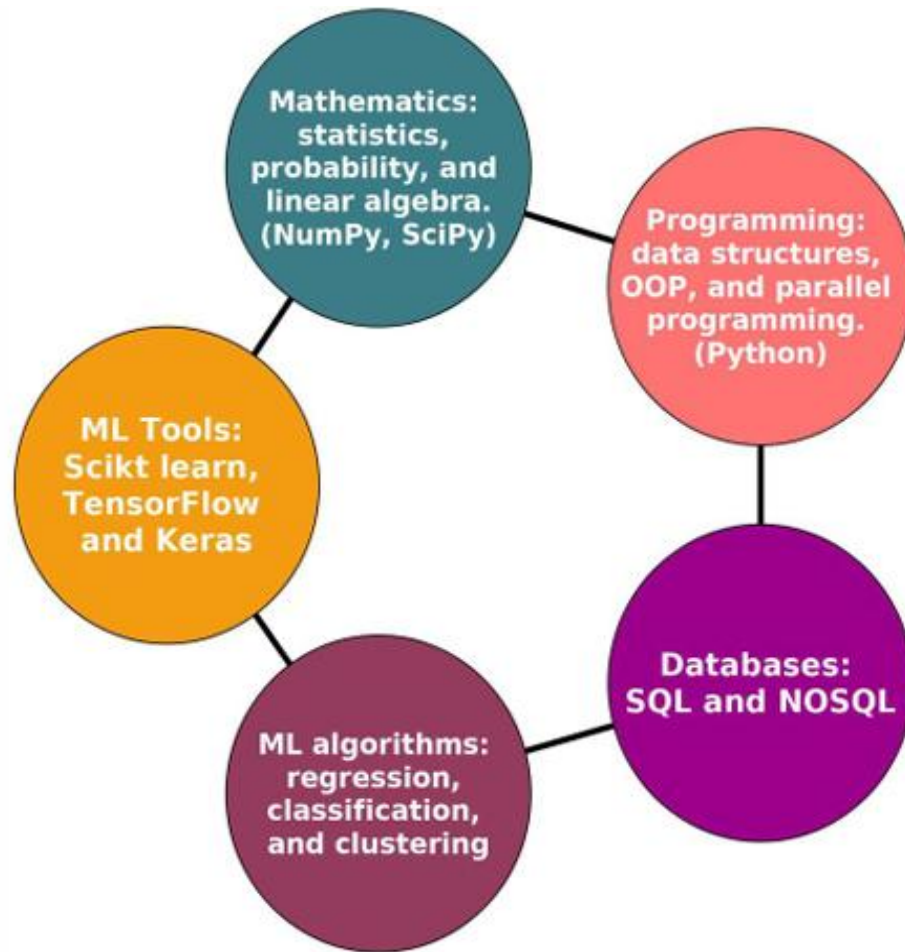
6) Loss Function

- A function that measures the difference, or loss, between a predicted label and a true label.

Applications of Machine Learning

- Machine learning is currently the preferred approach in the following domains:
 - 1) Speech Analysis: ex. speech recognition, synthesis.
 - 2) Computer vision: ex. object recognition/detection.
 - 3) Robotics: ex. position/map estimation.
 - 4) Bio-informatics: ex. sequence alignment, genetic analysis.
 - 5) E-commerce: ex. automatic trading, fraud detection.
 - 6) Financial analysis: ex. portfolio allocation, loan acceptance or rejection.
 - 7) Medicine: ex. diagnosis, therapy conception.
 - 8) Web: ex. Content management, social networks, etc.

How to get started with Machine Learning?



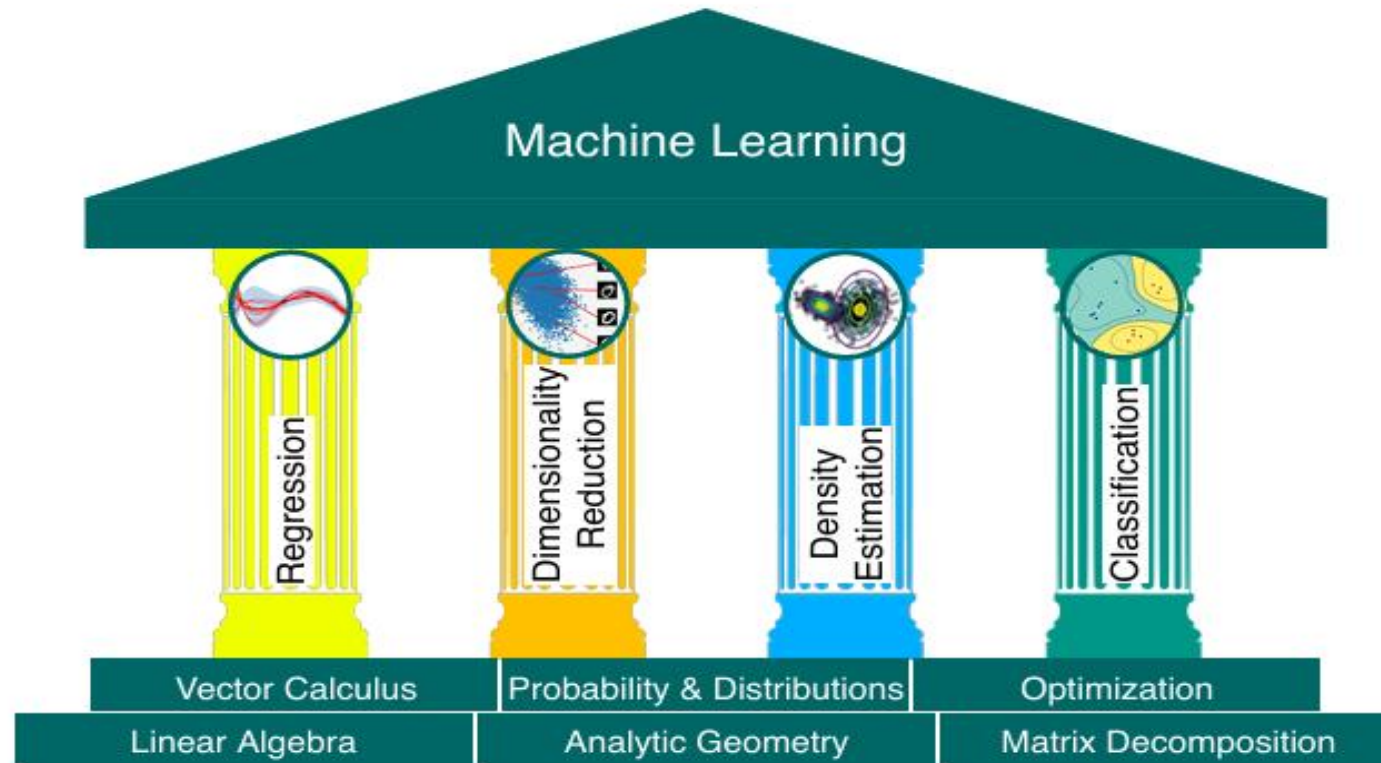
- 1) **Mathematics:** statistics, probability, and linear algebra.(NumPy, SciPy)
- 2) **Programming:** data structures, OOP, and parallel programming. (Python)
- 3) **Databases:** SQL and NOSQL.
- 4) **ML algorithms:** regression, classification, and clustering.
- 5) **ML Tools:** Scikit learn, TensorFlow and Keras.

4.1 Foundations and Four Pillars of Machine Learning

- There are three concepts that are at the core of machine learning: **data, a model, and learning**.
- Since machine learning is inherently data driven, **data** is at the core of machine learning & we represent data as vectors.
- The goal of machine learning is to design general purpose methodologies to extract valuable patterns from data, ideally without much domain-specific expertise.
- For example, given a large corpus of documents (e.g., books in many libraries), machine learning methods can be used to automatically find relevant topics that are shared across documents.
- To achieve this goal, models that are typically related to the process that generates data are designed, similar to model the dataset we are given. For example, in a regression setting, the model would describe a function that maps inputs to real-valued outputs.
- A **model** is said to learn from data if its performance on a given task improves after the data is taken into account.
- The goal is to find good models that generalize well to yet unseen data, which we may care about in the future.
- **Learning** can be understood as a way to automatically find patterns and structure in data by optimizing the parameters of the model.

4.1 Foundations and Four Pillars of Machine Learning

- The foundation and four pillars of Machine Learning is illustrated as:



4.1 Foundations and Four Pillars of Machine Learning

- Central to the core of machine learning: data, a model, and learning, we have four pillars of Machine Learning as:
 - Linear Regression
 - Dimensionality Reduction
 - Density Estimation
 - Classification
- However, to understand the 4 pillars we require a solid mathematical foundations and they are:
 - Linear Algebra
 - Analytic Geometry
 - Matrix Decomposition
 - Probability Theory
 - Vector Calculus
 - Optimization

4.1 Foundations and Four Pillars of Machine Learning

Linear Algebra

- Linear Algebra is the branch of mathematics dealing with vectors, matrices, systems of linear equations, and linear transformations.
- It is the most important mathematical foundation of machine learning because in Machine Learning, we represent numerical data as vectors and represent a table of such data as a matrix.
- Example if we consider a color image. In Machine Learning it is actually a huge matrix. A grayscale image is represented as:

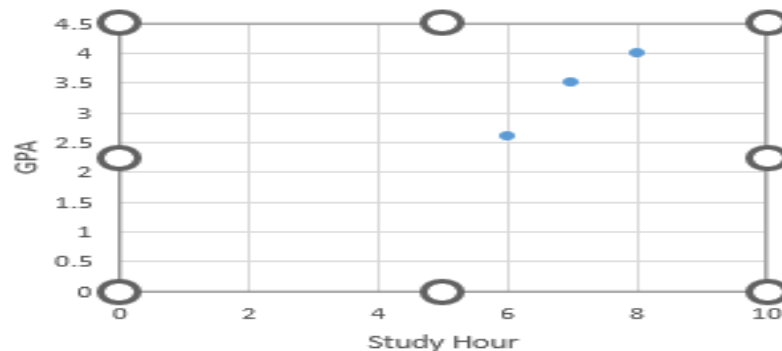
$$\begin{bmatrix} 105 & 125 & 145 \\ 70 & 100 & 130 \\ 80 & 105 & 125 \end{bmatrix}$$

- Every image recognition model works with matrices.
- Linear Algebra is used in data representation, neural networks, image processing, natural language processing, linear regression and logistic regression

4.1 Foundations and Four Pillars of Machine Learning

Analytic Geometry

- Analytic Geometry is the study of points, lines, planes, distances, and angles using coordinates. Machine learning often treats every data sample as a point in space.
- Example if each student has two features study hours and GPA then every student becomes a point. Machine learning tries to separate these points.
- Many algorithms classify data by drawing lines or planes. Example: Spam or not Spam. That separating line is called a decision boundary.



4.1 Foundations and Four Pillars of Machine Learning

- The idea is to determine the similarity between two vectors representing real-world items.
- Our Machine Learning method predicts that, comparable vectors will provide similar outcomes.
- To define similarity between vectors, we need to create operations that accept two vectors as input and output a number value indicating their similarity.
- Analytic geometry relies heavily on the construction of similarity and distance.
- Some methods used to calculate similarity or distance are Cosine Similarity, Euclidean Distance, Manhattan Distance, etc.
- Analytic geometry is used in Support Vector Machines (SVM), Logistic Regression, K-Means and K-Nearest Neighbor algorithm.

4.1 Foundations and Four Pillars of Machine Learning

Matrix decomposition

- Matrix decomposition is the process of breaking a large matrix into two or more smaller matrices whose multiplication reconstructs the original matrix.
- Large datasets contain redundant information, noise and unnecessary dimensions and matrix decomposition helps remove them.
- Used in Principal Component Analysis (PCA) where if you have 100 features and many are highly correlated, PCA reduces 100 features to 20 important features such that the training becomes faster, more accurate and uses less memory.

4.1 Foundations and Four Pillars of Machine Learning

Probability

- We frequently view data as noisy observations of a real underlying signal.
- We hope that using machine learning, we can separate the signal from the noise.
- We need a vocabulary to quantify what "noise" implies.
- Predictors should include the ability to represent uncertainty, such as quantifying our confidence in a forecast at a certain test data point.
- Probability theory focuses on quantifying uncertainty.
- Probability allows machine learning models to predict uncertain events, estimate confidence and make decisions with incomplete information

4.1 Foundations and Four Pillars of Machine Learning

Vector Calculus

- Vector Calculus studies derivatives, gradients, partial derivatives and multivariable functions.
- Machine learning models have many variables, sometimes millions.
- To train machine learning models, we typically find parameters that maximize some performance measure.
- Many optimization techniques require the concept of a gradient, which tells us the direction in which to search for a solution. This is the core about vector calculus.
- Suppose the function be $y=f(w, b, x)$ which represent a neural network with 3 parameters (weight, bias and input). To improve the model, we can calculate $\frac{\partial Loss}{\partial w}$. This tells us how much changing the weight changes the error i.e. vector calculus tells the model "Which direction should the weights move to reduce error?"
- Used in Backpropagation, Deep Learning, CNN, RNN and Transformers.

4.1 Foundations and Four Pillars of Machine Learning

Optimization

- Optimization means finding the best possible solution. In ML, that means finding parameters that minimize prediction error.
- Optimization concept is used to find maxima/minima of functions.
- Example: If model has loss function as $\text{Loss} = (\text{Actual} - \text{Predicted})^2$, then optimization minimizes this loss.
- Optimization answers “How can the model adjust its parameters to achieve the lowest possible error?”
- Without optimization, models cannot improve during training.
- Common Optimization Algorithms used are Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Momentum, Adam

4.1 Foundations and Four Pillars of Machine Learning

- Almost every machine learning algorithm is built around four fundamental learning problems:
 - Linear Regression
 - Classification
 - Density Estimation
 - Dimensionality Reduction
- These are commonly referred to as the four pillars of machine learning because they represent the four fundamental tasks that a machine learning system can perform.
- Almost every modern algorithm including deep learning models can be viewed as solving one or more of these tasks.

4.1 Foundations and Four Pillars of Machine Learning

- Every machine learning algorithm is designed to answer one of the following questions:

Question	Pillar
Can we predict a continuous value?	Linear Regression
Can we assign a category?	Classification
Can we discover hidden structure?	Density Estimation
Can we simplify complex data?	Dimensionality Reduction

4.1 Foundations and Four Pillars of Machine Learning

Linear Regression

- The objective of linear regression is to find functions that map inputs to corresponding observed function values, which we can interpret as the labels of their respective inputs.
- Linear Regression is used when we want to predict a continuous numerical value. Examples House price, Temperature, Salary, Stock price and Student marks. The output is always a number.
- The regression model is $y = wx + b$, where: x = input feature, w = slope (weight), b = intercept (bias) and y = predicted output
- Machine learning learns the values of w and b from the training data.
- Regression answers the question: "How much?". Whenever we predict a numerical quantity, we are solving a regression problem.

4.1 Foundations and Four Pillars of Machine Learning

Classification

- Classification predicts a category or class rather than a number.
- The objective here is to map input x to corresponding labels y .
- Examples:

Email → Spam or Not Spam

Medical diagnosis → Disease or Healthy

Loan Approval → Approved or Rejected.

- Classification answers: "Which category does this belong to?"

4.1 Foundations and Four Pillars of Machine Learning

Density Estimation

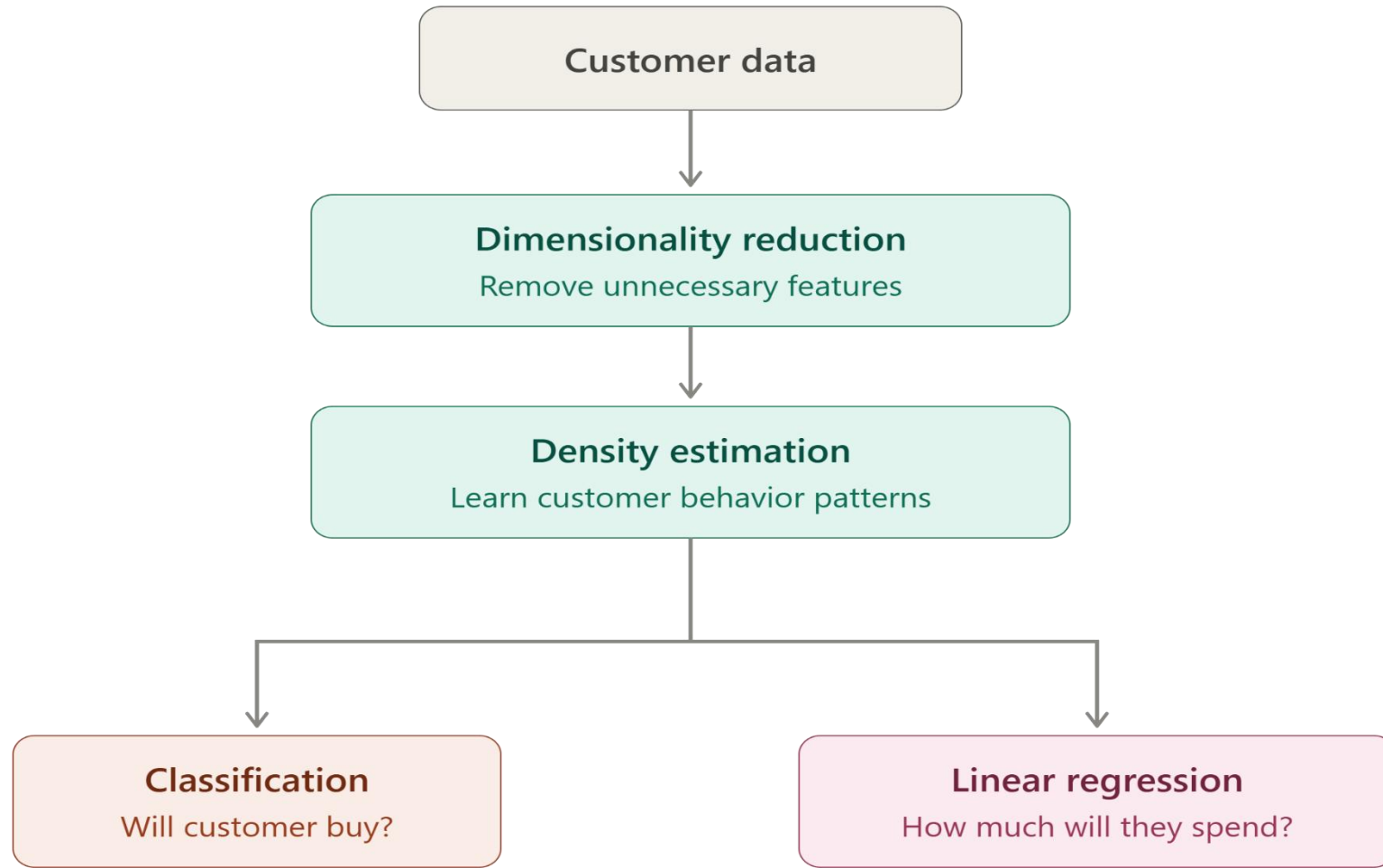
- Density Estimation tries to learn how the data is distributed.
- Instead of predicting labels, it estimates the probability of observing different data points.
- The objective of density estimation is to find a probability distribution that describes a given dataset.
- Example: If we record the customer ages as 20, 22, 23, 24, 25, 24, 23, 22 instead of predicting a class, the algorithm estimates the probability density function that answers questions like: Where are most data points concentrated? , Which observations are rare?, Which points are unusual (anomalies)?
- So, if a new customer has Age = 85, then density estimation identifies that 85 is highly unlikely compared with the observed distribution.
- Used in Anomaly Detection, Fraud Detection & Network Intrusion Detection.
- Density estimation answers: "How is the data distributed?". It allows machines to understand the underlying structure of the data rather than just assigning labels.

4.1 Foundations and Four Pillars of Machine Learning

Dimensionality Reduction

- Real-world datasets often have hundreds or even thousands of features. Many of them contain redundant or irrelevant information.
- Dimensionality Reduction compresses the data while preserving as much useful information as possible.
- The key objective of dimensionality reduction is to find a compact, lower-dimensional representation of high-dimensional data, which is often easier to analyze than the original data.
- High-dimensional data often suffers from the curse of dimensionality, i.e. high dimension means more computation, slower training, increased risk of overfitting and harder visualization. Reducing dimensions can make models faster, simpler, and sometimes more accurate.
- Dimensionality Reduction answers: "Can we represent the data more simply without losing much information?"
- Used in image compression, face recognition, data visualization, noise reduction and feature extraction.

4.1 Foundations and Four Pillars of Machine Learning



4.2 Review of Mathematics for Machine Learning

- Machine Learning is fundamentally about representing data and discovering patterns.
- Nearly every machine learning algorithm relies on mathematical concepts.
- Mathematics helps us to represent data, measure similarity, transform data, optimize models and understand learning algorithms.
- Machine Learning relies heavily on Linear Algebra, Calculus, Probability & Statistics and Optimization.
- Focus on this course is on Linear Algebra.

4.2 Review of Mathematics for Machine Learning

Linear Algebra

- Linear algebra is the formal study of vectors and the algebraic rules used to manipulate them.
- This field provides the foundational framework for many machine learning concepts, including dimensionality reduction and regression.
- Linear Algebra is a branch of mathematics that deals with: Vectors, Matrices, Linear equations & Transformations .
- It provides mathematical tools for representing and solving problems involving multiple variables and dimensions.

4.2 Review of Mathematics for Machine Learning

- The motivation behind using linear algebra for machine learning can be explained through an example. Suppose we want to predict house prices. Each house can be represented by several features:

Feature	Value
Area	1500 Sq ft
Bedrooms	3
Bathrooms	2
Age	5 years

- Instead of writing them separately, we represent them as:

$$x = \begin{bmatrix} 1500 \\ 3 \\ 2 \\ 5 \end{bmatrix}$$

- This column of numbers is called a vector. Machine learning algorithms process millions of such vectors.

4.2 Review of Mathematics for Machine Learning

- Linear Algebra is one of the most important mathematical foundations of Machine Learning (ML) because ML algorithms work with:
 - Vectors
 - Matrices
 - Transformations
 - High-dimensional data
- Concepts used in ML associated with Linear Algebra are:
 - Vector Space
 - Linear Combinations
 - Basis

4.2 Review of Mathematics for Machine Learning

Vector

- A vector is an ordered collection of numbers.
- In general, vectors are special mathematical objects that can be added together and multiplied by scalars to produce another object of the same kind.
- While often intuitively understood as "geometric vectors" (directed segments with magnitude and direction), the concept is much broader.
- Examples of vectors include:
 - Geometric Vectors: Directed segments used in physics and high school math.
 - Polynomials: Abstract objects that can be added and scaled, resulting in new polynomials.
 - Audio Signals: Series of numbers that can be added or scaled to create new signals.
 - Elements of R^n : Tuples of n real numbers, which are the primary focus of most linear algebra algorithms

4.2 Review of Mathematics for Machine Learning

- Example: $x = \begin{bmatrix} 1 \\ 5 \end{bmatrix}$ and $y = \begin{bmatrix} 4 \\ 1 \end{bmatrix}$
- Each element is called a component.
- Geometric Interpretation: In 2D, $x = (1,5)$ represents a point or an arrow from the origin.
- Applications in Machine Learning:
 - Vectors represent images, documents, audio signals, sensor readings, medical records, customer information.

4.2 Review of Mathematics for Machine Learning

Vector Operations:

A. Vector Addition

- Add corresponding vector components.
- Example: $a = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$ and $b = \begin{bmatrix} 5 \\ 4 \end{bmatrix}$ then $a + b = \begin{bmatrix} 7 \\ 7 \end{bmatrix}$

B. Scalar Multiplication

- Multiply every component by a number.
- Example: $3 \begin{bmatrix} 2 \\ 5 \end{bmatrix} = \begin{bmatrix} 6 \\ 15 \end{bmatrix}$.
- The scalar stretches or shrinks the vector.

4.2 Review of Mathematics for Machine Learning

- Example: If we want to build a machine learning model that predicts student performance. We collect the following information.

Student	Study Hours	Attendance (%)	GPA
A	2	80	3.0
B	4	90	3.5
C	6	95	3.8

- A machine learning model converts every student into a vector.

Student A

$$x_1 = \begin{bmatrix} 2 \\ 80 \\ 3.0 \end{bmatrix}$$

Student B

$$x_2 = \begin{bmatrix} 4 \\ 90 \\ 3.5 \end{bmatrix}$$

Student C

$$x_3 = \begin{bmatrix} 6 \\ 95 \\ 3.8 \end{bmatrix}$$

These vectors live inside a vector space.

4.2 Review of Mathematics for Machine Learning

Vector Space

- A vector space is a structured space in which vectors live, defined by a set V and two operations: vector addition (+) and multiplication by scalars.
- A vector space is a collection of vectors where vectors can be added and can be multiplied by scalars and still remain inside the same space.
- Example: 2D vectors: $(1, 2)$, $(3, 4)$ belong to a vector space because:

Addition = $(1, 2) + (3, 4) = (4, 6)$ & Scalar multiplication = $2 (1, 2) = (2, 4)$ is possible.

4.2 Review of Mathematics for Machine Learning

- Examples 1: All 2-dimensional vectors $\mathbb{R}^2$ can be represented as $(1, 2)$, $(3, 5)$, $(0, 0)$. All belong to the same vector space.
- Example 2: Three-dimensional vectors $\mathbb{R}^3$ can be $(4, 2, 8)$.
- In Machine Learning, each data point can be represented as a vector.
- Example: A student record: $[\text{Age}, \text{Marks}, \text{Attendance}] = [20, 85, 90]$. This is a vector in a 3-dimensional vector space.

4.2 Review of Mathematics for Machine Learning

- For previous example of Students performance, vector space can be visualized as:

Student A:

$$A = (2, 80, 3.0)$$

Student B:

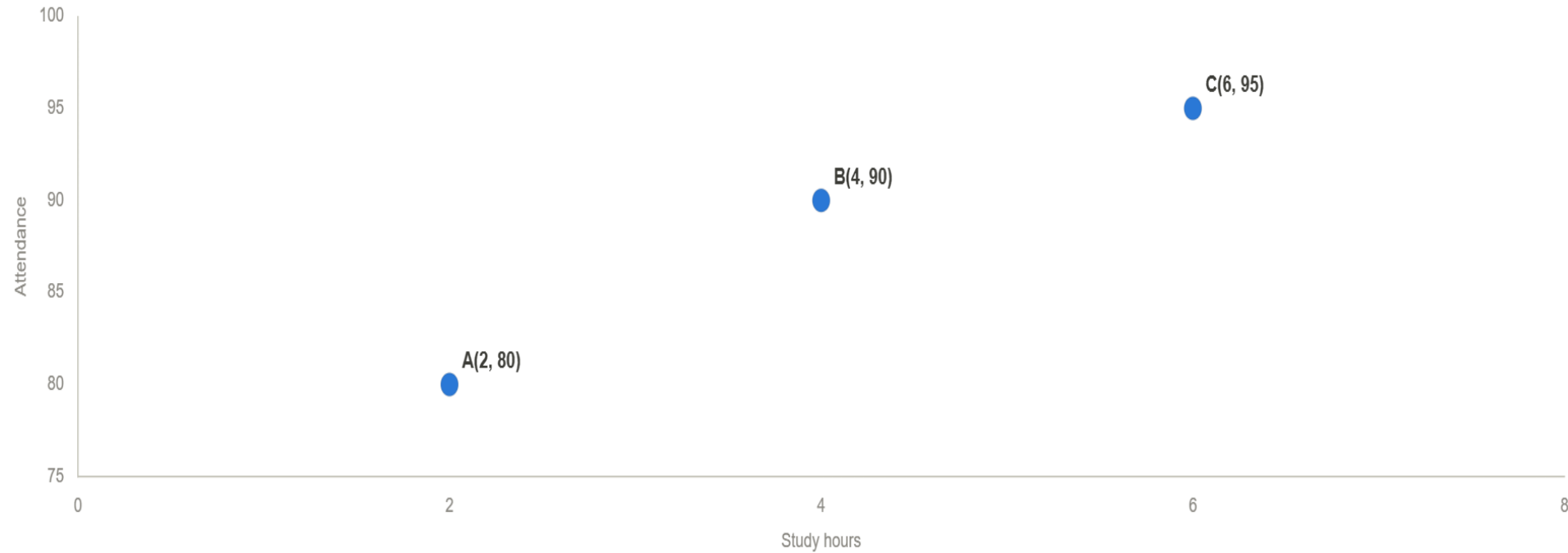
$$B = (4, 90, 3.5)$$

Student C:

$$C = (6, 95, 3.8)$$

- These vectors belong to the three-dimensional vector space $\mathbb{R}^3$.

4.2 Review of Mathematics for Machine Learning



- Every point is a vector.
- This entire coordinate system is the vector space.
- Now if a new student with Study Hours = 5, Attendance = 92 and GPA = 3.65 is added then the machine stores (5,92,3.65) as another point inside the same vector space.

4.2 Review of Mathematics for Machine Learning

Linear Combination

- A linear combination is formed by multiplying vectors by scalars and adding them.
- Mathematically $a_1v_1 + a_2v_2 + \dots + a_nv_n$, where: a_i = scalars , v_i = vectors
- Example: Given $v_1 = (1, 2)$, $v_2 = (3, 1)$. A linear combination $2v_1 + 3v_2$ is calculated as: $2v_1 = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$ and $3v_2 = \begin{bmatrix} 9 \\ 3 \end{bmatrix}$ that gives $= \begin{bmatrix} 11 \\ 7 \end{bmatrix}$.
- Another example can be of arrows. A linear combination mixes different arrows together to create a new arrow. Just like mixing colors.
- Different vectors combine to create many new vectors.

4.2 Review of Mathematics for Machine Learning

- In Machine Learning, almost every machine learning model is a weighted combination of inputs.
- Such as Neural networks, linear regression and feature transformations.
- Example: Linear Regression:

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$

- Example: Prediction equation used by neural network:

$$y = w_1x_1 + w_2x_2 + b$$

4.2 Review of Mathematics for Machine Learning

- Example in Neural Network if Feature 1 = Study Hours, Feature 2 = Attendance and Feature 3 = GPA the Weight vector, $w = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$ and Student vector, $x = \begin{bmatrix} 5 \\ 80 \end{bmatrix}$.
- Prediction is made using a linear combination:
$$\begin{aligned} y &= 4(5) + 2(80) \\ &= 20 + 160 \\ &= 180 \end{aligned}$$

4.2 Review of Mathematics for Machine Learning

Span of Vectors

- The span is the set of all possible linear combinations of a group of vectors.
- Let $v_1 = (1,0)$ and $v_2 = (0,1)$ then every point in the plane can be written as: $av_1 + bv_2$. Hence, they span the entire 2D plane.
- Example: Can $(5, 7)$ be represented?
$$5(1,0) + 7(0,1) = (5,7)$$

4.2 Review of Mathematics for Machine Learning

Linear Independence

- Suppose we have vectors v_1 and v_2 . They are linearly independent if no vector can be written using the other.
- Example: $(1, 0)$ and $(0, 1)$ are linearly independent as neither depends on the other. While $(2, 4)$ and $(1, 2)$ are dependent because $(2, 4) = 2(1, 2)$. One vector is redundant.
- Why is Linear Independence Important?
 - Machine learning avoids redundant features.
 - Suppose: Height (cm) and Height (m). These provide the same information. Keeping both increases computational cost without adding new information.

4.2 Review of Mathematics for Machine Learning

- Example 1: Let the student data contains given dataset.

Student	Height (cm)	Height (m)	Weight (kg)
1	170	1.70	65
2	160	1.60	60
3	180	1.80	75
4	175	1.75	80

Vectors of given data are:

$$H_{cm} = \begin{bmatrix} 170 \\ 160 \\ 180 \\ 175 \end{bmatrix}$$

$$H_m = \begin{bmatrix} 1.70 \\ 1.60 \\ 1.80 \\ 1.75 \end{bmatrix}$$

Here,

$$H_{cm} = 100H_m$$

- Therefore, these two vectors are linearly dependent.
- Both describe exactly the same information in different units.
- A machine learning model does not gain additional knowledge by keeping both.
- Hence, only two features height(cm) and weight(kg) is used as the feature.

4.2 Review of Mathematics for Machine Learning

- Example 2: Let we are building a machine learning model to predict house prices. For four houses, we collect three features as shown in the table. Now find out whether the given features are linearly independent or not?

House	Feature 1 (Bedrooms)	Feature 2 (Rooms)	Feature 3 (Master Bedrooms)
1	1	2	2
2	2	6	4
3	3	5	6
4	4	4	8

4.2 Review of Mathematics for Machine Learning

- Solution,
- Represent each feature as a vector.
- $F_1 = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}, F_2 = \begin{bmatrix} 2 \\ 3 \\ 5 \\ 6 \end{bmatrix}, F_3 = \begin{bmatrix} 2 \\ 4 \\ 6 \\ 8 \end{bmatrix}$
- Here, $F_3 = 2F_1$ because $2 \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 6 \\ 8 \end{bmatrix}$
- Therefore, F_1 & F_3 are linearly dependent. Feature 3 contains no new information. If we already know Bedrooms = 3 then Master Bedrooms = 6
- is automatically known. So, Feature 3 is redundant.
- Hence, instead of using Bedrooms , Rooms and Master Bedrooms the model only needs
Bedrooms and Rooms as feature for predicting the house price.

4.2 Review of Mathematics for Machine Learning

Basis

- A basis is the smallest set of linearly independent vectors that can generate every vector in the space.
- Can be thought of as the "building blocks" of a vector space.
- Standard basis of $\mathbb{R}^2$ or in 2D is $e_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $e_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. Every vector (x, y) can be written as: $xe_1 + ye_2$. Example: $(3, 5) = 3(1, 0) + 5(0, 1)$.
- Basis in 3D: Standard basis $(1,0,0)$, $(0,1,0)$ and $(0,0,1)$. Any 3D vector can be represented using these three vectors.

4.2 Review of Mathematics for Machine Learning

- Example: $b_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $b_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Can we represent $x = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$ using the linear combiner relation $x = ab_1 + cb_2$.

- Solution,

$$a \begin{bmatrix} 1 \\ 1 \end{bmatrix} + c \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$$

$$a + c = 8 \quad \text{.....(i)}$$

$$a - c = 2 \quad \text{.....(ii)}$$

- Solve the equations we get $2a = 10$ & $a = 5$. Now, substituting value of a we get; $5 + c = 8$. Hence, $c = 3$.
- Verification:
- $5 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + 3 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix} + \begin{bmatrix} 3 \\ -3 \end{bmatrix} = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$
- Thus, the vector (8,2) is expressed as a linear combination of the basis vectors.

4.2 Review of Mathematics for Machine Learning

- Example: In Machine Learning, basis vectors help:
 - Represent data efficiently
 - Reduce dimensions
 - Extract important features
- Used in:
 - PCA (Principal Component Analysis)

4.2 Review of Mathematics for Machine Learning

Dimension

- The dimension of a vector space equals the number of vectors in its basis.

- Examples:

Space	Basis	Dimension
$\mathbb{R}^2$	2 Vectors	2
$\mathbb{R}^3$	3 Vectors	3
$\mathbb{R}^{100}$	100 Vectors	100

- Significance in Machine Learning
 - A dataset with Height, Weight, Age and Salary can be viewed as points in a 4-dimensional space. Each feature corresponds to one dimension.
 - Higher-dimensional data often require techniques such as dimensionality reduction to simplify analysis while preserving important information.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Optimization

- Optimization is the process of finding the best possible solution among all feasible solutions.
- Example: Google Maps finds the shortest route because the objective is to minimize the travel distance; in online shopping a customer wants to buy the cheapest laptop with good specifications to minimize cost while maximizing quality.
- In Artificial Intelligence, the "best" solution usually means solution that has:
 - minimum error
 - maximum accuracy
 - minimum cost
 - maximum reward
- Mathematical Representation: Let $f(x)$ is an objective function, then optimization means either $\min f(x)$ or $\max f(x)$ where, x = parameters & $f(x)$ = objective function.
- Every machine learning algorithm learns by optimizing a loss function. Without optimization, a machine learning model cannot improve its predictions.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Components of an Optimization Problem:

1. Objective Function

- The mathematical function to minimize or maximize.
- Example: $f(x) = x^2 + 2x + 1$

2. Decision Variables

- Unknown quantities to determine.
- Example: x, w, b

3. Constraints (optional)

- Restrictions on variables.
- Example: $x \geq 0$

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- In optimization, the most popular method used is gradient that tells how a function changes with respect to its variables.
- It points in the direction of the steepest increase of the function, while the negative gradient points in the direction of the steepest decrease.
- Since optimization often aims to minimize a function (called the objective function or loss function), algorithms such as Gradient Descent move in the direction of the negative gradient.
- If we have a function of multiple variables, such as $f(x, y)$, its gradient is
$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Example: If we are standing on a mountain. The mountain represents the objective function. Our position represents the current values of the parameters. Our goal is to reach the lowest point (minimum). Then, the gradient acts like a compass:
 - It points uphill (direction of maximum increase).
 - The opposite direction points downhill, which is where optimization algorithms move.
- Why is Gradient Important?
- The gradient answers two important questions:
 1. Which direction should we move?
 - The gradient gives the direction in which the function increases the fastest.
 - For minimization, Move in direction $-\nabla f(x)$.
 2. How steep is the function?
 - The magnitude (length) of the gradient tells us how steep the surface is.
 - Large gradient $\rightarrow$ Steep slope
 - Small gradient $\rightarrow$ Flat region
 - Zero gradient $\rightarrow$ Possible optimum

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Example 1: Single Variable Function

- Consider $f(x) = x^2$
- Derivative: $\frac{df}{dx} = 2x$
- Here the gradient is simply: $\nabla f = 2x$
- At $x = 3$, Function value: $f(3) = 9$ & Gradient: $\nabla f = 2(3) = 6$ which means function is increasing. To minimize it, move left.
- Suppose learning rate: $\alpha = 0.1$
- Gradient Descent update: $x_{new} = x - \alpha \nabla f$
 $= 3 - 0.1(6) = 2.4$
- The function value becomes: $f(2.4) = 5.76$ & Gradient: $\nabla f = 2(2.4) = 4.8$
- Therefore, the loss is decreased.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Example 2: Two Variables

- Suppose $f(x, y) = x^2 + y^2$ then Partial derivatives are $\frac{\partial f}{\partial x} = 2x$ & $\frac{\partial f}{\partial y} = 2y$ & Gradient becomes $\nabla f = \begin{bmatrix} 2x \\ 2y \end{bmatrix}$.
- Let $x = 2, y = 3$ then Gradient $\nabla f = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$.
- Here, increase in x increases the function by about 4 units per unit change while increase in y increases the function by about 6 units per unit change. Overall steepest ascent direction is (4, 6).
- Therefore, the steepest descent direction is (-4, -6).

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Gradient in Machine Learning

- In machine learning, we want to minimize a loss function.
- For example, $L(w, b)$ where: w = weights & b = bias.
- The gradient is $\nabla L = \left[\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b} \right]$
- It tells us how much each weight contributes to the error, and how each weight should be adjusted to reduce that error.

Gradient Descent Update Rule:

- Every iteration updates the parameters as $\theta_{new} = \theta_{old} - \alpha \nabla J(\theta)$

where:

θ = model parameters

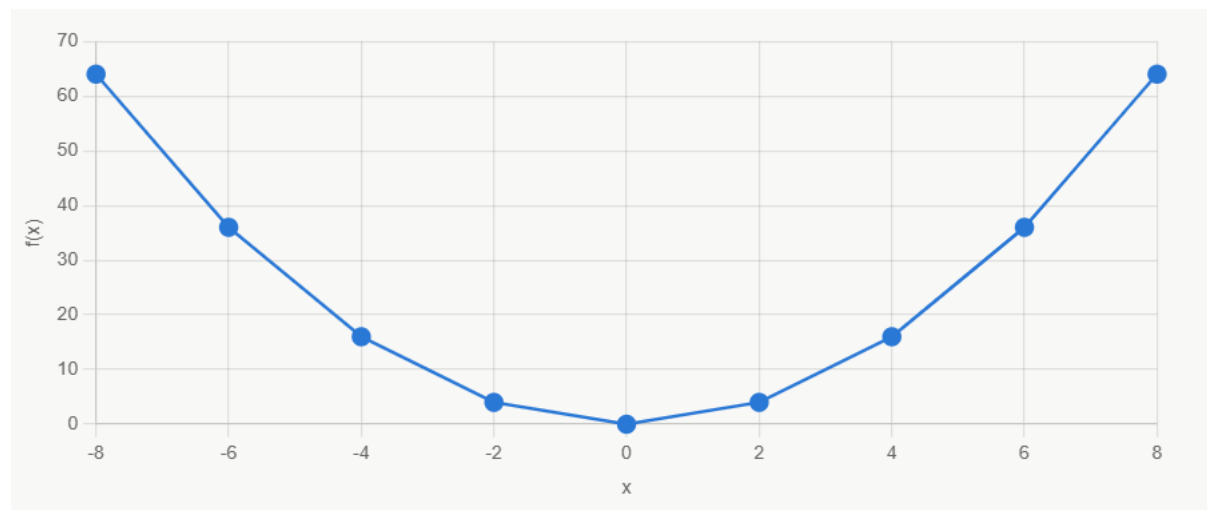
α = learning rate

$J(\theta)$ = objective (loss) function

$\nabla J(\theta)$ = gradient

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Consider the function $f(x) = x^2$.
- Its graph is a smooth bowl shape with a single minimum.
- A simple optimization algorithm can easily find the lowest point by moving downhill.

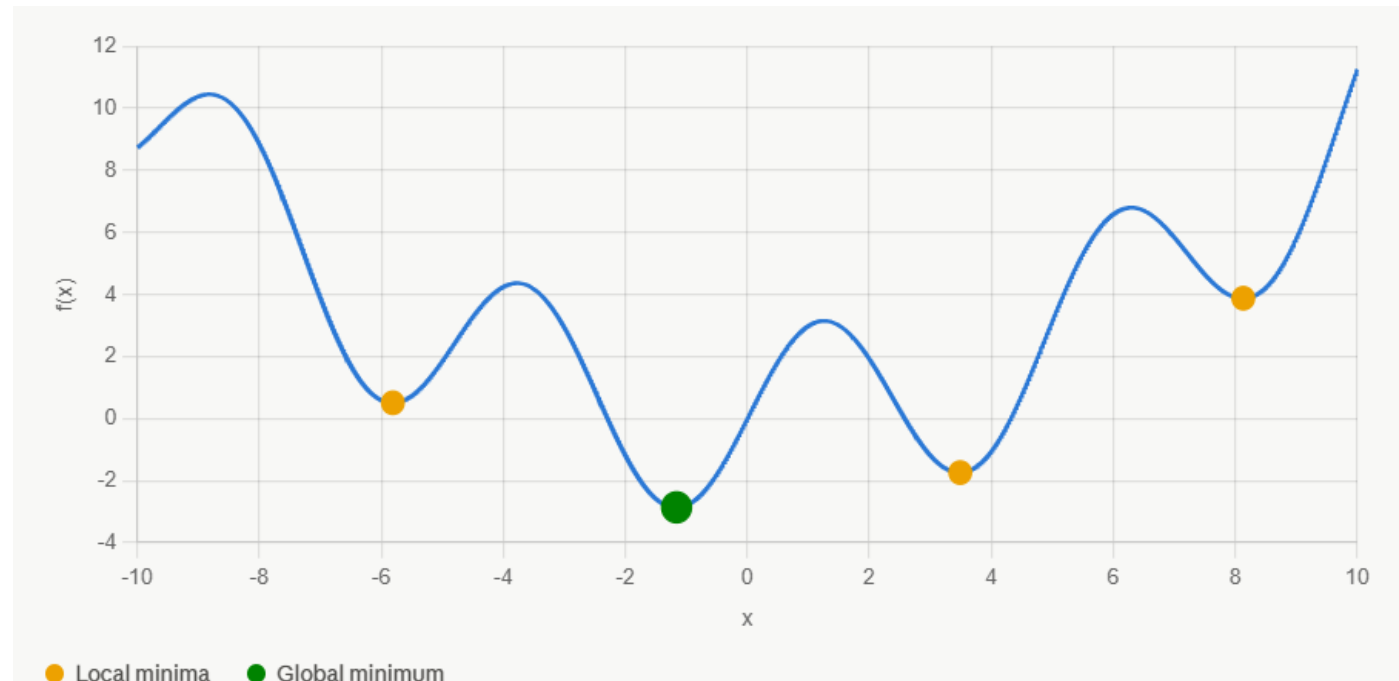


4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Simple optimization works well only for smooth, simple problems. However, real-world AI problems are much more complex. They may consist of:

1. Multiple Local Minima

- The objective function may have many valleys.
- A simple optimization method may stop at the first valley it reaches, even if it is not the lowest one.



4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

2. Saddle Points

- Some points have zero gradient but are neither minima nor maxima. Optimization algorithms may slow down or stop there

3. Flat Regions (Plateaus)

- When the gradient is close to zero over a large region, learning becomes very slow.

4. High-Dimensional Search Spaces

- Modern AI models may have millions or billions of parameters, making optimization much harder.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Continuation Optimization

- Continuation Optimization is a technique in which a difficult optimization problem is solved by starting with a simplified version and continuously transforming it into the original problem, using the solution from each stage as the starting point for the next.
- Continuation optimization follows the principle that says solve an easy problem first, then gradually increase its difficulty.
- It is also commonly known as Homotopy Continuation or Graduated Optimization in optimization and machine learning literature.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Mathematical Interpretation: Suppose the original optimization problem is $\min_x f(x)$ where $f(x)$ is difficult to optimize.
- Continuation optimization first creates a simpler function: $f_0(x)$. Then a continuation parameter is introduced as $0 \leq \lambda \leq 1$ and define a new objective function: $F(x, \lambda) = (1 - \lambda)f_0(x) + \lambda f(x)$.
- When $\lambda = 0$, the optimizer solves only the easy problem.
- When $\lambda = 0.5$, the objective is halfway between the easy and original problems.
- When $\lambda = 1$, the optimizer solves the original optimization problem.
- Thus, the optimization problem is continued from an easy version to the actual difficult version.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- For $F(x, \lambda) = (1 - \lambda)x^2 + \lambda(x^4 - 4x^2 + x)$, explain how the objective function changes as λ varies from 0 to 1.
- Solution,
- Let $x = 2$ and λ varies from 0 to 1.
- $F(2,0) = (1 - 0)2^2 + 0(2^4 - 4 * 2^2 + 2) = 2^2 = 4$
- $F(2,0.5) = (1 - 0.5)2^2 + 0.5(2^4 - 4 * 2^2 + 2) = 3$
- $F(2,1) = (1 - 1)2^2 + 1(2^4 - 4 * 2^2 + 2) = 2$

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- If we are climbing a mountain in thick fog:
- Gradient Descent only look at the ground around the feet and checks “Which nearby direction goes downhill?”. It keeps taking small downhill steps. If it enters a small valley, it may never discover the deeper valley elsewhere.
- While Continuation optimization first smooths landscape such that small bumps disappear. It then finds the bottom of this smooth landscape and slowly restores the bumps while staying near the valley. This gives a better chance of ending up in the global minimum.
- Continuation optimization usually uses gradient descent to solve each intermediate problem.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Unconstrained Optimization

- Unconstrained optimization is an optimization technique that only optimizes the objective function as there are no any restrictions/constraints imposed on the variables.

- General form: $\min f(x)$.

- Example 1: Find the minimum of $f(x) = x^2 - 6x + 13$.

- Solution,

Differentiate the given equation as: $f'(x) = 2x - 6$

Now, set the equation equal to zero: $2x - 6 = 0$

$$x = 3$$

Substitute,

$$f(3) = 4$$

- Therefore, minimum occurs at $x = 3$.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Example 2: Find the minimum of $f(x) = x^2 - 8x + 20$.

➤ Solution,

Differentiate the given equation as: $f'(x) = 2x - 8$

Now, set the equation equal to zero: $2x - 8 = 0$

$$x = 4$$

Substitute,

$$f(4) = 4$$

➤ Therefore, minimum occurs at $x = 4$.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- First-Order Optimality Condition

- A stationary point occurs when $\nabla f(x) = 0$ where; ∇ is called the gradient.

- $\nabla f(x)=0$ says at this point, the function is momentarily "flat" i.e. no direction leads immediately downhill or uphill.

- Since the gradient points in the direction of steepest increase, if it's zero, there's no "uphill" direction to point to. We are standing at a spot where the ground is level in every direction.

- Second-Order Condition

- Once we have found a flat point, the second derivative (curvature) tells us what kind of flat point it is:

- If $\nabla^2 f(x) > 0$, the point is a minimum i.e. the function curves upward around that point, like the bottom of a bowl that gives local minimum.

- If $\nabla^2 f(x) < 0$, the point is a maximum i.e. the function curves downward, like the top of a dome that gives local maximum.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Example 1: $f(x) = x^2$

Step 1: First-order condition

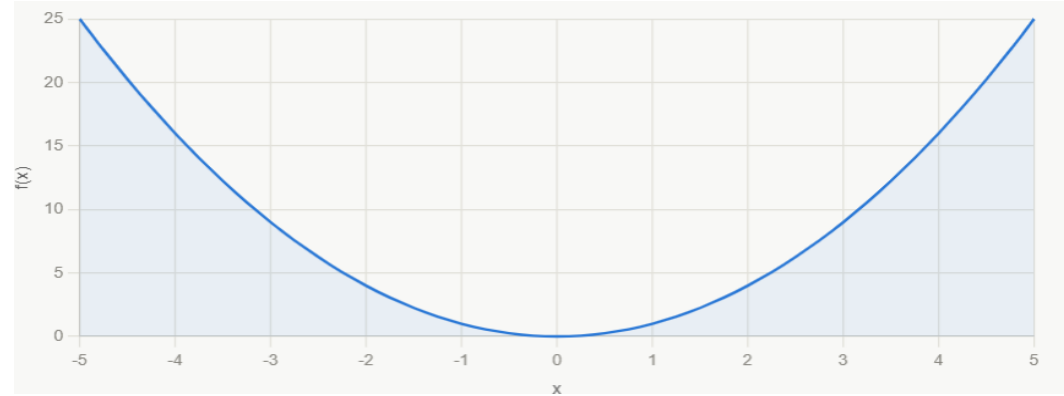
- $f'(x) = 2x$

or, $2x = 0 \implies x = 0$

- So, $x = 0$ is a stationary point which is a candidate for a min or max.

Step 2: Second-order condition

- $f''(x) = 2$
- Since $f''(x) = 2 > 0$ everywhere (including at $x = 0$), the curve bends upward there like the bottom of a parabola. So $x = 0$ is a local minimum.



4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Example 2: $f(x) = -x^2$

Step 1: First-order condition

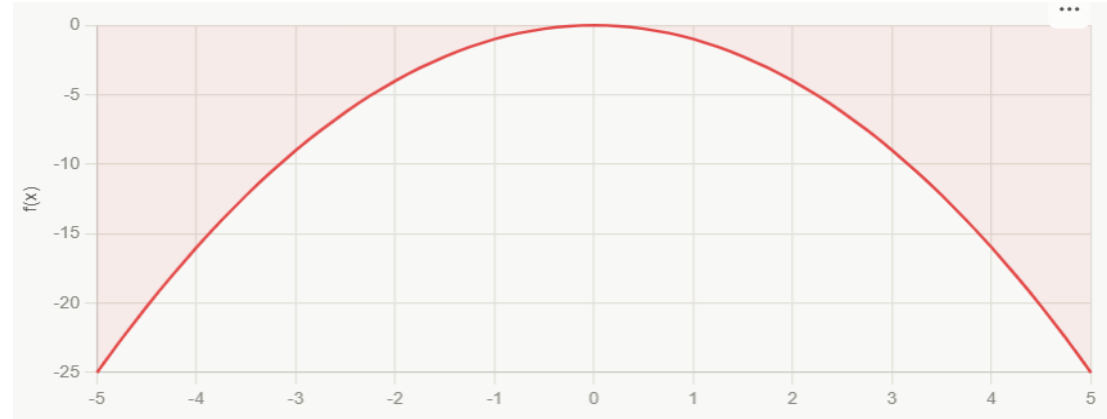
- $f'(x) = -2x$

or, $2x = 0 \implies x = 0$

- So, $x = 0$ is a stationary point which is a candidate for a min or max.

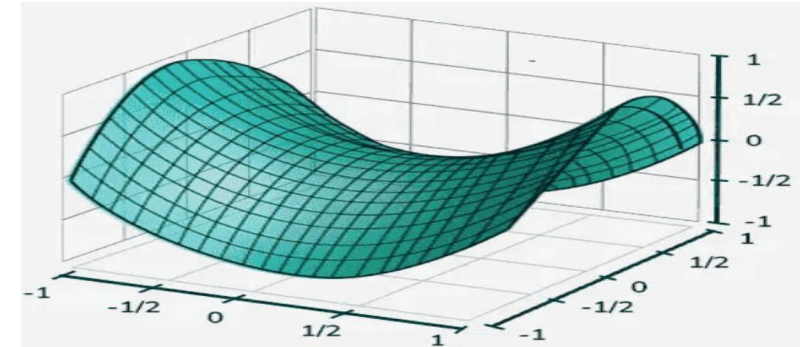
Step 2: Second-order condition

- $f''(x) = -2$
- Since $f''(x) = -2 < 0$ everywhere (including at $x = 0$), the curve bends downward. So, $x = 0$ is a local maximum.



4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Example 3: $f(x)=x^3$
- $f'(x) = 3x^2 = 0 \implies x = 0$
- $f''(x) = 6x$, and at $x = 0$: $f''(0) = 0$
- Here the second-order test is inconclusive and indeed, $x = 0$ is neither a min nor a max for x^3 ; it's an inflection point (a 1D analogue of a saddle point).
- This shows why the second-order condition is so useful, it distinguishes true minima/maxima from these ambiguous flat spots.



4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Review of Stochastic Gradient Descent (SGD)
- Why Gradient Descent?
 - Machine learning models contain many parameters.
 - Instead of solving equations analytically, we iteratively improve the parameters using gradients.
- Example: Linear Regression:

$$f(w) = \frac{1}{n} \sum (y - \hat{y})^2$$

Here the goal is to find w that minimizes $f(w)$.

Gradient Descent

- Update rule: $w_{new} = w_{old} - \alpha \nabla f(w)$ where; α = learning rate & $\nabla f(w)$ = gradient

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Types of Gradient Descent

1. Batch Gradient Descent

➤ Batch Gradient Descent uses the entire dataset to compute the gradient at every step.

2. Stochastic Gradient Descent (SGD)

➤ Uses only one training sample at a time for the update.

➤ instead of using all N examples, approximate the gradient using just *one* randomly chosen example i .

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Batch Gradient Descent

Example: Model is $\hat{y} = wx$ with dataset ($N = 3$ points).

x	y
1	1
2	2
3	4

Loss function (Mean Squared Error): $f(w) = \frac{1}{N} \sum_{i=1}^N (wx_i - y_i)^2$

Gradient (derivative of f w .r. t. w , using the chain rule): $\frac{df}{dw} = \frac{2}{N} \sum_{i=1}^N (wx_i - y_i) x_i$

Batch Gradient Descent update rule: $w_{i+1} = w_i - \alpha \frac{df}{dw} |_{w_i}$

We use learning rate $\alpha = 0.1$ and initialize $w_0 = 0$.

- Iteration 1:

x_i	y_i	$\hat{y}_i = w_0 x_i$	error = $\hat{y}_i - y_i$	error $\times x_i$
1	1	0	-1	-1
2	2	0	-2	-4
3	4	0	-4	-12

Gradient uses all three rows (that's the "batch" part):

$$\frac{df}{dw} = \frac{2}{3}(-1 - 4 - 12) = \frac{2}{3}(-17) = -11.33$$

Update:

$$w_1 = 0 - 0.1 \times (-11.33) = 1.133$$

Iteration 2 ($w_1 = 1.133$):

x_i	y_i	$\hat{y}_i = w_0 x_i$	error = $\hat{y}_i - y_i$	error $\times x_i$
1	1	1.133	0.133	0.133
2	2	2.267	0.267	0.533
3	4	3.40	-0.601	-1.800

$$\frac{df}{dw} = \frac{2}{3}(0.133 + 0.533 - 1.800) = \frac{2}{3}(-1.133) = -0.756$$

$$w_2 = 1.133 - 0.1 \times (-0.756) = 1.209$$

- Iteration 3 ($w_2 = 1.209$):

xi	y_i	$\hat{y}_i = w_0 x_i$	error = $\hat{y}_i - y_i$	error $\times x_i$
1	1	1.209	0.209	0.209
2	2	2.418	0.418	0.836
3	4	3.627	-0.373	-1.120

$$\frac{df}{dw} = \frac{2}{3}(0.209 + 0.836 - 1.120) = \frac{2}{3}(-0.076) = -0.050$$

$$w_3 = 1.209 - 0.1 \times (-0.050) = 1.214$$

Here, w is settling down: $0 \rightarrow 1.133 \rightarrow 1.209 \rightarrow 1.214$. The gradient magnitude is shrinking each step ($-11.33 \rightarrow -0.756 \rightarrow -0.050$) & is approaching a minimum. This signifies that the model is converging towards optimum

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Stochastic Gradient Descent (SGD)

Example: Model is $\hat{y} = wx$ with dataset ($N = 3$ points).

x	y
1	1
2	2
3	4

Per-example loss (squared error on just *one* point, no averaging over N):

$$\ell_i(w) = (wx_i - y_i)^2$$

Per-example gradient:

$$\frac{d\ell_i}{dw} = 2(wx_i - y_i) x_i$$

SGD update rule (using $\alpha = 0.1$):

$$w_{i+1} = w_i - \alpha \frac{d\ell_i}{dw}$$

- The key difference from Batch Gradient Descent is we update w after every single point, not after summing over all of them.
- One full pass through all N points is called an epoch.

Epoch 1 (starting from $w_0 = 0$)

Point used	x_i	y_i	$\hat{y}_i = wx_i$	error	gradient = $2(\text{error})x_i$	new w
(1, 1)	1	1	0.000	-1.000	-2.000	$0 - 0.1(-2.000) = 0.200$
(2, 2)	2	2	0.400	-1.600	-6.400	$0.200 - 0.1(-6.400) = 0.840$
(3, 4)	3	4	2.520	-1.480	-8.880	$0.840 - 0.1(-8.880) = 1.728$

After just the first point w jumps to 0.200 which is nowhere near using all the data like Batch Gradient Descent did. By the end of the epoch, $w = 1.728$, which has already overshoot the true optimum of 1.214 we found earlier.

Epoch 2 (w starts at 1.728)

Point used	x_i	y_i	$\hat{y}_i$	error	gradient	new w
(1, 1)	1	1	1.728	0.728	1.456	$1.728 - 0.1(1.456) = 1.582$
(2, 2)	2	2	3.165	1.165	4.660	$1.582 - 0.1(4.660) = 1.116$
(3, 4)	3	4	3.348	-0.652	-3.912	$1.116 - 0.1(-3.912) = 1.507$

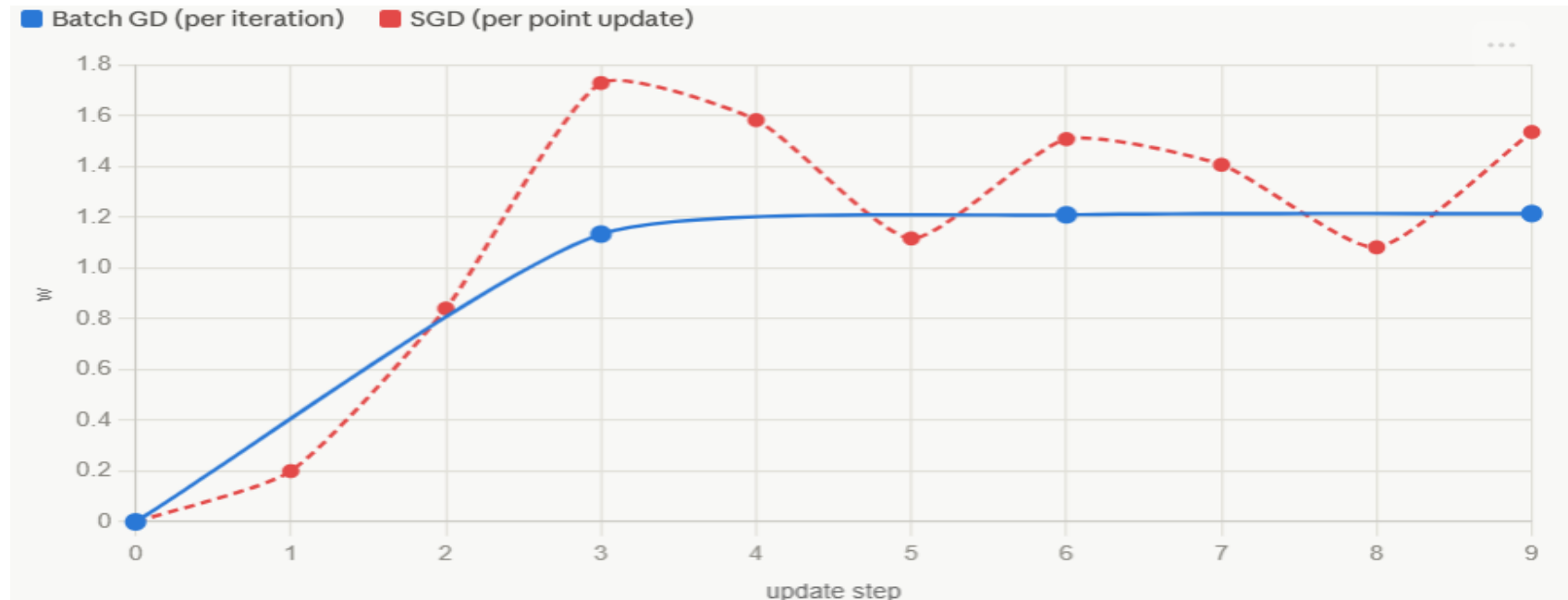
Epoch 3 (w starts at 1.507)

Point used	x_i	y_i	$\hat{y}_i$	error	gradient	new w
(1, 1)	1	1	1.507	0.507	1.014	$1.507 - 0.1(1.014) = 1.406$
(2, 2)	2	2	2.812	0.812	3.248	$1.406 - 0.1(3.248) = 1.081$
(3, 4)	3	4	3.243	-0.757	-4.542	$1.081 - 0.1(-4.542) = 1.535$

w bounces around: $0 \rightarrow 1.728 \rightarrow 1.507 \rightarrow 1.535 \rightarrow \dots$ — which shows it's clearly heading toward the neighborhood of the true optimum (1.214), but it overshoots, corrects, and jitters rather than gliding smoothly there like Batch Gradient Descent did (which went $0 \rightarrow 1.133 \rightarrow 1.209 \rightarrow 1.214$ almost monotonically).

Batch Vs Stochastic Gradient Descent

	Batch Gradient Descent	Stochastic Gradient Descent
Gradient uses	all N points every step	1 point per step
Cost per update	expensive (full dataset)	cheap
Path to minimum	smooth, direct	noisy, zig-zagging
Updates per epoch	1	N



4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Limitations of Gradient Descent
- Although Gradient Descent is effective, it uses only first-order derivative information.
- It does not provide information about:
 - Curvature of the loss surface.
 - Direction changes in multiple dimensions.
 - Relationships between variables.
 - These limitations motivate the use of more advanced derivative concepts.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Beyond Gradient Descent:

1. Jacobians

2. Hessians

3. Calculating derivatives of vector valued function

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Jacobian Matrix

- The Jacobian matrix is a grid of first-order partial derivatives representing how vector outputs change when inputs are tweaked.
- In optimization, it translates nonlinear problems into locally linear systems, enabling algorithms to iteratively step toward optimal minima or maxima
- The Jacobian generalizes the derivative to vector-valued functions.
- The Jacobian matrix of a vector-valued function compiles all the first-order partial derivatives. For $f: \mathbb{R}^m \rightarrow \mathbb{R}^n$, it helps track how each output changes in response to each input variable.
- Imagine a recipe where inputs (x, y) are ingredients and outputs (f_1, f_2) are taste and texture. The Jacobian matrix (J) maps exactly how a small change in one ingredient alters both taste and texture.

$$J(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

$J(\mathbf{x})$ is the Jacobian matrix of f at point $\mathbf{x}$.
It has m rows and n columns.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Example: Let $f: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ take two inputs and produce two outputs:

$$f(x_1, x_2) = \begin{bmatrix} f_1(x_1, x_2) \\ f_2(x_1, x_2) \end{bmatrix} = \begin{bmatrix} x_1^2 x_2 \\ x_1 + x_2^2 \end{bmatrix}$$

Compute all four partial derivatives:

- $\frac{\partial f_1}{\partial x_1} = 2x_1 x_2$, $\frac{\partial f_1}{\partial x_2} = x_1^2$, $\frac{\partial f_2}{\partial x_1} = 1$, $\frac{\partial f_2}{\partial x_2} = 2x_2$
- Assemble the Jacobian (row i = partials of output f_i , column j = w. r. t. input x_j)

$$J(x_1, x_2) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 x_2 & x_1^2 \\ 1 & 2x_2 \end{bmatrix}$$

Evaluate at the point $(x_1, x_2) = (1, 2)$

- $J(1, 2) = \begin{bmatrix} 2(1)(2) & 1^2 \\ 1 & 2(2) \end{bmatrix} = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}$
- Here, a small increase in x_1 changes f_1 by roughly $4 \times$ that amount and changes f_2 by $1 \times$ that amount; a small increase in x_2 changes f_1 by $1 \times$ and f_2 by $4 \times$. Each row is literally the gradient of that one output. Hence, the Jacobian is just gradients of every output stacked together.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Example 2: Find the Jacobian matrix at the point (1,2) of the following function: $f(x,y) = \begin{pmatrix} x^4 + 3y^2x \\ 5y^2 - 2xy + 1 \end{pmatrix}$.

- First of all, we calculate all the first-order partial derivatives of the function:
- $\frac{\partial f_1}{\partial x} = 4x^3 + 3y^2$, $\frac{\partial f_1}{\partial y} = 6yx$, $\frac{\partial f_2}{\partial x} = -2y$, $\frac{\partial f_2}{\partial y} = 10y - 2x$
- $J_f(x, y) = \begin{pmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{pmatrix} = \begin{pmatrix} 4x^3 + 3y^2 & 6yx \\ -2y & 10y - 2x \end{pmatrix}$
- Once we have found the expression of the Jacobian matrix, we evaluate it at point (1, 2): $J_f(1, 2) = \begin{pmatrix} 4 \cdot 1^3 + 3 \cdot 2^2 & 6 \cdot 2 \cdot 1 \\ -2 \cdot 2 & 10 \cdot 2 - 2 \cdot 1 \end{pmatrix}$
- And finally, we perform the operations: $J_f(1, 2) = \begin{pmatrix} 16 & 12 \\ -4 & 18 \end{pmatrix}$

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Hessian matrix

- The Hessian matrix is an $n \times n$ square matrix composed of the second-order partial derivatives of a function of n variables.
- The formula for the Hessian matrix is:

$$H(f)(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- The Hessian matrix will always be a square matrix whose dimension will be equal to the number of variables of the function. For example, if the function has 3 variables, the Hessian matrix will be a 3×3 matrix.
- The Hessian tells us about the curvature of the objective function. It helps determine whether a stationary point is:
 - Local minimum
 - Local maximum
 - Saddle point

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Interpreting results of Hessian Matrix for a stationary point.

Top-Left Entry	Determinant	Result	Classification	Function Quadratic =?	Interpretation
Positive (> 0)	Det > 0	Positive Definite	Local Minimum	Yes (Positive Definite Everywhere)	Local Minimum = Global Minimum
Negative (< 0)	Det > 0	Negative Definite	Local Maximum	Yes (Negative Definite Everywhere)	Local Maximum = Global Maximum
Any Sign	Det < 0	None	Saddle Point		

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Example 1: Calculate the Hessian Matrix for the given equation:

$$f(x, y) = 2x^2 + 3y^2 + 2xy - 4x - 6y + 5$$

Step 1: Find the gradient

$$\frac{\partial f}{\partial x} = 4x + 2y - 4, \quad \frac{\partial f}{\partial y} = 2x + 6y - 6$$

Step 2: Find the critical point by setting $\nabla f = 0$

$$4x + 2y - 4 = 0 \Rightarrow 2x + y = 2 \quad \& \quad 2x + 6y - 6 = 0 \Rightarrow x + 3y = 3$$

- Solving this system (substitute $y = 2 - 2x$ into the second equation):
 $x + 3(2 - 2x) = 3 \Rightarrow x + 6 - 6x = 3 \Rightarrow -5x = -3 \Rightarrow x = 0.6, y = 2 - 2(0.6) = 0.8$
- So, $(0.6, 0.8)$ is our candidate stationary point but is it a min, max, or saddle? That's exactly what the Hessian tells us.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Step 3: Compute the Hessian

$$\frac{\partial^2 f}{\partial x^2} = 4, \frac{\partial^2 f}{\partial y^2} = 6, \frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x} = 2 \text{ \& } H_f = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

- This Hessian is constant (doesn't depend on x, y), that's because f is a quadratic function; higher-order terms vanish after differentiating twice.

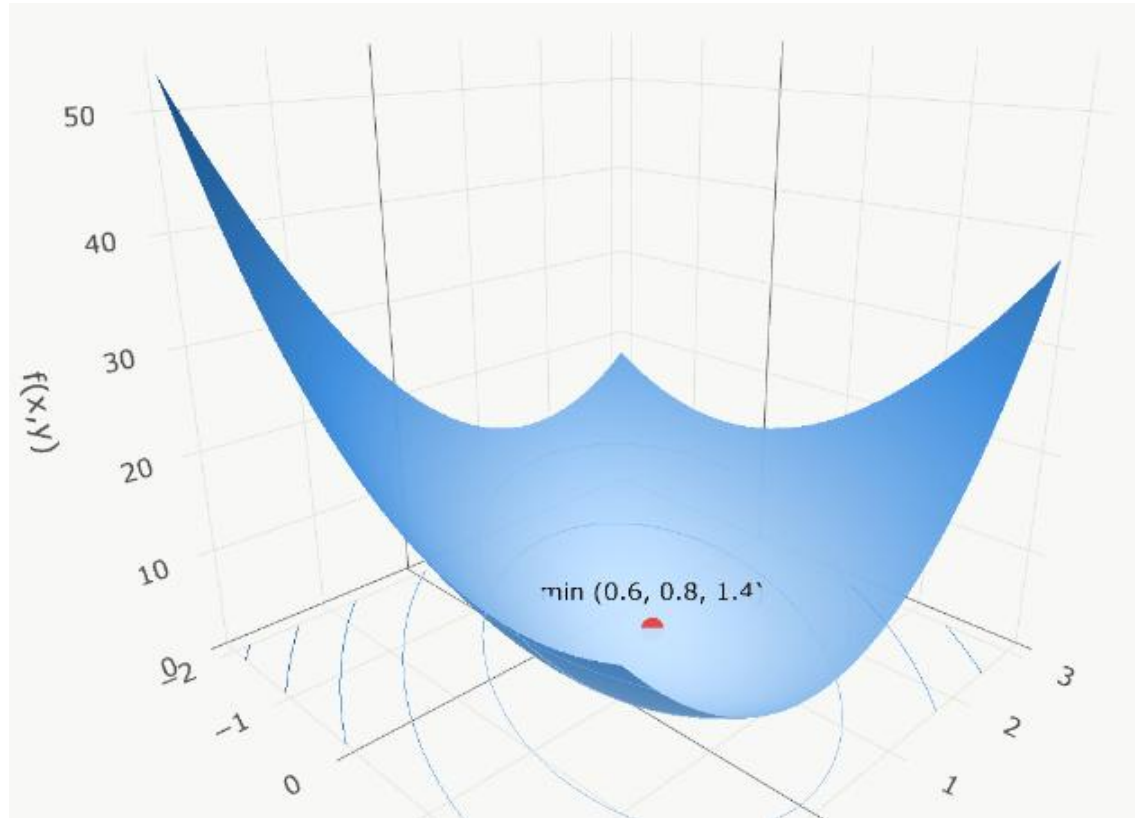
Step 4: Classify the critical point

- Check the top-left entry and the full determinant are both positive:
 $4 > 0$ and $\det(H_f) = (4)(6) - (2)(2) = 24 - 4 = 20 > 0$
- Both positive $\rightarrow$ confirms positive definite.
- Conclusion: since H_f is positive definite at $(0.6, 0.8)$, this critical point is a local minimum and since the Hessian is positive definite everywhere (it's constant), the function f is convex.

- Hence, this is actually the global minimum with value:

$$\begin{aligned} f(0.6, 0.8) &= 2(0.36) + 3(0.64) + 2(0.6)(0.8) - 4(0.6) - 6(0.8) + 5 \\ &= 0.72 + 1.92 + 0.96 - 2.4 - 4.8 + 5 \\ &= 1.4 \end{aligned}$$

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization



We can see it in true 3D — the actual "bowl" shape of $f(x,y) = 2x^2 + 3y^2 + 2xy - 4x - 6y + 5$, with the red marker sitting exactly at the bottom, $(0.6, 0.8, 1.4)$

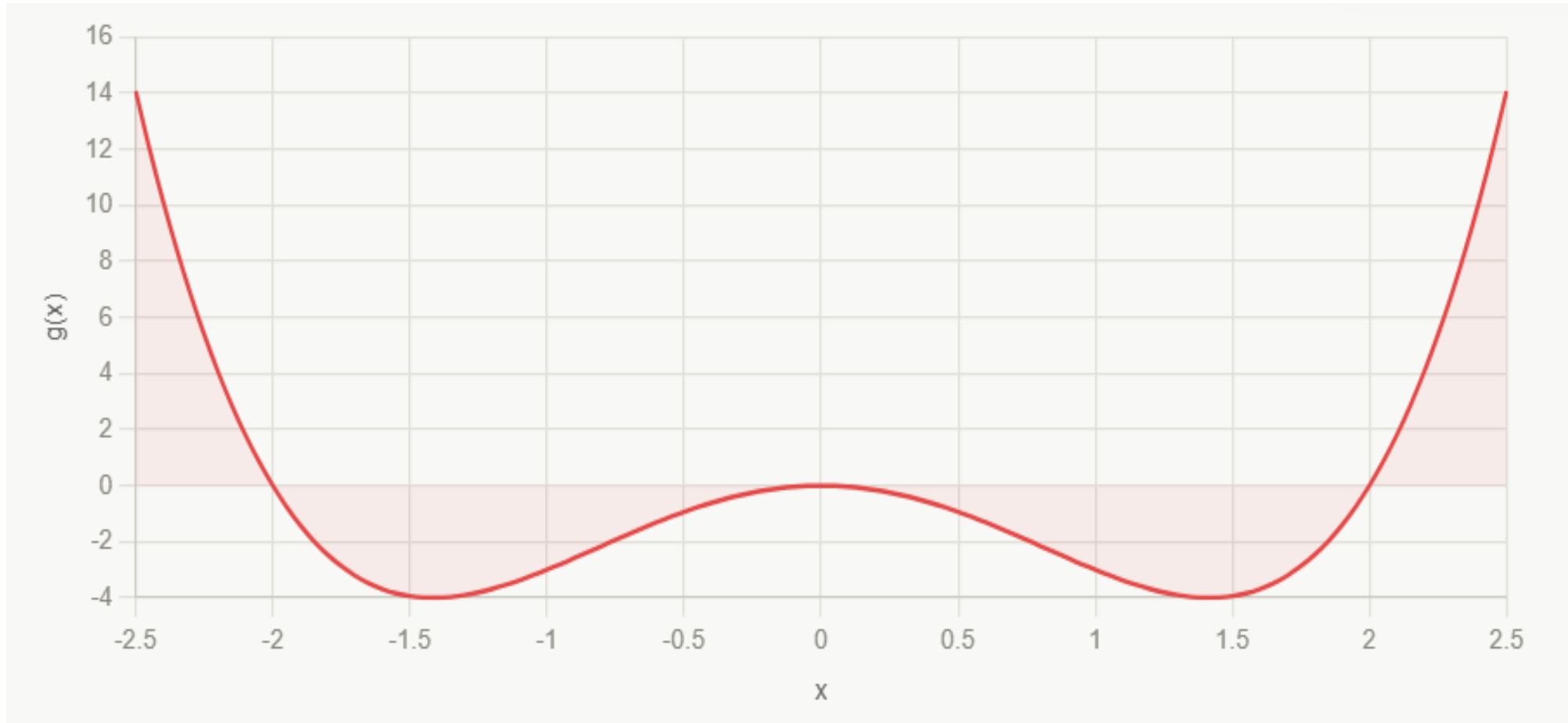
4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- When we computed the Hessian only at the critical point $(0.6, 0.8)$ and found it positive definite, that test *by itself* only tells us: "Locally, right around $(0.6, 0.8)$, the function curves upward in every direction."
- That's enough to say $(0.6, 0.8)$ is a local minimum but on its own, it says nothing about what the function does far away from that point. In general, a function can have several local minima at different heights, and the Hessian test at one of them tells us nothing about the others.
- So, if $Hf(x)$ is positive semi-definite at every point x in the domain, not just one), then any local minimum is automatically the global minimum.
- In our example the obtained matrix has no x or y in it at all, it's the same matrix no matter which point we plug in, because f is a pure quadratic (all its terms are degree ≤ 2 , so the third derivatives are all zero). That means when we showed it was positive definite, we didn't just show it at $(0.6, 0.8)$, we showed it holds at every single point in $\mathbb{R}^2$, since the matrix never changes. That satisfies the "everywhere" condition of the convexity theorem, which is what lets us upgrade "local minimum" to "global minimum" for free.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Example 2: $g(x)=x^4-4x^2$
- $g'(x) = 4x^3-8x = 4x(x^2-2)\Rightarrow$ critical points at $x=0, \sqrt{\pm 2}$.
- $g''(x) = 12x^2 - 8$
- At $x = \pm\sqrt{2}$: $g'' = 12(2) - 8 = 16 > 0 \rightarrow$ these are local minima.
- At $x = 0$: $g'' = -8 < 0 \rightarrow$ this is a local maximum.
- Here, $g''(x)$ is not positive everywhere as it's negative near $x = 0$. So g is not convex, and indeed we can see directly why the theorem doesn't apply: there isn't one global bowl, there are two separate valleys with a hill in between. In general, with a non-convex function, a positive Hessian at one critical point only certifies that *one* point as a local minimum, and we would have no guarantee it's the best one without checking elsewhere.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization



4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

	Convex bowl (our f)	Non-convex double well (our g)
Number of valleys	1	2
Hessian / second derivative	positive everywhere	changes sign
Local min = global min?	Yes, guaranteed	Not guaranteed in general

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Example 3: Calculate the Hessian matrix at the point (1,0) of the following multivariable function: $f(x,y) = y^4 + x^3 + 3x^2 + 4y^2 - 4xy - 5y + 8$.

Step 1: Compute the first-order partial derivatives of the function:

- $\frac{\partial f}{\partial x} = 3x^2 + 6x - 4y, \frac{\partial f}{\partial y} = 4y^3 + 8y - 4x - 5$

• Step 2: Calculate all the second-order partial derivatives of the function:

- $\frac{\partial^2 f}{\partial x^2} = 6x + 6, \frac{\partial^2 f}{\partial y^2} = 12y^2 + 8, \frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x} = -4$

• Step 3: Find the Hessian matrix using the formula for 2×2 matrices:

- $H_f(x,y) = \begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{pmatrix} = \begin{pmatrix} 6x + 6 & -4 \\ -4 & 12y^2 + 8 \end{pmatrix}$

• So, the Hessian matrix evaluated at the point (1,0) is:

- $H_f(1,0) = \begin{pmatrix} 6 \cdot 1 + 6 & -4 \\ -4 & 12 \cdot 0^2 + 8 \end{pmatrix}$

- $H_f(1,0) = \begin{pmatrix} 12 & -4 \\ -4 & 8 \end{pmatrix}$

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Step 4: Check definiteness
- Top-left entry: $12 > 0$
- $\det(H_f) = (12)(8) - (-4)(-4) = 96 - 16 = 80 > 0$
- Both positive $\rightarrow$ positive definite. Since $H_f(1, 0)$ is positive definite, $(1, 0)$ is a local minimum. The surface curves upward in every direction around that point, just like the bowl shape.
- However, $(1, 0)$ is a local minimum but whether it's the global minimum depends on whether f is convex everywhere which we can't determine from this single matrix alone; as the original equation is not purely quadratic.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Calculating derivatives of vector valued function

- Many machine learning models output vectors rather than single values.
- These derivatives are fundamental to backpropagation, where gradients are propagated through layers using the chain rule.
- Example:

$$\mathbf{r}(t) = \begin{bmatrix} t^2 \\ e^t \end{bmatrix}$$

Differentiate each component on its own:

$$r_1(t) = t^2 \Rightarrow r'_1(t) = 2t$$

$$r_2(t) = e^t \Rightarrow r'_2(t) = e^t$$

$$\mathbf{r}'(t) = \begin{bmatrix} 2t \\ e^t \end{bmatrix}$$

Evaluate at $t = 1$:

$$\mathbf{r}(1) = \begin{bmatrix} 1 \\ 2.718 \end{bmatrix}, \quad \mathbf{r}'(1) = \begin{bmatrix} 2 \\ 2.718 \end{bmatrix}$$

$\mathbf{r}(t)$ traces a curve in 2D space as t varies. At $t = 1$, you're at position $(1, 2.718)$ moving in the direction $(2, 2.718)$. This is exactly the right way to think about a gradient descent trajectory. Recall the SGD example from earlier: $w_0 \rightarrow w_1 \rightarrow w_2 \rightarrow \dots$ — if we treat the parameter path as a continuous curve $w(t)$ (a vector, since real models have many parameters, not just one), then $w'(t)$ describes how fast and in what direction the parameters are moving through parameter space and gradient descent is precisely choosing that direction to always point downhill, $-\nabla f(w)$.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Constraint Optimization
- Optimization in which variables must satisfy one or more constraints.
- General form: $\min f(x)$ Subject to $g_i(x) \leq 0, i = 1, \dots, m, h_j(x) = 0, j = 1, \dots, p$.
- Constrained optimization is where the theory finally meets most real ML/engineering problems head-on as almost nothing in practice is truly unconstrained.
- Many real-world problems include restrictions such as budget limits, battery capacity, time constraints, memory limitations. These restrictions are called constraints.
- Example: Minimize $f(x) = x^2$ subject to $x \geq 5$.
- Without the constraint, the minimum is at $x = 0$. However, since $x = 0$ is not feasible, the constrained minimum occurs at $x = 5$.
- Types of Constraints:
 1. Equality Constraints: (Example: $x + y = 5$)
 2. Inequality Constraints: (Example: $x + y \leq 5$)

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Equality Constraints

- Solved by using Lagrange Multipliers: $L(x, \lambda) = f(x) + \lambda(h(x) - c)$, we solve $\nabla L = 0$ to obtain the optimal solution.
- Example: Minimize $f(x, y) = x^2 + y^2$ subject to $x + y = 4$.
- Here, f can be considered as concentric circles centered at the origin (each circle is a "level set" where f is constant), and the constraint as a straight line. We want the smallest circle that still touches the line.

Step 1: Form the Lagrange Multiplier

$$\mathcal{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 4)$$

Step 2: Set all partial derivatives to zero

$$\frac{\partial \mathcal{L}}{\partial x} = 2x + \lambda = 0, \quad \frac{\partial \mathcal{L}}{\partial y} = 2y + \lambda = 0, \quad \frac{\partial \mathcal{L}}{\partial \lambda} = x + y - 4 = 0$$

Step 3: Solve the system

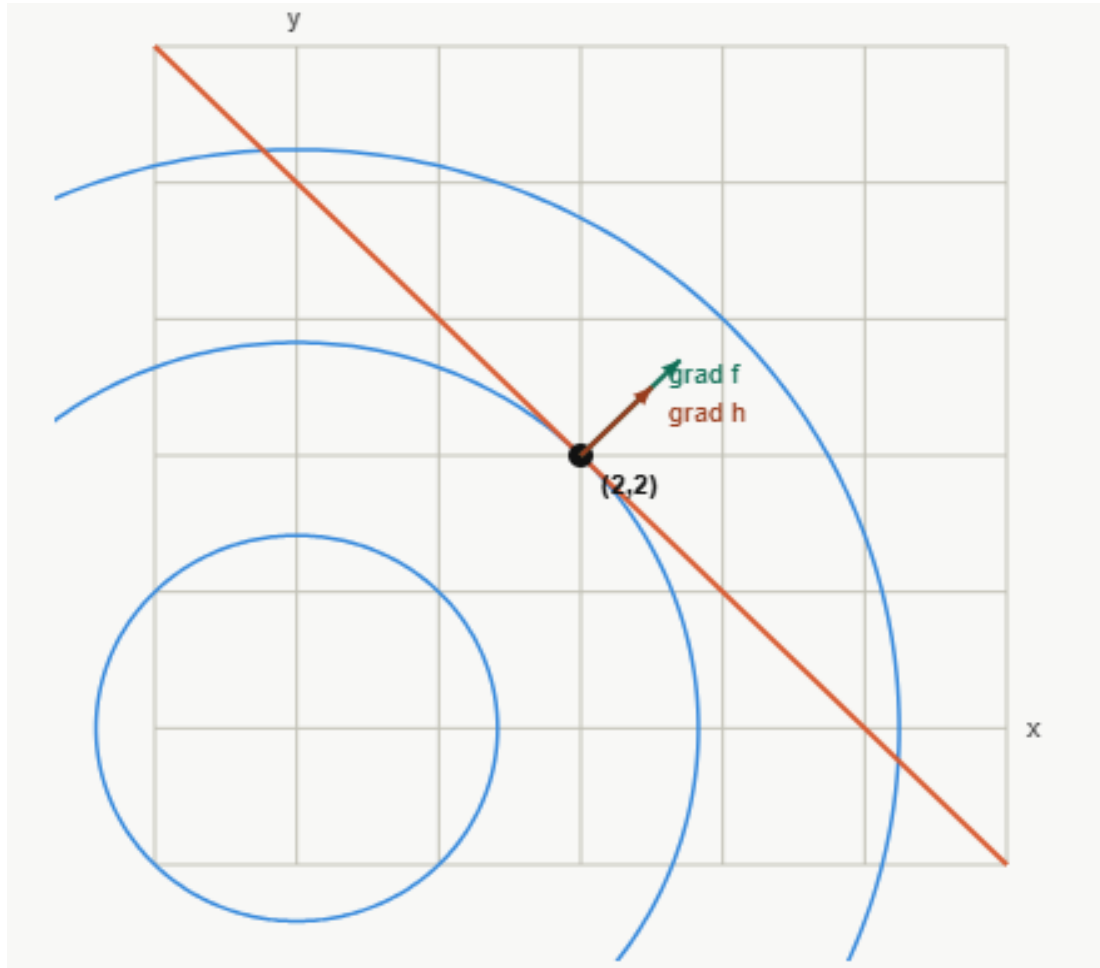
From the first two equations, $x = y = -\lambda/2$. Substituting into the third we get,

$$-\frac{\lambda}{2} - \frac{\lambda}{2} = 4 \Rightarrow -\lambda = 4 \Rightarrow \lambda = -4 \Rightarrow x = y = 2$$

Step 4: Answer: $(x, y) = (2, 2)$, with $f(2, 2) = 4 + 4 = 8$.

- Geometric meaning: at the optimum, the gradient of f , $\nabla f = (2x, 2y) = (4, 4)$ is parallel to the gradient of the constraint, $\nabla h = (1, 1)$ (indeed $(4, 4) = 4 \times (1, 1)$) — that parallelism is exactly what λ measures, and it's precisely the condition $\nabla f + \lambda \nabla h = 0$ we set up in Step 2.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization



The blue circles are level sets of f (bigger circle = higher value of f). The red diagonal line is the constraint. As we shrink the circle, the last one that still touches the line does so at exactly one point $(2,2)$ and at that touching point, the two gradient arrows point the same way. That tangency is the entire geometric content of Lagrange multipliers: we can't decrease f any further while staying on the line, because any small move along the line would require the circle to shrink and the line to be crossed, which isn't possible right at that tangent point.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Inequality Constraints

- Solved using Karush-Kuhn-Tucker (KKT) method. The KKT conditions extend the method of Lagrange multipliers to optimization problems with inequality constraints. The KKT Lagrangian function is:

$L(x, \mu) = f(x) + \sum_{i=1}^m \mu_i g_i(x)$ where: μ_i are the Lagrange multipliers (KKT multipliers) & each multiplier must satisfy $\mu_i \geq 0$.

KKT Conditions:

- A point x^* is optimal (under suitable regularity conditions) if it satisfies the following four conditions.

1. Stationarity: The gradient of the Lagrangian with respect to the decision variables must be zero.

$$\nabla_x \mathcal{L}(x^*, \mu^*) = \nabla f(x^*) + \sum_{i=1}^m \mu_i^* \nabla g_i(x^*) = 0$$

2. Primal Feasibility: The original inequality constraints must be satisfied. $g_i(x^*) \leq 0$ for all i

3. Dual Feasibility: All Lagrange multipliers must be non-negative. $\mu_i^* \geq 0$ for all i

4. Complementary Slackness: For each constraint, $\mu_i^* g_i(x^*) = 0$ for all i .

This means that for every constraint, either:

- the constraint is active ($g_i(x) = 0$), in which case μ_i may be positive, or the constraint is inactive ($g_i(x) < 0$), in which case $\mu_i = 0$.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Example 1: Minimize $f(x, y) = x^2 + y^2$ subject to $x + y \geq 1$.
- First, we notice the unconstrained minimum of f is at the origin $(0, 0)$ but that point violates $x + y \geq 1$ (since $0 < 1$). So, the constraint must be active (binding) at the true optimum; we can't just ignore it.

Step 1: Rewrite in standard form ($g(x) \leq 0$)

$$g(x, y) = 1 - x - y \leq 0$$

Step 2: Form the Lagrangian (using $\mu \geq 0$ for an inequality multiplier)

$$\mathcal{L}(x, y, \mu) = x^2 + y^2 + \mu(1 - x - y)$$

Step 3: Stationarity

$$\frac{\partial \mathcal{L}}{\partial x} = 2x - \mu = 0 \Rightarrow x = \frac{\mu}{2}, \quad \frac{\partial \mathcal{L}}{\partial y} = 2y - \mu = 0 \Rightarrow y = \frac{\mu}{2}$$

Step 4: Complementary slackness: $\mu(1 - x - y) = 0$. This splits into two cases:

Case A: $\mu = 0$. Then $x = y = 0$. Check feasibility: $x + y = 0 \geq 1$? False. Infeasible, we reject this case.

Case B: constraint active, $x + y = 1$. Substituting $x = y = \mu/2$: $\frac{\mu}{2} + \frac{\mu}{2} = 1 \Rightarrow \mu = 1$. Then $x = y = 0.5$.

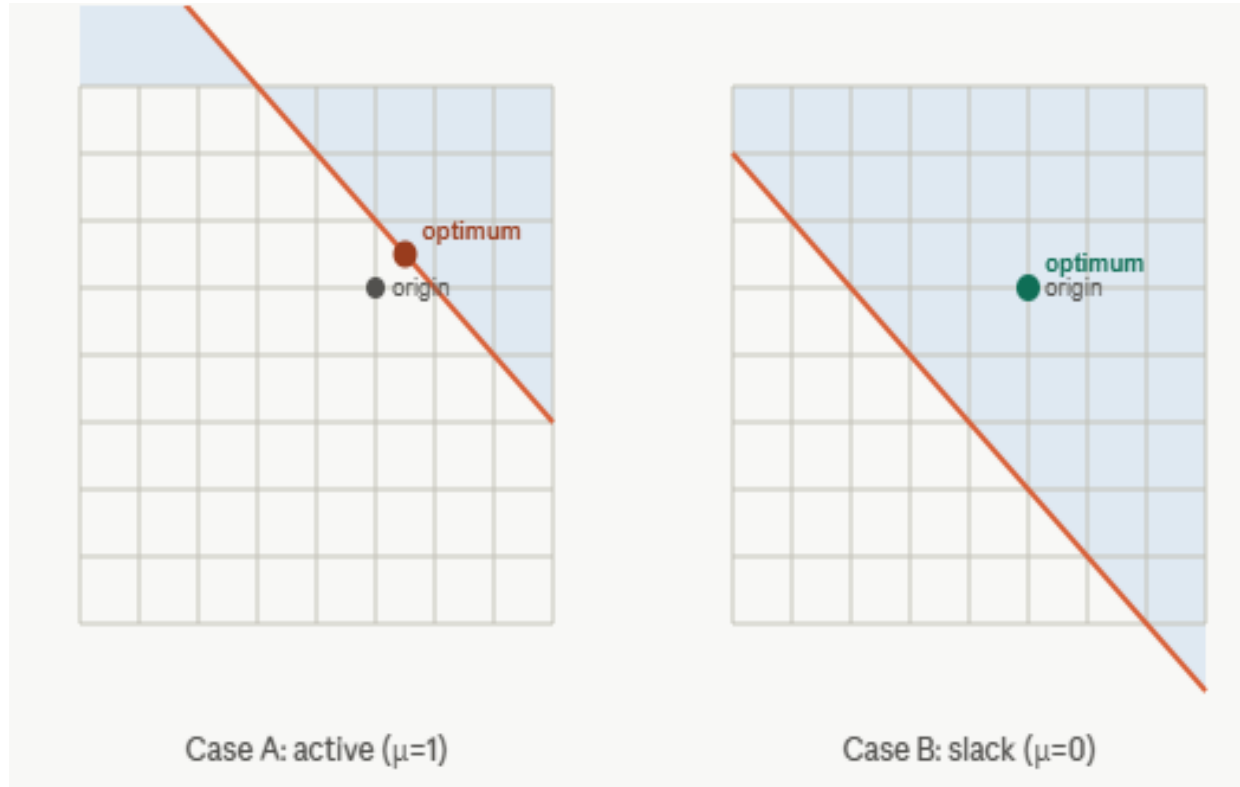
Step 5: Check dual feasibility: $\mu = 1 \geq 0$. ✓.

- Answer: $(x, y) = (0.5, 0.5)$, with $f(0.5, 0.5) = 0.25 + 0.25 = 0.5$, and multiplier $\mu = 1 > 0$ confirming the constraint is genuinely active (pressing against the boundary).
- Complementary slackness is what let us systematically discover which case applied — we didn't have to guess. If we'd instead had a constraint like $x + y \geq -5$ (which the unconstrained minimum $(0, 0)$ already satisfies), Case A ($\mu = 0$) would have been the valid one instead, and the constraint would simply be irrelevant to the answer.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Example 2: Minimize $f(x, y) = x^2 + y^2$ subject to constraint $x + y \geq -3$.
- Here, constraint $x + y \geq -3 \rightarrow g(x, y) = -3 - x - y \leq 0$, multiplier μ .
- Same stationarity equations: $x = y = \mu/2$. Testing $\mu = 0$: get $(0, 0)$, and now $g(0, 0) = -3 - 0 - 0 = -3 \leq 0$, feasible! This case works.
- Checking the other case for completeness: active constraint would force $\mu = -3$, violating $\mu \geq 0$, correctly rejected.
- Result: $(x, y) = (0, 0)$, $\mu = 0$, $f = 0$, the constraint was so far away it never affected the answer at all; complementary slackness automatically detected this and switched its multiplier off.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization



We can see it directly in the two panels:
In Case A, the origin (the unconstrained minimum) sits outside the shaded feasible region, so the optimizer gets pushed onto the boundary line showing the constraint is doing real work, hence $\mu > 0$.
In Case B, the origin already sits comfortably inside the feasible region, so the constraint never needed to intervene. The optimizer just stays put at the unconstrained answer, and $\mu = 0$.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

Convex optimization

- Convex optimization focuses on problems where the objective function and feasible region are convex.

1. Convex Sets

- A set C is convex if, for any two points $x, y \in C$ and any $t \in [0, 1]$, the point in between is also in C : $tx + (1 - t)y \in C$.
- In plain words: draw a straight line between any two points in the set & the whole line must stay inside.
- Let $C = \{(x, y): x^2 + y^2 \leq 4\}$ (a disk of radius 2). Take two points on the boundary, $x = (2, 0)$ and $y = (0, 2)$, and $t = 0.5$.
- $0.5(2, 0) + 0.5(0, 2) = (1, 1)$.
- Check membership: $1^2 + 1^2 = 2 \leq 4$ is true. Inside the disk. If we try it at any t , any two points in a disk, and it always works i.e. disks (and more generally, all norm-balls) are convex.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

2. Convex Functions

- f is convex if its domain is a convex set and, for all x, y in that domain and $t \in [0, 1]$:

$$f(tx + (1 - t)y) \leq tf(x) + (1 - t)f(y)$$

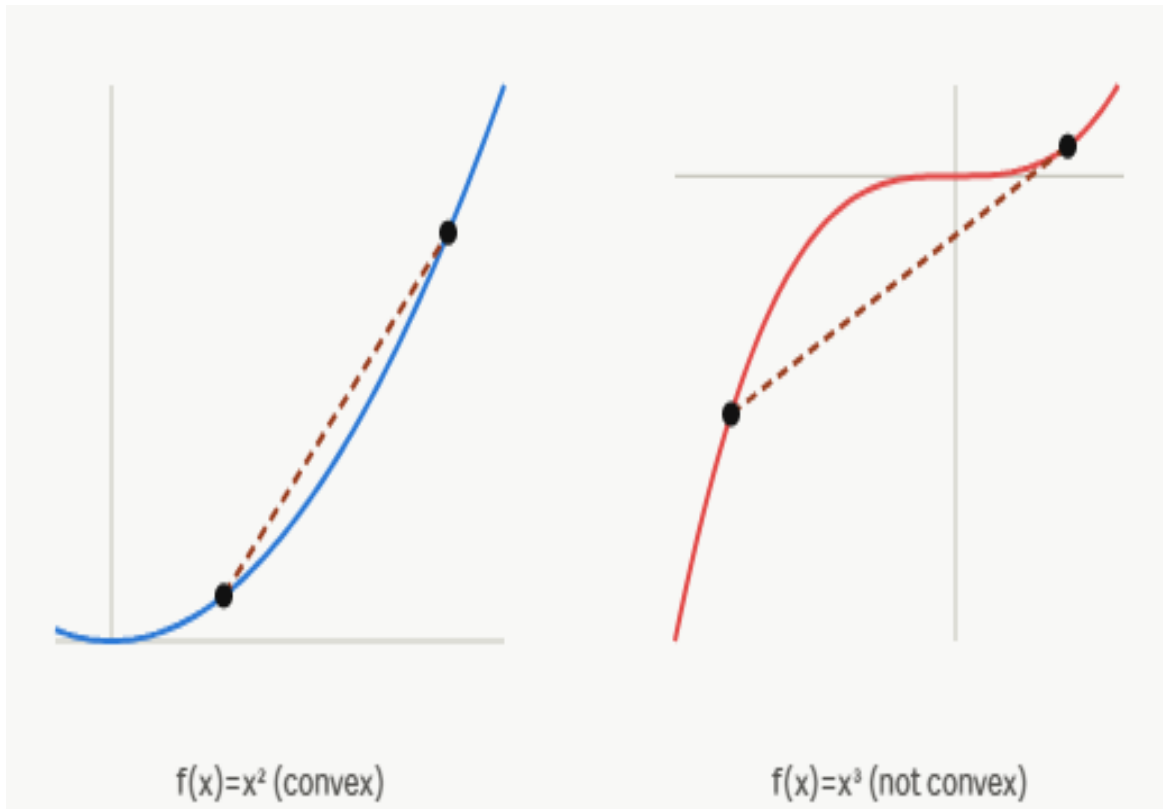
- This is called Jensen's inequality. In plain words: the function value at any blend of two points is never more than the same blend of the function values i.e., the straight chord connecting any two points on the graph always sits on or above the curve.
- Let $f(x) = x^2$. Take $x = 1, y = 3, t = 0.5$
- Left side : $f(0.5 \times 1 + 0.5 \times 3) = f(2) = 4$
- Right side: $0.5f(1) + 0.5f(3) = 0.5(1) + 0.5(9) = 0.5 + 4.5 = 5$
- Checking $4 \leq 5$ is true i.e. Jensen's inequality holds here. (It holds for every pair of points for x^2 , which is what makes it convex.

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- Non-Convex Function

- Let $f(x) = x^3$. Take $x = -2, y = 1, t = 0.5$.
- Left side: $f(0.5 \times (-2) + 0.5 \times 1) = f(-0.5) = -0.125$
- Right side: $0.5f(-2) + 0.5f(1) = 0.5(-8) + 0.5(1) = -4 + 0.5 = -3.5$
- Checking is $-0.125 \leq -3.5$? No i.e. -0.125 is larger than -3.5 . Jensen's inequality fails, confirming x^3 is not convex over the reals (it only becomes convex if we restrict to $x \geq 0$).

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization



Here, on the left graph, the dashed chord stays above the blue curve everywhere between the two dots, that's convexity, visually.

On the right, the dashed chord dips below the red curve near the middle that visual gap is exactly the failed inequality we computed numerically above ($-0.125 > -3.5$).

4.3 Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization

- For a convex function, gradient descent (or SGD) is guaranteed to converge to the *global* minimum as there are no other local minima or saddle regions hiding elsewhere to get trapped in.
- *For a* not convex function the Hessian is negative near $x = 0$ and consequently *does* have multiple competing local minima.
- That's the entire reason linear regression, logistic regression, and SVMs are considered "easy" to train reliably, while training a deep neural network whose loss surface is emphatically not convex, full of saddle points and many local minima is comparatively difficult and why so much of modern deep learning research is about heuristics (momentum, adaptive learning rates, good initialization) to cope with a landscape that convex optimization theory simply doesn't have to worry about.

4.4 Review of Machine Learning Pipeline

- A Machine Learning Pipeline is a series of interconnected stages through which data passes from collection to prediction.

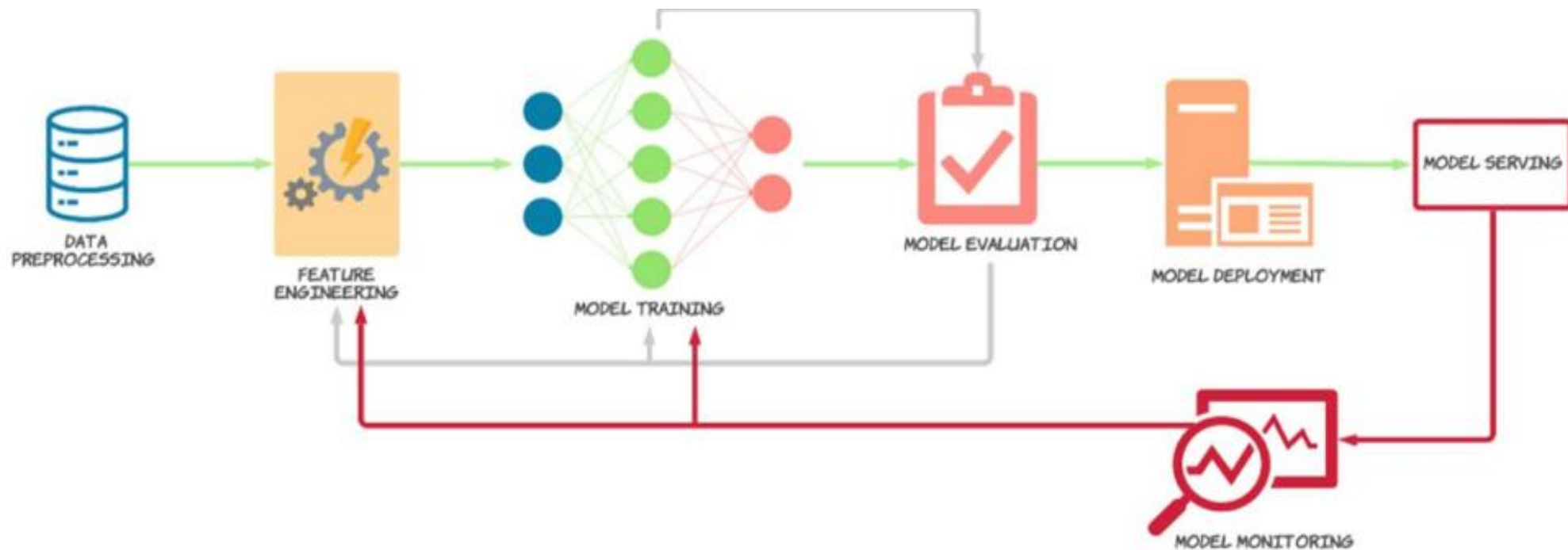


Figure: Lifecycle of Machine Learning Model

4.4 Review of Machine Learning Pipeline

Stage 1: Problem Definition

- Before building a model, we clearly define What problem are we solving?, What type of prediction is required?, What data is available? and What is the expected outcome?
- Examples: Spam Detection –Classification, Customer Segmentation-Clustering

Stage 2: Data Collection

- Data is the foundation of Machine Learning. Input data may be available in different formats such as images, text, audio, etc.
- Various sources of data are Databases, CSV files, Sensors, Web scraping, APIs, Social media and IoT devices.
- While collecting data from various sources some challenges may be faced such as:
 - Missing data
 - Inconsistent data
 - Noisy data
 - Insufficient data

4.4 Review of Machine Learning Pipeline

Stage 3: Data Preprocessing

- Raw data is rarely suitable for machine learning.
- Therefore, the objectives of data pre-processing is to clean data, handle missing values, remove noise and standardize formats.
- Common Tasks

1. Handling Missing Values

Age	Salary
25	40000
NULL	50000

Methods:

- Remove records
- Mean substitution
- Median substitution

4.4 Review of Machine Learning Pipeline

2. Removing Duplicates

- If there are multiple occurrence of same records then remove the multiple occurrences.
- Example: Seeta, 50
Seeta, 50
- Only one record is considered.

3. Encoding Categorical Data

- If we have categorical data as Yes & No, we encode them as 1 & 0.

4. Exploratory Data Analysis (EDA)

- EDA helps understand the data by identifying patterns, detecting outliers, understanding relationships between metrics.
- Methods used are:
 - Histograms
 - Box plots
 - Scatter plots
 - Correlation matrices

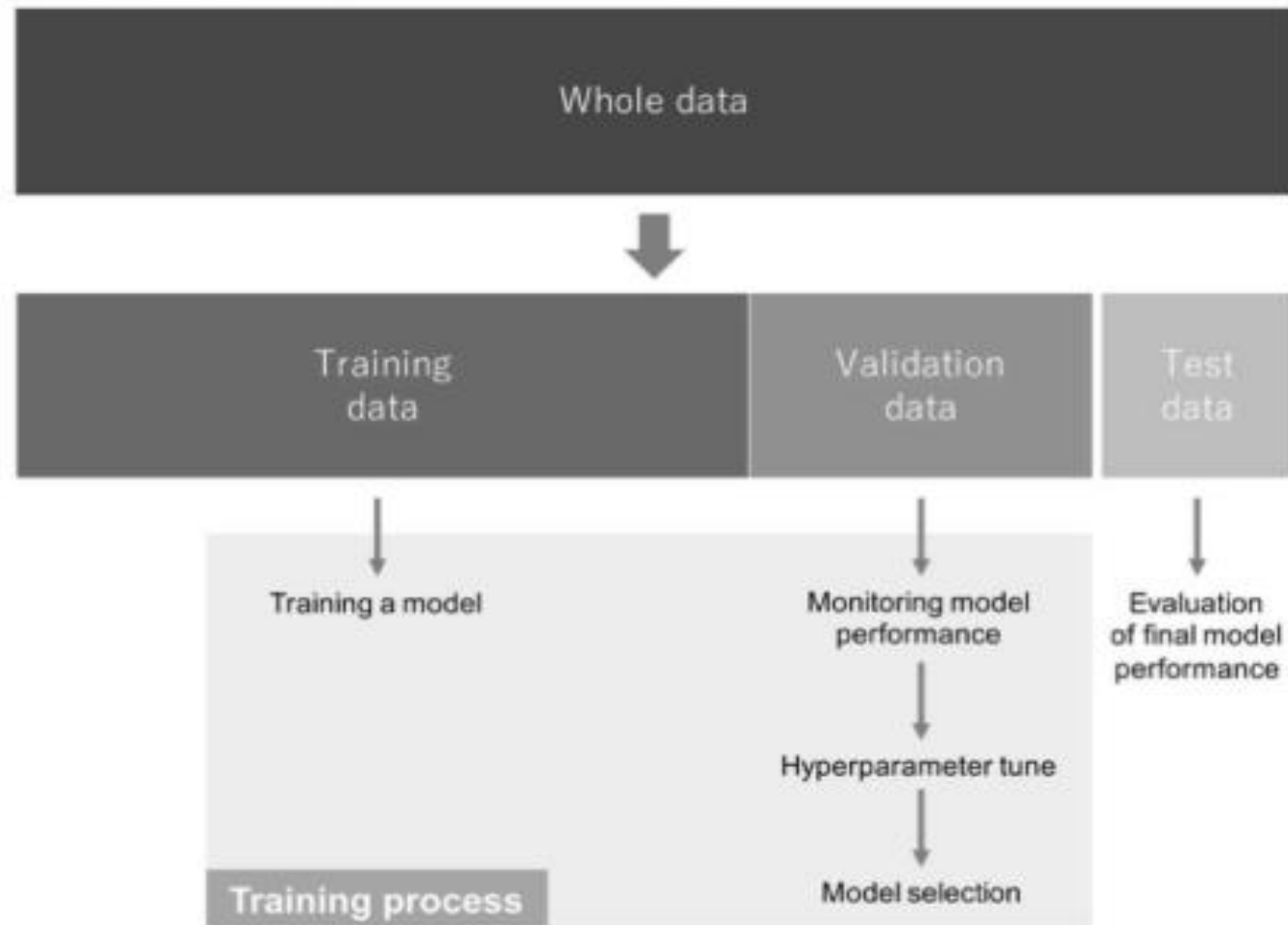
4.4 Review of Machine Learning Pipeline

Stage 4: Feature Engineering

- Feature engineering creates better input variables for learning.
- The process of selecting, transforming, or creating features that improve model performance.
- Example: Original Features: Date of Birth then New Feature: Age = Current Date - Date of Birth
- Techniques Used:
 1. Feature Creation
 - Combine existing features. Example: $BMI = Weight / Height^2$
 2. Feature Selection
 - Choose relevant features using methods such as Correlation analysis, Chi-square test, etc.
 3. Dimensionality Reduction
 - Reduce number of features.

4.4 Review of Machine Learning Pipeline

Stage 5: Splitting the Dataset



4.4 Review of Machine Learning Pipeline

Stage 6: Model Selection

- Choose an algorithm appropriate for the task. Such as Classification, Regression, Clustering.
- Choosing the right model depends on various factors such as dataset size, complexity, accuracy requirements, computational cost and interpretability.

Stage 7: Model Training

- Training means teaching the model to use data.
- The objective is to minimize the prediction error.

Stage 8: Hyperparameter Tuning

- Hyperparameters control learning behavior.
- They are used to estimate the model parameters.
- They are set manually.
- The choice of hyperparameter decide how efficient the training is.

4.4 Review of Machine Learning Pipeline

Stage 9: Model Evaluation

- After training, we evaluate the performance of the model using various metrics such as Accuracy, Precision, F1 Score, Recall etc.

Stage 10: Model Deployment

- After successful evaluation, the model is deployed for real-world use.
- Various deployment methods used are Web applications, Mobile applications, Cloud services, Embedded systems and APIs.

Stage 11: Monitoring and Maintenance

- Deployment is not the end.
- We need to monitor the model because data changes over time and this phenomenon is called data drift. Data drift is caused due to changes in input data distribution or changes in relationship between inputs and outputs.
- In case of change in data, we periodically retrain the model using new data.

4.4.1 Model development: Generative versus Discriminative

- Model development is the process of selecting, training, and optimizing a machine learning algorithm so that it can learn patterns from data and make predictions on unseen examples.
- A machine learning model attempts to learn a relationship between the input features (X) and the target variable (Y).
- For example: Input (X): Study Hours, Attendance and Assignment Score & Output (Y): Pass or Fail
- The way a model learns this relationship depends on the learning strategy it uses.
- Two of the most important strategies are:
 - Generative Models
 - Discriminative Models

4.4.1 Model Development: Generative versus Discriminative

- Generative Models

- Generative models are machine learning models and, as the name suggests, aim to capture the underlying distribution of data, and generate new data comparable to the original training data.
- These types of models, mainly used for unsupervised learning, are categorized as a class of statistical models capable of generating new data instances.
- Instead of directly predicting the class, it first learns the probability distribution of each class, and how the features are distributed within each class.
- From a mathematical perspective, the process of training a classifier involves estimating either, a function represented as $f: (X \rightarrow Y)$, or the probability $P(Y | X)$.
- Generative classifiers first assumes a particular functional form for the probabilities such as $P(Y)$, $P(X|Y)$. *It then directly estimate the parameters of probability such as $P(X | Y)$, $P(Y)$ from the training dataset and calculates the posterior probability $P(Y/X)$ using the Bayes Theorem.*

4.4.1 Model Development: Generative versus Discriminative

- Characteristics of Generative Models:
 - They learn the data distribution.
 - Generate new samples.
 - Handle missing data well.
 - Require stronger assumptions about data.
 - Work well even with smaller datasets.
- Popular Generative Models:
 - Bayesian Network
 - Generative Adversarial Network (GAN)
 - Variational Autoencoders (VAEs)
 - Hidden Markov Model (HMM)

4.4.1 Model Development: Generative versus Discriminative

- Example: Email Spam Detection(Naïve Bayes Classifier)
- Let we want to classify an email as either Spam (S) or Not Spam (N). We will use only one feature: Does the email contain the word "Free"? So, $X = \text{"Contains Free"}$ & $Y = \{\text{Spam, Not Spam}\}$.

Email	Contains "Free" = ?	Class
Free Offer	Yes	Spam
Win Free Prize	Yes	Spam
Free Gift	Yes	Spam
Meeting Today	No	Not Spam
Project Report	No	Not Spam
Lunch Tomorrow	No	Not Spam
Free Coupon	Yes	Spam
Family Dinner	No	Not Spam
Total Emails = 8		

4.4.1 Model Development: Generative versus Discriminative

- Solution,

Total Emails = 8

Spam Email = 4

Not Spam Email = 4

$P(\text{Spam}) = 4/8 = 0.5$

$P(\text{Not Spam}) = 4/8 = 0.5$

Now, among all the emails labelled as spam every email contains “free”.

Hence, $P(\text{Free} | \text{Spam}) = 4/4 = 1$

Again, among all the emails labelled as not spam none of the email contains “free”.

Hence, $P(\text{Free} | \text{Not Spam}) = 0/4 = 0$

4.4.1 Model Development: Generative versus Discriminative

- Now, if a new email appears that has content as “Free Vacation” then

$$\begin{aligned}P(\text{Spam} | \text{Free}) &= \frac{P(\text{Free}|\text{Spam}) * P(\text{Spam})}{P(\text{Free})} \\&= \frac{P(\text{Free}|\text{Spam}) * P(\text{Spam})}{P(\text{Free}|\text{Spam}) * P(\text{Spam}) + P(\text{Free}|\text{Not Spam}) * P(\text{Not Spam})} \\&= \frac{1 * 0.5}{1 * 0.5 + 0 * 0.5} \\&= 1 \\P(\text{Not Spam} | \text{Free}) &= \frac{P(\text{Free}|\text{Not Spam}) * P(\text{Not Spam})}{P(\text{Free})} \\&= \frac{P(\text{Free}|\text{Spam}) * P(\text{Spam})}{P(\text{Free}|\text{Spam}) * P(\text{Spam}) + P(\text{Free}|\text{Not Spam}) * P(\text{Not Spam})} \\&= \frac{0 * 0.5}{1 * 0.5 + 0 * 0.5} \\&= 0\end{aligned}$$

- Therefore, this new email is classified as “Spam”.

4.4.1 Model Development: Generative versus Discriminative

- Discriminative models

- Discriminative models are machine learning models and, as the name suggests, concentrate on modeling the decision boundary between several classes of data using probability estimates and maximum likelihood.
- These types of models, mainly used for supervised learning, are also known as conditional models.
- A discriminative model learns the direct relationship between input features and output labels.
- Instead of learning how the data was generated, it learns $P(Y | X)$ & only focuses on finding the best decision boundary between classes.
- From a mathematical perspective, the process of training a classifier involves estimating either, a function represented as $f : (X \rightarrow Y)$, or the probability $P(Y | X)$. However, discriminative classifiers first assumes a particular functional form of probability $P(Y/X)$, and directly estimate the parameters of probability $P(Y/X)$ from the training dataset.

4.4.1 Model Development: Generative versus Discriminative

- Characteristics of Discriminative Models:

- They Learn only the boundary.
- Usually require more labeled data.
- Often achieve higher classification accuracy.
- Train faster because they do not model the entire data distribution.
- Cannot naturally generate new data.

- Popular Discriminative Algorithms:

- Logistic Regression
- Support Vector Machine (SVM)
- Decision Tree
- Random Forest
- Neural Networks

4.4.1 Model Development: Generative versus Discriminative

- Example: Email Spam Detection(Logistic Regression)
- Instead of learning probabilities like $P(\text{Free} \mid \text{Spam})$ it directly learns a rule that separates Spam from Not Spam.
- Converting the features into numbers:
- Now the datasets becomes:

Contains Free	Value
No	0
Yes	1

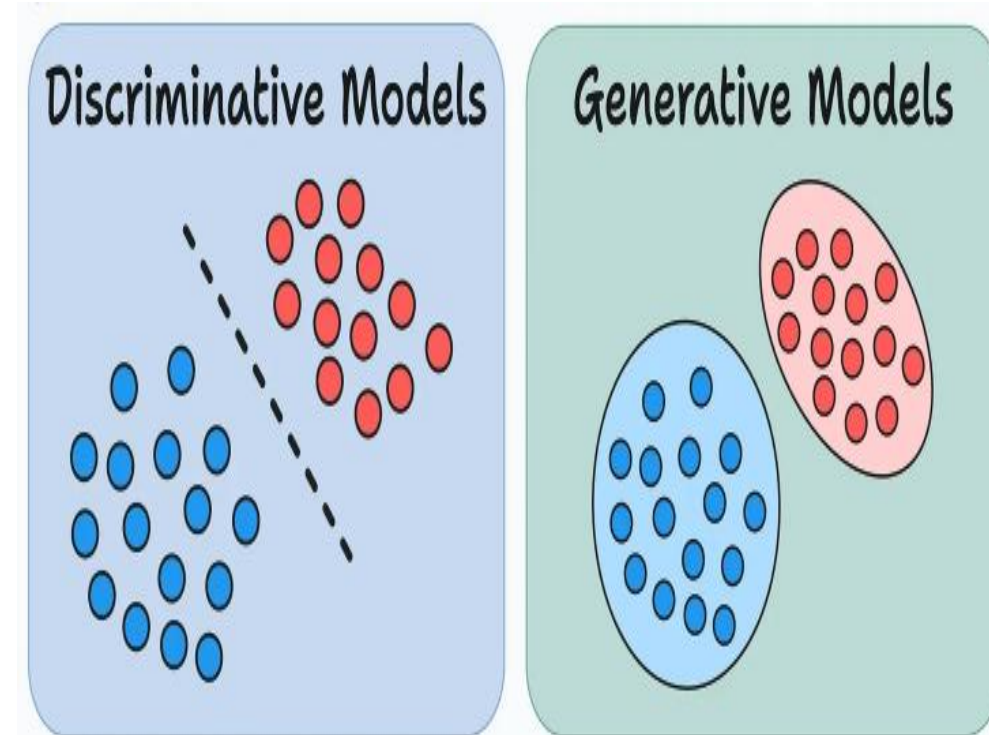
X	Y
1	Spam
1	Spam
1	Spam
0	Not Spam
0	Not Spam
0	Not Spam
1	Spam
0	Not Spam

4.4.1 Model Development: Generative versus Discriminative

- Instead of calculating probability distributions for each class, logistics regression finds a decision boundary.
- For this simple dataset, it learns a rule like:
- If $X = 1 \rightarrow \text{Spam}$ & If $X = 0 \rightarrow \text{Not Spam}$ Or mathematically, If $x > 0.5$, predict Spam otherwise, Predict Not Spam.
- Now , for a new email with "Free Vacation", it checks if it contains Free? -> Yes.
- So, $x = 1$. Since, $1 > 0.5$. Predict Spam

4.4.1 Model Development: Generative versus Discriminative

Characteristic	Discriminative Models	Generative Models
Objective	Focus on learning the boundary between different classes directly from the data. Their primary objective is to classify input data accurately based on the learned decision boundary.	Aim to understand the underlying data distribution and generate new data points that resemble the training data. They focus on modeling the process of data generation, allowing them to create synthetic data instances.
Probability Distribution	Estimates the parameters of probability $P(Y X)$ from the training dataset.	Calculates the posterior probability $P(Y X)$ using the Bayes Theorem.
Handling Outliers	Relatively robust to outliers	Prone to outliers
Property	They do not possess generative properties.	They possess discriminative properties.
Applications	Commonly used in classification tasks, such as image recognition and sentiment analysis.	Commonly used in tasks like data generation, anomaly detection, and data augmentation, beyond traditional classification tasks.
Examples	Logistic regression, Support vector machines, Decision trees, neural nets etc.	Variational Autoencoders (VAEs), Generative adversarial network (GAN), Nave Bayes etc.



4.4.2 Learning Algorithm

Learning Methods

- There are two different kinds of information processing which must be considered in a machine learning system.
- *Inductive learning* is concerned with determining general patterns, organizational schemes, rules, and laws from raw data, experience or examples.
- *Deductive learning* is concerned with determination of specific facts using general rules or the determination of new general rules from old general rules. $A=B$, $B=C$, then $A =C$.
- Deductive learning is accomplished through a sequence of deductive inference steps using known facts.
- From the known facts, new facts or relationships are logically derived.
- Deductive learning usually requires more inference than the other methods.

Learning may be classified into three categories:

- Supervised Learning
- Unsupervised Learning
- Semi-Supervised Learning
- Reinforcement Learning

Supervised

Unsupervised

Semi-Supervised

Reinforcement

Supervised Machine Learning

- The network is presented with inputs together with the target (teacher signal) outputs and it tries to produce an output as close as possible to the target signal.

Learn through examples of which we know the desired output (what we want to predict).

The training data is labeled.

Is this a cat or a dog?

Are these emails spam or not?

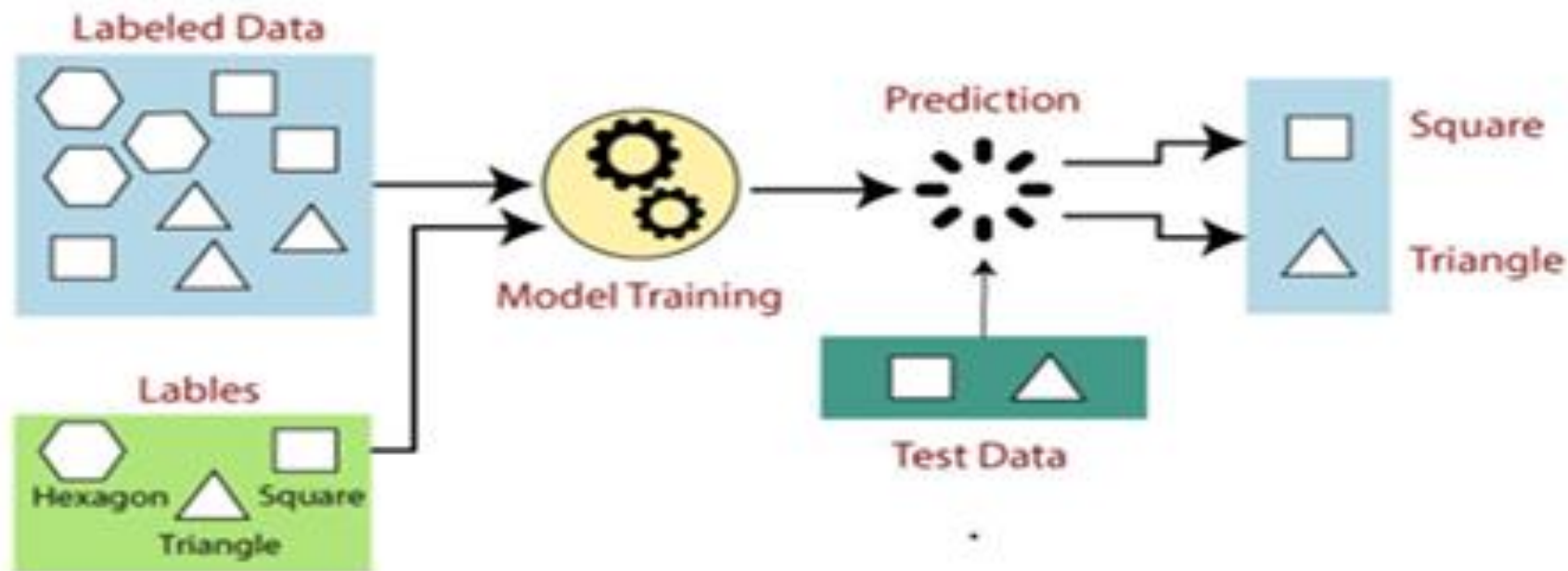
Predict the market value of houses, given the square meters, number of rooms, neighborhood, etc.

Supervised Machine Learning

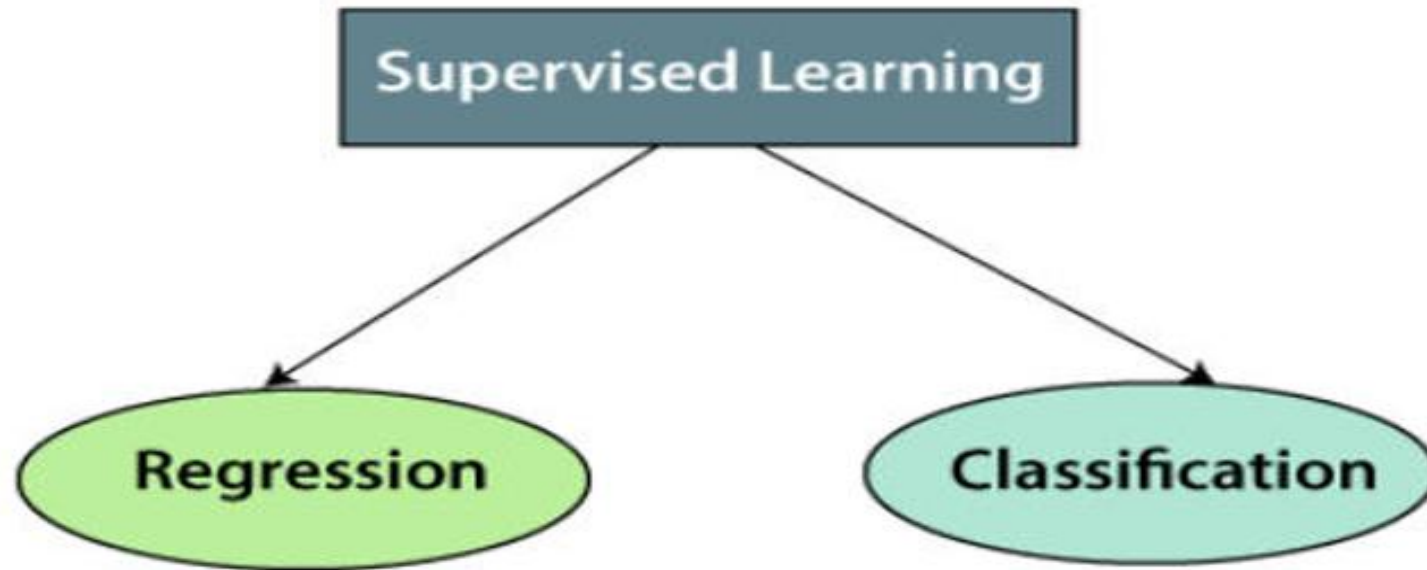
- Supervised learning is the types of machine learning in which machines are trained using well "labelled" training data.
- The labelled data means some input data is already tagged with the correct output.
- On basis of that data, machines predict the output.
- In supervised learning, the training data provided to the machines work as the supervisor that teaches the machines to predict the output correctly.
- The aim of a supervised learning algorithm is to find a mapping function to map the input variable(x) with the output variable(y).
- Example: Risk Assessment, Image classification, Fraud Detection, spam filtering, etc.

How Supervised Learning Work?

- After the training process is completed, the model is tested on the basis of test data (a subset of the training set), and then it predicts the output.



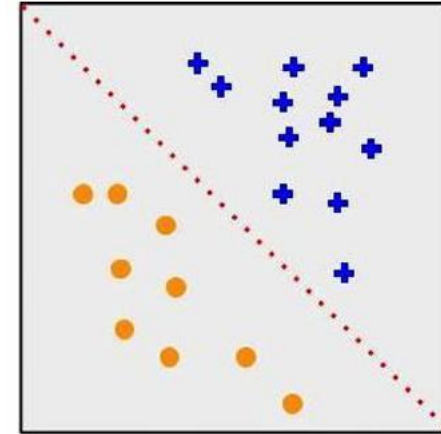
Types of Supervised Machine Learning Algorithms



Supervised Machine Learning

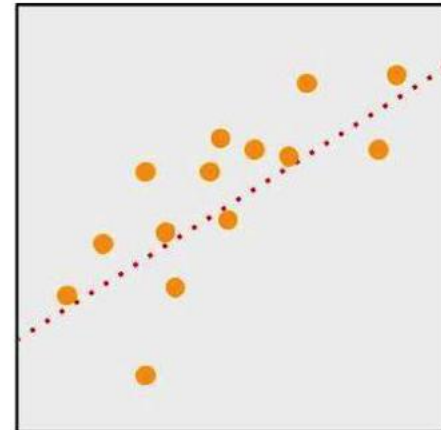
1. Classification

- Output is a discrete variable(yes/no)
- Example: cat/dog, spam filtering, etc



2. Regression

- Output is continuous (numerical)
- Example: price, temperature, housing price, etc



Advantages of Supervised Learning

1. High Accuracy & Reliability

- Since models learn from correct labeled data, they can achieve high accuracy when trained properly. Examples: Image classification, medical diagnosis, spam detection.

2. Easy Interpretation & Understanding

- Models like linear regression and decision trees provide interpretable insights. Example: A bank can explain why a loan application was approved or rejected.

3. Predictable & Generalizable

- Supervised models can make accurate predictions on new, unseen data if trained well. Example: A weather forecasting system trained on past data can predict future weather trends.

4. Wide Range of Applications

- Used in finance, healthcare, marketing, and self-driving cars. Example: Fraud detection systems in banks use supervised learning to classify legitimate vs. fraudulent transactions.

Unsupervised Learning

- In unsupervised learning, there is no teacher (target signal) from outside and the network adjusts its weights in response to only the input patterns.

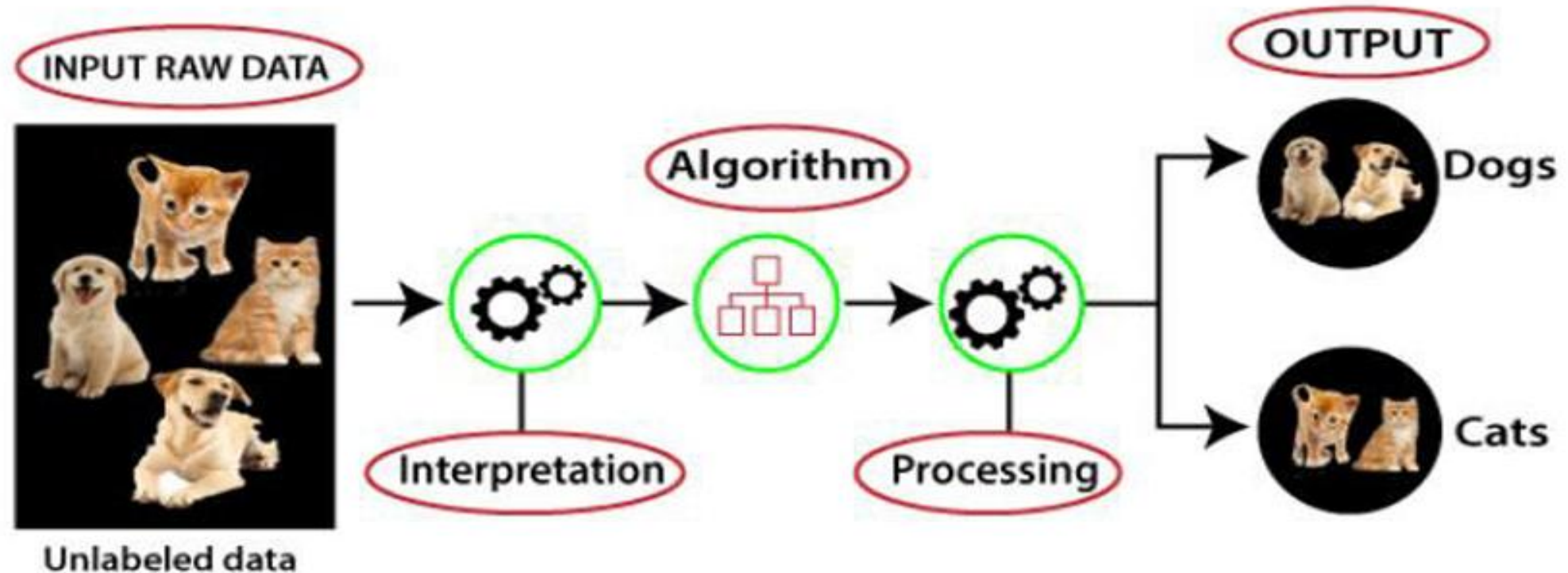
*There is no desired output. Learn something about the data.
Latent relationships.*

The training data is unlabeled.

I have photos and want to put them in 20 group

I want to find anomalies in the credit card usage patterns of my customers.

Working of Unsupervised Learning



Types of Unsupervised Learning Algorithms

Clustering

- Clustering is a method of grouping the objects into clusters such that objects with most similarities remain into a group
- Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.

Association

- It is used for finding the relationships between variables in the large database.
- Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item.

Advantages of Unsupervised Learning

1. No Need for Labeled Data

- Unlike supervised learning, no manual labeling is required, saving time and effort. Useful when labeled data is scarce or expensive to obtain. Example: Clustering customer data without predefined categories.

2. Identifies Hidden Patterns & Structures

- Helps uncover hidden relationships in data. Useful for exploratory data analysis. Example: Detecting customer segments in marketing data without predefined groups.

3. Can Handle Large & Complex Datasets

- Works well on big data, especially when patterns are unknown. Can find non-obvious correlations between data points. Example: Analyzing genomic sequences in biology.

4. Adaptability & Real-Time Learning

- Can adapt to new data dynamically, making it ideal for real-time applications. Useful for tasks where data evolves continuously. Example: Anomaly detection in network security (e.g., identifying new cyber threats).

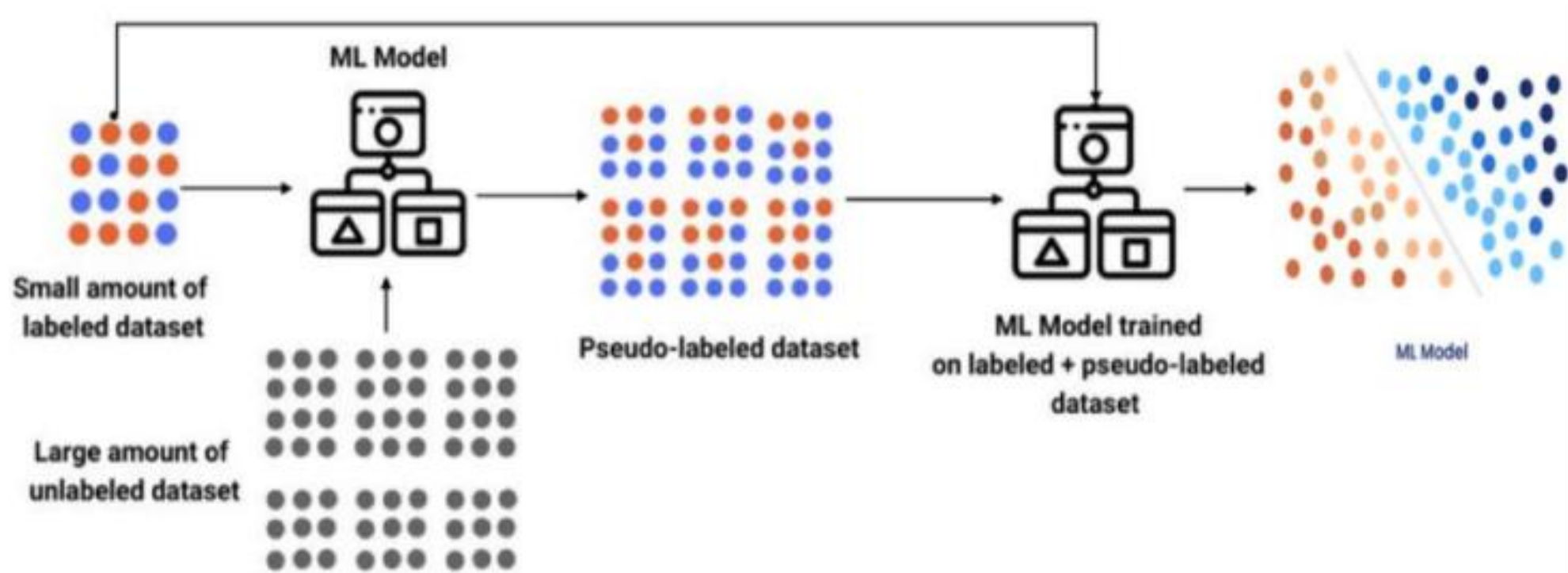
Disadvantages of Unsupervised Learning

- Unsupervised learning is intrinsically more difficult than supervised learning as it does not have corresponding output.
- The result of the unsupervised learning algorithm might be less accurate as input data is not labeled, and algorithms do not know the exact output in advance.
- The more the features, the more the complexity increases.
- It is also a time-consuming process. The learning phase of the algorithm might take a lot of time, as it analyses and calculates all possibilities.

Semi-Supervised Learning

- Semi-Supervised Learning is a machine learning paradigm that trains a model using both labeled and unlabeled data.
- Typically, small amount of labeled data & large amount of unlabeled data.
- The model learns from both to improve prediction accuracy.
- Most semi-supervised learning algorithms are combinations of unsupervised and supervised algorithms.
- Working Principle
 - Train model using labeled data.
 - Predict labels for unlabeled data.
 - Select highly confident predictions.
 - Add these samples into labeled dataset.
 - Retrain the model.
 - Repeat until convergence.

Semi-Supervised Learning



Advantages of Semi-Supervised Learning

- Reduces labeling cost.
- Improves accuracy compared to supervised learning with few labels.
- Utilizes abundant unlabeled data.
- Particularly useful for deep learning applications.
- Effective when expert annotation is expensive.

Reinforcement Learning

- Reinforcement learning is similar to supervised learning in that some feedback is given, however instead of providing a target output a reward is given based on how well the system performed.
- The aim of reinforcement learning is to maximize the reward the system receives through trial-and error.
- This paradigm relates strongly with how learning works in nature, for example an animal might remember the actions it's previously taken which helped it to find reward.
- An agent interacts with an environment and watches the result of the interaction.
- Environment gives feedback via a positive or negative reward signal.

Reinforcement Learning

- Reinforcement Learning is a feedback-based Machine learning technique.
- Reinforcement Learning solves a specific type of problem where decision making is sequential, and the goal is long-term, such as game-playing, robotics, etc.
- The agent interacts with the environment and explores it by itself. The primary goal of an agent in reinforcement learning is to improve the performance by getting the maximum positive rewards.
- The agent learns with the process of hit and trial, and based on the experience, it learns to perform the task in a better way.

Reinforcement Learning

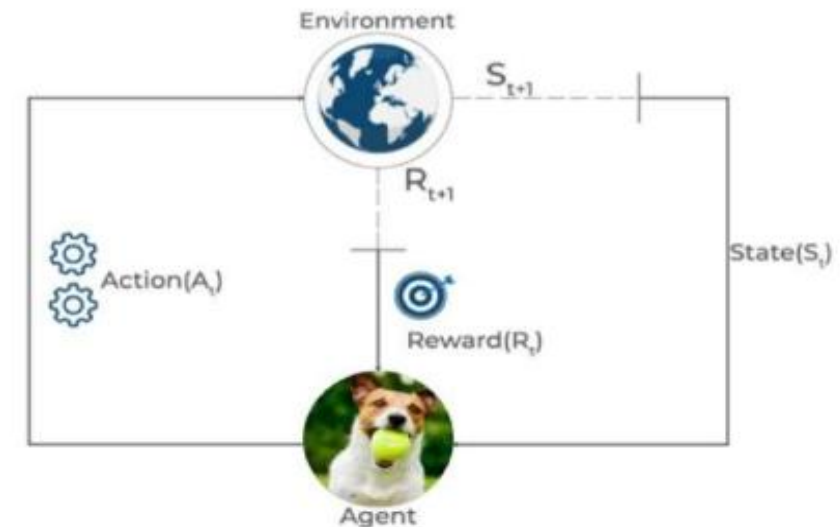
Example: Suppose you have a cute male puppy. You want to train the puppy to bring the tennis ball back to you whenever you threw it away. How do you train him? One way will be by giving him delicious foods (such as meat).

- Suppose you have delicious food (meat) in your one hand and the tennis ball in another hand. Now, you will throw the ball and command that puppy to bring back that ball.
- If he brings that ball, then you will give him some meat. Otherwise, whatever he does, you won't give him even a small bite of that meat.
- The training completes whenever he learns to bring back that ball to you in any situation.



In **Reinforcement learning**,

- Puppy acts as an **agent**.
- Delicious food (meat) can be referred to as a **reward**.
- The tennis ball and overall surroundings act as an **environment**.
- Puppy's movements after your command can be referred to as **actions**.



How Does Reinforcement Learning Work?



Key Features of Reinforcement Learning

- In Reinforcement Learning, the agent is not instructed about the environment and what actions need to be taken.
- It is based on the hit and trial process.
- The agent takes the next action and changes states according to the feedback of the previous action.
- The agent may get a delayed reward.
- The environment is stochastic, and the agent needs to explore it to reach to get the maximum positive rewards.

Types of Reinforcement Learning

Positive Reinforcement

- Here, we try to add a reward for every good result in order to increase the likelihood of a good result.
- Too much positive reinforcement may lead to an overload of states that can reduce the consequences.

Negative Reinforcement

- It is opposite to the positive reinforcement as it increases the tendency that the specific behavior will occur again by avoiding the negative condition.
- It can be more effective than the positive reinforcement depending on situation and behavior, but it provides reinforcement only to meet minimum behavior.

Advantages of Reinforcement Learning

- It can solve higher-order and complex problems. Also, the solutions obtained will be very accurate.
- This model will undergo a rigorous training process that can take time. This can help to correct any errors.
- Since the model learns constantly, a mistake made earlier would be unlikely to occur in the future.

Disadvantages of Reinforcement Learning

- The usage of reinforcement learning models for solving simpler problems won't be correct. The reason being, the models generally tackle complex problems.
- We need lots of data to feed the model for computation. Reinforcement Learning models require a lot of training data to develop accurate results.
- This consumes time and lots of computational power.

Application of Reinforcement Learning

- Companies can analyze customer preferences and help in the better advertisement of the products.
- We have Deep learning algorithms like AlphaGo, AlphaZero that are gaming algorithms for games like chess, shogi and go.
- RL is now a big help in recommendation systems like news, music apps, and web-series apps like Netflix, etc. These apps work as per customer preferences.

4.4.3. Capacity, Overfitting and Underfitting

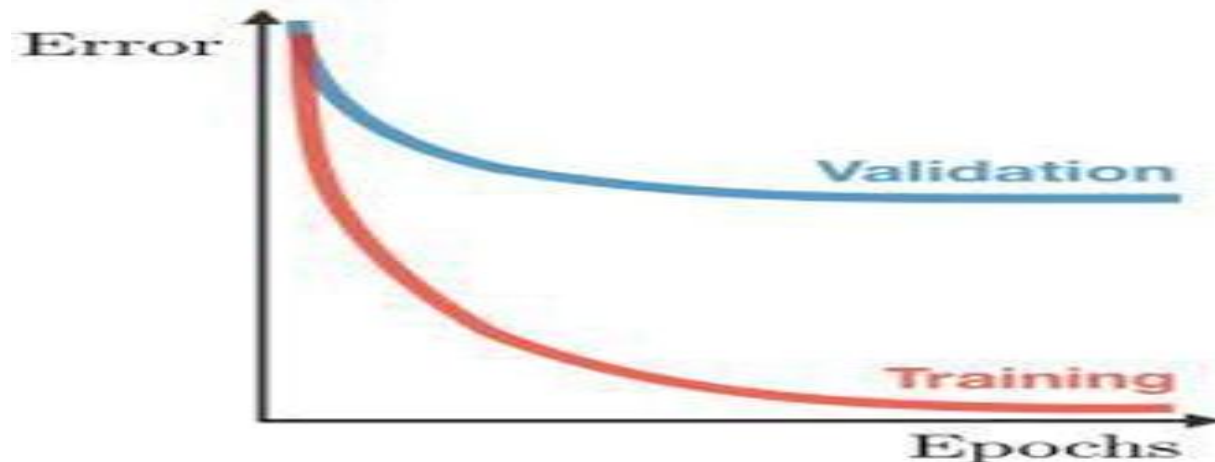
- The goal of a learning algorithm is not to minimize error on the training set but it is to minimize error on new, unseen data drawn from the same distribution. This is called generalization.
- We define two key quantities:
 - Training Error: Error measured on the data used to fit the model.
 - Generalization Error (Test Error): Expected error on new samples from the same data-generating distribution.
- A model that achieves near-zero training error but performs poorly on new data has not learned the underlying pattern it has memorized noise.
- A model should neither:
 - Learn too little from the data (Underfitting)
 - Learn too much, including noise (Overfitting)

Capacity

- Model Capacity is the ability of a machine learning model to learn a wide variety of functions or patterns from data.
- Low-capacity models can learn only simple relationships (e.g., linear models).
- High-capacity models can learn highly complex relationships (e.g., high-degree polynomials, deep neural networks with many parameters).
- A model with appropriate capacity captures the true relationship between input and output without memorizing random noise.

Overfitting

- Overfitting happens when a model learns the detail and noise in the training data to the extent that it negatively impacts the performance of the model on new data.
- This means that the noise or random fluctuations in the training data is picked up and learned as concepts by the model.
- The problem is that these concepts do not apply to new data and negatively impact the model's ability to generalize.
- The model performs well on the training data, but it does not generalize well i.e. training accuracy becomes very high while testing accuracy decreases or we can say that the model generates very low training error but very high testing error.

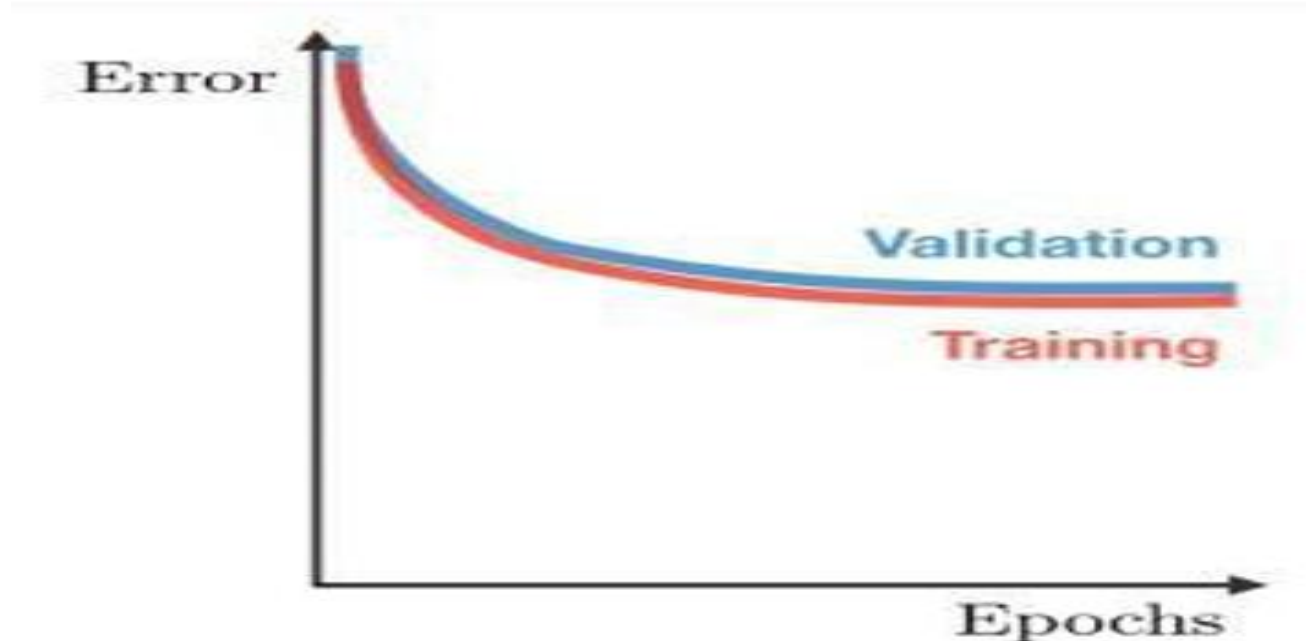


Removing Overfitting

- To mitigate overfitting
 - **Regularization:** Adding penalty terms to the loss function to limit the magnitude of model parameters.
 - **Cross-validation:** Splitting the data into multiple subsets for training and evaluation to ensure better generalization.
 - **Feature selection:** Selecting only the most relevant features to reduce noise and complexity.
 - **Ensemble methods:** Combining multiple models to reduce individual model's tendencies to overfit.
 - **Training with more data**
 - **Early stopping the training:** Fed of training data can be stopped at an early stage, due to which the model may not learn enough from the training data.

Underfitting

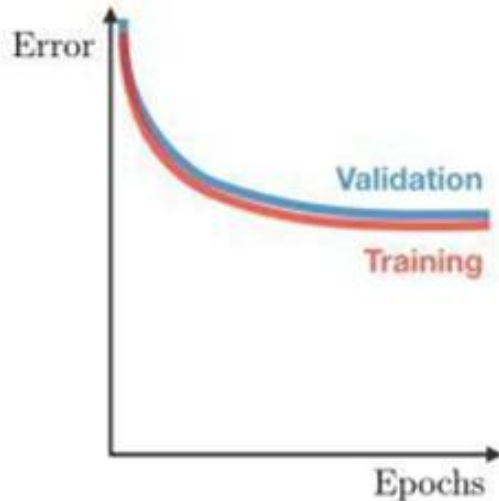
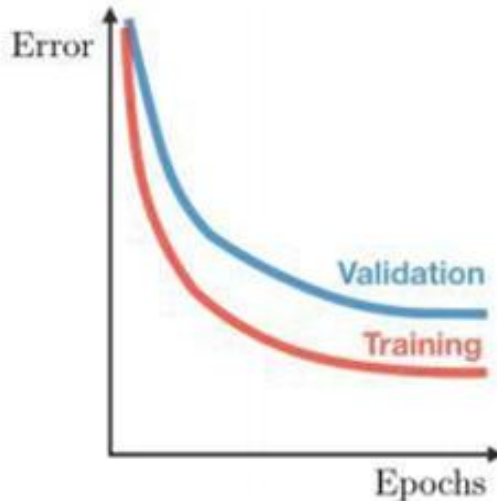
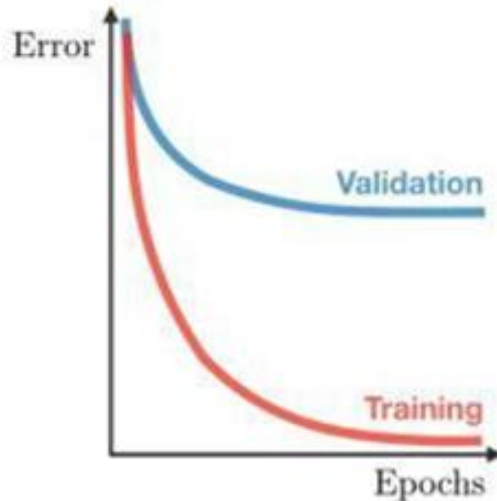
- Underfitting is the opposite of overfitting: it occurs when the model is too simple to learn the underlying structure of the data.
- The model performs poorly on both the training dataset and unseen testing dataset.
- Here, the model generates very high training error as well as very high testing error.



Removing Underfitting

- **Use more complex models:** Choose algorithms that are capable of capturing more intricate relationships in the data.
- **Feature engineering:** Transforming or combining features to better represent the underlying patterns.
- **Increase model capacity:** Allow the model to have more parameters or complexity to better fit the data.
- **Increase the training time of the model.**

Overfitting Vs Underfitting

	Underfitting	Just right	Overfitting
Symptoms	- High training error - Training error close to test error - High bias	- Training error slightly lower than test error	- Low training error - Training error much lower than test error - High variance
	 The graph shows Error on the y-axis and Epochs on the x-axis. Two curves, Training (red) and Validation (blue), are nearly identical and both remain at a high error level throughout the training process.	 The graph shows Error on the y-axis and Epochs on the x-axis. The Training error (red) curve is slightly below the Validation error (blue) curve, and both curves decrease as the number of epochs increases.	 The graph shows Error on the y-axis and Epochs on the x-axis. The Training error (red) curve decreases sharply and levels off at a low value. The Validation error (blue) curve initially decreases but then increases significantly after a certain point, indicating overfitting.

4.4.3. Capacity, Overfitting and Underfitting

- Example: Suppose we train polynomial regression models of increasing degree on the same dataset.

Polynomial Degree	Training Accuracy	Testing Accuracy
1 (Linear)	68%	65%
2 (Quadratic)	88%	86%
4	96%	94%
8	99%	75%

Degree 1: The model is too simple and cannot capture the relationship, resulting in low training and testing accuracy (underfitting).

Degree 2 and Degree 4: These models capture the underlying pattern well and generalize effectively.

Degree 8: Although the model achieves nearly perfect training accuracy, testing accuracy drops significantly because it has memorized the training data (overfitting).

4.4.4. Hyperparameters and Validation Sets

- Hyperparameters are values that are specified before training begins and control the learning process.
- Examples: Learning rate, Number of hidden layers, Batch size, Activation Function
- Unlike parameters, hyperparameters are not learned from the training data.
- Selection of hyperparameters is crucial because incorrect hyperparameters can lead to:
 - Underfitting
 - Overfitting
 - Slow training
 - Poor prediction accuracy
 - Unstable learning
- Choosing suitable hyperparameters often makes a larger difference than selecting a different algorithm.
- Hyperparameter tuning is the process of finding the combination of hyperparameter values that yields the best validation performance.

Parameters Vs Hyperparameters

Parameters	Hyperparameters
They are required for making predictions	They are required for estimating the model parameters
They are estimated by optimization algorithms(Gradient Descent, Adam, Adagrad)	They are estimated by hyperparameter tuning
They are not set manually	They are set manually
The final parameters found after training will decide how the model will perform on unseen data.	The choice of hyperparameters decide how efficient the training is. In gradient descent the learning rate decide how efficient and accurate the optimization process is in estimating the parameters.
For example the weights of a deep neural network.	For example, the learning rate in deep neural networks.

Validation Set

- Validation set is the dataset used to compare models, tune hyperparameters, monitor overfitting and perform early stopping
- The validation set does not update the model parameters. It provides an unbiased estimate of how different hyperparameter settings perform during development.

Cross-Validation

- When the dataset is small, holding out a single fixed validation set wastes data and gives a noisy, high-variance estimate of performance (the estimate depends heavily on which points happened to land in that split). Cross-validation addresses this.

K-Fold Cross Validation

- The "**K**" in K-Fold refers to the number of data subsets or "folds" into which the dataset is divided.

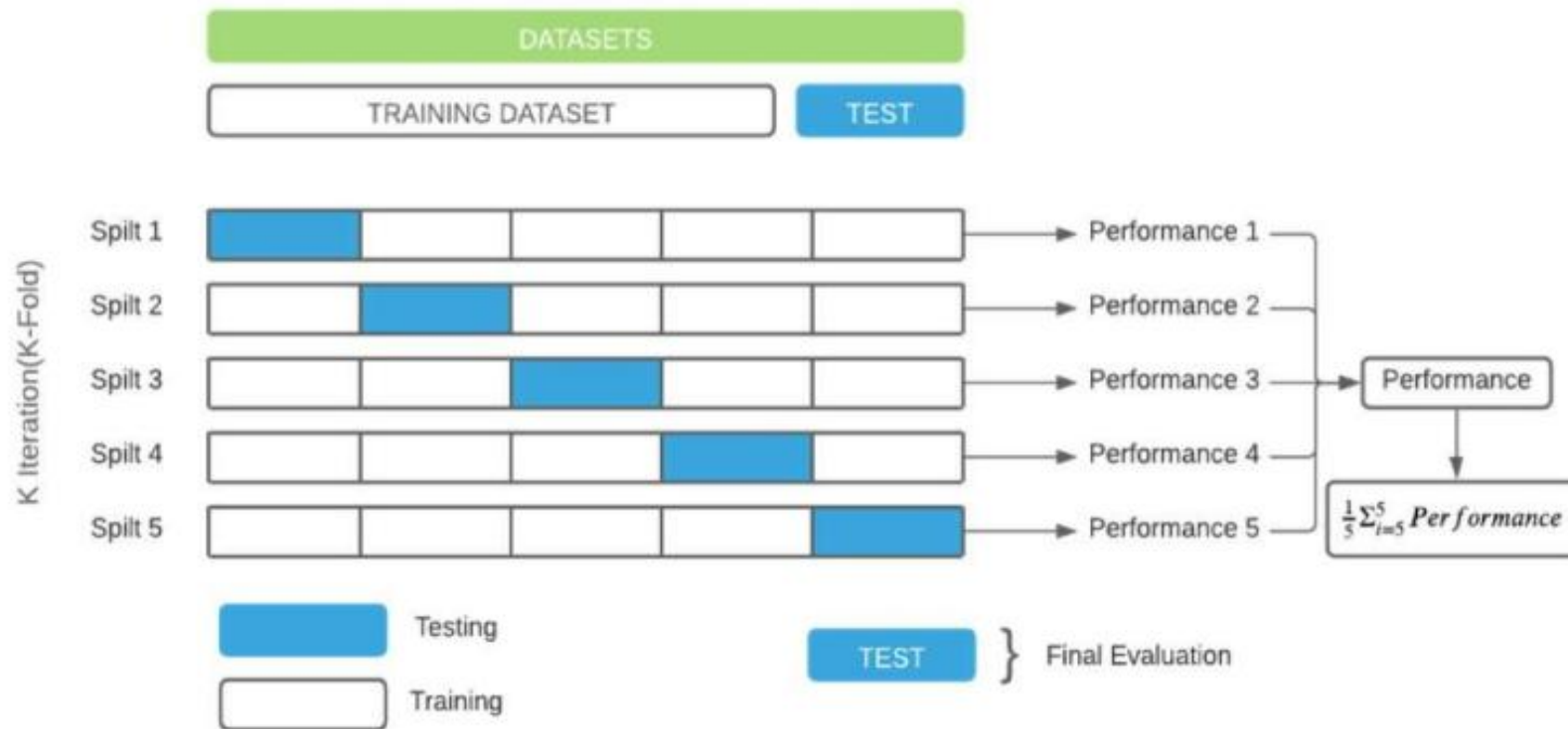


Figure: Five-fold cross validation

4.4.5. Estimators, Bias and Variance

Estimator

- An estimator is a mathematical rule or formula used to estimate an unknown population parameter from sample data.
- In simple words, Estimator = Formula & Estimate = Numerical value produced by the formula.
- Suppose five students score: 70, 75, 80, 85, 90
- The estimator for population mean is $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$
- The estimate is: $\bar{x} = \frac{70+75+80+85+90}{5} = 80$.
- In machine learning the model itself acts as an estimator.
- Examples: Linear Regression estimates regression coefficients, Decision Trees estimate decision boundaries & Neural Networks estimate weights and biases.
- The quality of these estimates depends on bias and variance.

Bias Vs Variance

- Bias and variance are two of the most important concepts in machine learning because they explain why a model performs well or poorly. They also explain the causes of underfitting and overfitting.
- A useful way to think about them is:
- Bias measures how wrong the model's assumptions are.
- Variance measures how sensitive the model is to changes in the training data.
- Bias measures accuracy.
- Variance measures consistency.

Bias

- Bias measures how far the model's predictions are from the true relationship due to oversimplification.
- A high-bias model cannot capture the complexity of the data.
- Characteristics of High Bias:
 - Model is too simple.
 - Misses important patterns.
 - High training error.
 - High testing error.
 - Causes underfitting.

Bias

- Example: Let us suppose that the True average salary of employees in a company is Rs. 50,000.
- Using any machine learning model if the model repeatedly estimates the average salary as: 46,000, 47,000, 48,000, 47,000 & 46,000
- Then, the Average estimate becomes 46,800.
- Now, Bias: $46,800 - 50,000 = -3,200$
- The estimator consistently underestimates the true value.

Variance

- Variance measures how sensitive the model is to small changes in the training data.
- A high-variance model memorizes the training data instead of learning the true pattern.
- Characteristics of High Variance:
 - Model is very complex.
 - Learns noise.
 - Very low training error.
 - High testing error.
 - Causes overfitting.

Variance

- Example: Let us suppose we are using two different machine learning model to estimate the average height of students using different random samples.
- In multiple iteration, Model A estimates average height as: 169, 170, 171, 170, 170
- It shows model is very stable and there is very low variance.
- However, Model B estimates average height as: 155, 180, 172, 165 & 178
- It shows the estimates are highly variable & there is high variance. Hence, causing the model to overfit.

Relationship with Underfitting and Overfitting

Model	Bias	Variance
Underfitting	High	Low
Good Fit	Moderate	Moderate
Overfitting	Low	High

Bias–Variance Tradeoff

- Increasing model complexity usually reduces bias because the model can learn more complex patterns. However, excessive complexity can increase variance, making the model sensitive to noise in the training data.
- Conversely simplifying the model usually reduces variance because it behaves more consistently across different datasets. But if the model becomes too simple, bias increases because it cannot capture the true relationship.
- This tradeoff is called the bias–variance tradeoff.
- The optimal model is found where the total prediction error is minimized, balancing both bias and variance.

4.4.6. Review of MLE & MAP and Cross Entropy

- Machine learning models learn by estimating the best values of their parameters (such as weights and biases) from the available training data.
- A natural question is: How does a machine determine which parameter values are the "best"?
- Statistical estimation provides the answer.
- Two widely used estimation techniques are:
 - Maximum Likelihood Estimation (MLE)
 - Maximum A Posteriori Estimation (MAP)
- Once the parameters are estimated, we also need a way to measure how well the model predicts the true labels. One of the most important measures used in classification is Cross Entropy Loss.

Probability vs Likelihood

- Suppose we toss a coin 10 times and observe: H H H H H H H H T T
- Observed data: Heads = 8 & Tails = 2
- Now let us consider three possible values of the probability of heads:
- $\theta = 0.5$, $\theta = 0.7$ & $\theta = 0.8$
- Question here is “Which value of θ most likely generated the observed data?” It is addressed by likelihood.
- The Likelihood Function measures how likely the observed data is for different parameter values.
- For independent observations, $L(\theta) = P(D | \theta)$ where; D = observed data & θ = model parameters
- The goal is to find the parameter value that maximizes the likelihood.

Probability vs Likelihood

Probability	Likelihood
Parameters are fixed; data is unknown	Data is fixed; parameters are unknown
Predicts the chance of observing data	Evaluates how well parameters explain the observed data
Predicts the chance of observing data	Example: Which coin is most likely to have generated the observed tosses?

Maximum Likelihood Estimation (MLE)

- Maximum Likelihood Estimation chooses the parameter values that make the observed training data most likely.
- Mathematically, $\hat{\theta}_{MLE} = \arg \max_{\theta} P(D | \theta)$ where; $P(D | \theta)$ is the likelihood.
- Suppose we toss a coin 10 times and obtain: Heads = 8 & Tails = 2.
- The likelihood of observing this result is: $L(\theta) = \theta^8(1 - \theta)^2$
- Evaluate the likelihood for several values of θ .

θ	Likelihood
0.5	0.00098
0.6	0.00269
0.7	0.00545
0.8	0.00671
0.9	0.00387

- The likelihood is highest when $\theta = 0.8$.

Therefore,

$$\hat{\theta}_{MLE} = 0.8$$

- Based only on the observed data, the best estimate of the probability of heads is 0.8.

Maximum A Posteriori (MAP) Estimation

- MAP entails introducing a prior over the parameters and is actually equivalent to finding the maximum of the posterior distribution.
- Suppose we already have prior knowledge about the parameter before observing new data.
- Example: In previous experiments let us suppose that a coin is likely to be fair (i.e. Head=50% & Tail =50%). Should we ignore this information?
- The answer is No. MAP estimation combines Observed data * Prior knowledge.
- MAP is based on Bayes' theorem. $P(\theta | D) = \frac{P(D|\theta) P(\theta)}{P(D)}$ where;

$P(D | \theta)$ = likelihood

$P(\theta)$ = prior probability

$P(\theta | D)$ = posterior probability

- The objective of MAP is to estimate parameter to maximize the posterior probability.

$$\hat{\theta}_{MAP} = \arg \max_{\theta} P(\theta | D)$$

- Since $P(D)$ is constant with respect to θ , $\hat{\theta}_{MAP} = \arg \max_{\theta} P(D | \theta) P(\theta)$

Cross Entropy

- Entropy measures the uncertainty or amount of information in a random variable.
- Examples: A fair coin (50% Heads, 50% Tails) has high uncertainty. A coin that always lands Heads has zero uncertainty. Higher uncertainty implies higher entropy.
- For a discrete probability distribution, $H(P) = -\sum_i P(i) \log P(i)$
- Cross Entropy measures how different the predicted probability distribution is from the true distribution.
- If the predicted probabilities closely match the true labels, Cross Entropy is small.
- If they differ significantly, Cross Entropy is large.
- For classification, $CE = -\sum_i y_i \log(\hat{y}_i)$ where; y_i = true label & $\hat{y}_i$ = predicted probability

- Suppose we are classifying an image as either Cat or Dog.
- True label: Cat = 1 & Dog = 0
- Model A: Predicted probabilities: Cat = 0.90 & Dog = 0.10 .
- Cross Entropy $CE = -\log(0.90) \approx 0.105$. Very small loss because the model is confident and correct.
- Model B: Predicted probabilities: Cat = 0.40 & Dog = 0.60
- Cross Entropy $CE = -\log(0.40) \approx 0.916$. Larger loss because the model assigns a low probability to the correct class.
- Model C: Predicted probabilities: Cat = 0.01 & Dog = 0.99
- Cross Entropy $CE = -\log(0.01) \approx 4.605$. Very large loss because the model is confidently wrong.
- Compared to simple classification accuracy, Cross Entropy considers prediction confidence.

4.5 Supervised Learning Algorithm: Decision Tree and Support Vector Machine

Decision Tree

- AI systems using supervised learning method learns by example.
- Systems learning by example can be viewed as searching a concept space by means of a decision tree.
- The best-known approach to constructing a decision tree is called **ID3 (Iterative Dichotomizer 3)** developed by J. Ross Quinlan in 1975)

Decision Trees

- a decision tree consists of:

Nodes:

- test for the value of a certain attribute

Edges:

- correspond to the outcome of a test
- connect to the next node or leaf

Leaves:

- terminal nodes that predict the outcome

Decision Tree Learning

- In Decision Tree Learning, a new example is classified by submitting it to a series of tests that determine the class label of the example.
- These tests are organized in a hierarchical structure called a *decision tree*.
- The training examples are used for choosing appropriate tests in the decision tree.
- Typically, a tree is built from top to bottom, where tests that maximize the information gain about the classification are selected first.

ID3

- ID3 is an algorithm used in decision tree learning to generate a decision tree.
- Decision tree consists of decision nodes and leaf nodes connected by arcs.
- ID3 builds the tree from the top to down.
- Entropy or Information gain is used to select the most useful attribute for classification.

$$\text{Entropy}(H) = - \sum (p_i) \log_2(p_i)$$

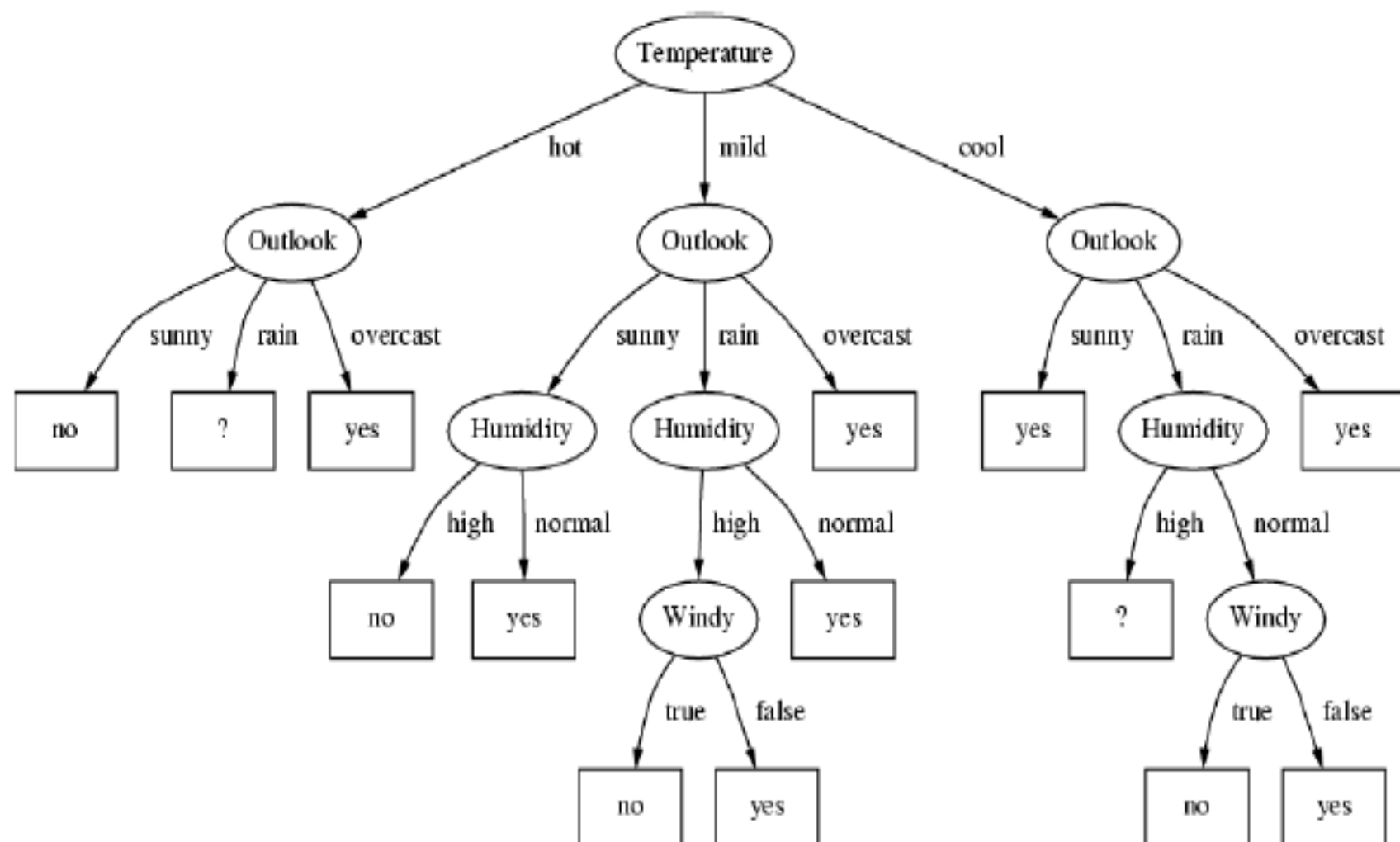
- Entropy is the basis of Information Theory.
- Entropy is a measure of randomness, hence the smaller the entropy the greater the information content.

ID3 algorithm

- Create a root node for tree
- If all examples are positive, then create a positive node and stop.
- If all examples are negative, then create a negative node and stop
- Otherwise
 - Calculate entropy, information gain to select root node and branch node.(nodes with highest information gain or minimum entropy is selected as root node).
 - Partition the examples into subset.
 - Repeat until all examples are classified.

A Sample Task

<i>Day</i>	<i>Temperature</i>	<i>Outlook</i>	<i>Humidity</i>	<i>Windy</i>	<i>Play Golf?</i>
07-05	hot	sunny	high	false	no
07-06	hot	sunny	high	true	no
07-07	hot	overcast	high	false	yes
07-09	cool	rain	normal	false	yes
07-10	cool	overcast	normal	true	yes
07-12	mild	sunny	high	false	no
07-14	cool	sunny	normal	false	yes
07-15	mild	rain	normal	false	yes
07-20	mild	sunny	normal	true	yes
07-21	mild	overcast	high	true	yes
07-22	hot	overcast	normal	false	yes
07-23	mild	rain	high	true	no
07-26	cool	rain	normal	true	no
07-30	mild	rain	high	false	yes



→ Solⁿ

$$\log_a(b) = \frac{\log_n(b)}{\log_n(a)}$$

Play Golf		Temperature			Outlook			Humidity		Windy	
		Hot	Cool	Mild	Sunny	Rainy	Overcast	High	Normal	False	True
Yes	9	2	3	4	3	3	4	3	6	6	3
No	5	2	1	2	2	2	0	4	1	2	3
Total	14	4	4	6	5	5	4	7	7	8	6

Entropy for play golf $E(S)$.

$$\begin{aligned} E(S) &= -P_{\text{yes}} \log_2 P(\text{yes}) - P_{\text{no}} \log_2 P(\text{no}) \\ &= -\frac{9}{14} \log_2 \left(\frac{9}{14}\right) - \frac{5}{14} \log_2 \left(\frac{5}{14}\right) \\ &= 0.9402 \end{aligned}$$

Entropy for Temperature

For Hot:-

$$\begin{aligned} E(\text{Hot}) &= -\frac{2}{4} \log_2 \left(\frac{2}{4}\right) - \frac{2}{4} \log_2 \left(\frac{2}{4}\right) \\ &= 1 \end{aligned}$$

$$\begin{aligned} E(\text{Cool}) &= -\frac{3}{4} \log_2 \left(\frac{3}{4}\right) - \frac{1}{4} \log_2 \left(\frac{1}{4}\right) \\ &= 0.81127 \end{aligned}$$

$$\begin{aligned} E(\text{Mild}) &= -\frac{4}{6} \log_2 \left(\frac{4}{6}\right) - \frac{2}{6} \log_2 \left(\frac{2}{6}\right) \\ &= 0.9182 \end{aligned}$$

Average Entropy for temperature:-

$$\begin{aligned} I(\text{temp}) &= \frac{4}{14} \times 1 + \frac{4}{14} \times 0.81127 + \frac{6}{14} \times 0.9182 \\ &= 0.911 \end{aligned}$$

Information Gain for Temperature:

$$\begin{aligned} \text{Gain}(\text{temp}) &= E(S) - I(\text{temp}) \\ &= 0.9402 - 0.911 \\ &= 0.0292 \end{aligned}$$

Now, Entropy for Outlook:

$$E(\text{Sunny}) = -\frac{3}{5} \log_2(3/5) - \frac{2}{5} \log_2(2/5) \\ = 0.9709$$

$$E(\text{Rainy}) = -\frac{2}{5} \log_2(2/5) - \frac{3}{5} \log_2(3/5) \\ = 0.9709$$

$$E(\text{Overcast}) = -\frac{4}{4} \log_2(4/4) - 0 \log_2(0) \\ = 0$$

Average Entropy for Outlook:-

$$I(\text{Outlook}) = \frac{5}{14} \times 0.9709 + \frac{5}{14} \times 0.9709 + \frac{4}{14} \times 0 \\ = 0.6935$$

Information Gain for Outlook:

$$\text{Gain}(\text{Outlook}) = E(S) - I(\text{Outlook}) \\ = 0.9402 - 0.6935 \\ = 0.2467$$

Entropy for Humidity

$$E(\text{High}) = -\frac{3}{7} \log_2(3/7) - \frac{4}{7} \log_2(4/7) \\ = 0.985$$

$$E(\text{Normal}) = -\frac{6}{7} \log_2(6/7) - \frac{1}{7} \log_2(1/7) \\ = 0.5916$$

Average Entropy for Humidity:

$$I(\text{Humidity}) = \frac{7}{14} \times 0.985 + \frac{7}{14} \times 0.5916$$
$$= 0.7883$$

Information Gain for Humidity :-

$$\text{Gain}(\text{Humidity}) = E(S) - I(\text{Humidity})$$
$$= 0.9402 - 0.7883$$
$$= 0.1519$$

Entropy for Windy:-

$$E(\text{Windy}) =$$
$$E(\text{false}) = -\frac{6}{8} \log_2(6/8) - \frac{2}{8} \log_2(2/8)$$
$$= 0.8117$$

$$E(\text{True}) = -\frac{3}{6} \log_2(3/6) - \frac{3}{6} \log_2(3/6)$$
$$= 1$$

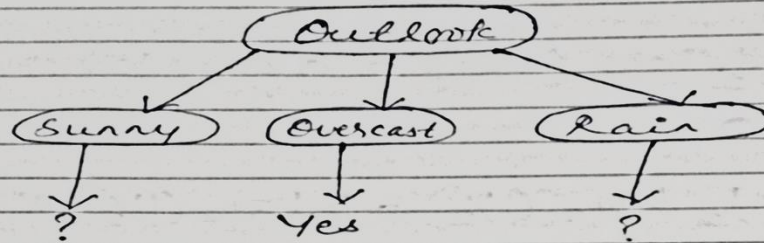
Average Entropy for Windy:-

$$I(\text{Windy}) = \frac{8}{14} \times 0.8117 + \frac{6}{14} \times 1$$
$$= 0.8924$$

Information Gain for Windy:-

$$\text{Gain}(\text{Windy}) = E(S) - I(\text{Windy})$$
$$= 0.9402 - 0.8924$$
$$= 0.0478$$

Since, entropy of information gain of outlook is max^m among the given attributes. So, the root node is outlook & it is expressed as:-



Now, for selecting the leaf node of outlook, the overcast has all condⁿ as 'yes' so it concludes that if overcast is selected so surely plays golf.

But when sunny or rainy is selected the next condⁿ are to be calculated for confidentiality. Now, for selecting 2nd node the table can be constructed as:-

	Sunny	Temperature			Humidity		Windy	
		Hot	Cold	Mild	High	Normal	False	True
Yes	3	0	1	1	0	2	1	1
No	2	2	0	1	3	0	2	1
	5	2	1	2	3	2	3	2

Entropy for Sunny:-

$$\begin{aligned}
 E(\text{Sunny}) &= -\frac{3}{5} \log_2 \left(\frac{3}{5} \right) - \frac{2}{5} \log_2 \left(\frac{2}{5} \right) \\
 &= 0.9709
 \end{aligned}$$

Entropy for Temperature with respect to Sunny:-
 $E(\text{Hot}) = -0 \log_2(0) - \frac{2}{2} \log_2(2/2)$
 $= 0$

$E(\text{Cool}) = -\frac{1}{1} \log_2(1/1) - 0 \log_2(0)$
 $= 0$

$E(\text{Mild}) = -\frac{1}{2} \log_2(1/2) - \frac{1}{2} \log_2(1/2)$
 $= 1$

Average Entropy for Temperature:-
 $I(\text{Temp}) = 0 + 0 + \frac{2}{3} \times 1$
 $= 0.4$

Information Gain for Temp w.r.t Sunny:-
 $\text{Gain}(\text{Temp}) = E(\text{Sunny}) - I(\text{Temp})$
 $= 0.9709 - 0.4$
 $= 0.5709$

Entropy for Humidity w.r.t Sunny
 $E(\text{High}) = -0 \log_2(0) - \frac{3}{3} \log_2(3/3)$
 $= 0$

$E(\text{Normal}) = -\frac{2}{2} \log_2(2/2) - 0 \log_2(0)$
 $= 0$

$I(\text{Humidity}) = 0 + 0 = 0$

$$\begin{aligned} \text{Gain}(\text{Humidity}) &= E(\text{Sunny}) - I(\text{Humidity}) \\ &= 0.9709 - 0 \\ &= 0.9709 \end{aligned}$$

Entropy for Windy w.r.t Sunny:

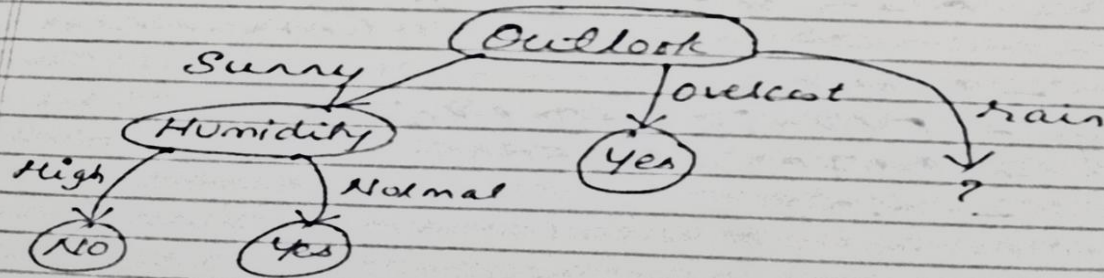
$$\begin{aligned} E(\text{True}) \cdot E(\text{False}) &= -\frac{1}{3} \log_2(1/3) - \frac{2}{3} \log_2(2/3) \\ &= 0.9182 \end{aligned}$$

$$\begin{aligned} E(\text{True}) &= -\frac{1}{2} \log_2(1/2) - \frac{1}{2} \log_2(1/2) \\ &= 1 \end{aligned}$$

$$I(\text{Windy}) = \frac{3}{5} \times 0.9182 + \frac{2}{5} \times 1 = 0.95092$$

$$\begin{aligned} \text{Gain}(\text{Windy}) &= E(\text{Sunny}) - I(\text{Windy}) \\ &= 0.9709 - 0.95092 \\ &= 0.01998 \end{aligned}$$

Since information gain of humidity w.r.t Sunny is max^m among the given attributes. So, the next node is humidity for sunny i.e.



From the table, when the humidity is high, the golf is not played & if normal yes played.
Now, for Rain option we have to maintain table as:-

Rain		Temperature			Windy	
		Hot	Cold	Mild	High	Low
Yes	2	0	1	2	3	0
No	3	0	1	1	0	2
	5	0	2	3	3	2

Entropy for Rain:-

$$E(\text{Rain}) = -\frac{2}{5} \times \log_2(2/5) - \frac{3}{5} \log_2(3/5)$$

$$= 0.9709$$

Entropy for Temp² w.r.t Rain:-

$$E(\text{Hot}) = 0$$

$$E(\text{Cold}) = -\frac{1}{2} \log_2(1/2) - \frac{1}{2} \log_2(1/2) = 1$$

$$E(\text{Mild}) = -\frac{2}{3} \log_2(2/3) - \frac{1}{3} \log_2(1/3) = 0.9182$$

$$I(\text{Temp}) = 0 + \frac{2}{5} \times 1 + \frac{3}{5} \times 0.9182$$

$$= 0.9509$$

$$\text{Gain}(\text{Temp}) = E(\text{Rain}) - I(\text{Rain})$$

$$= 0.9709 - 0.9509$$

$$= 0.02$$

Again, Entropy for windy with Rain

$$E(\text{false}) = -\frac{2}{3} \log_2 \left(\frac{2}{3} \right) - 0 \log_2 (0)$$

$$= 0$$

$$E(\text{True}) = -0 \log_2 (0) - \frac{2}{2} \log_2 \left(\frac{2}{2} \right)$$

$$= 0$$

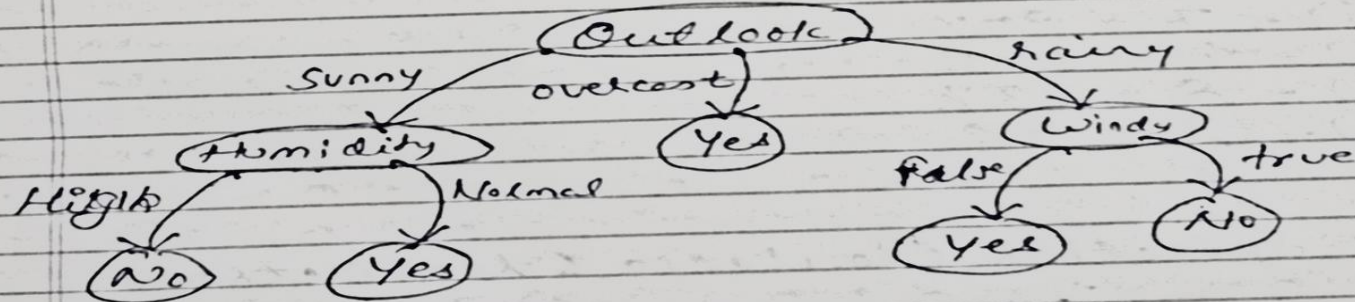
$$I(\text{Windy}) = 0$$

$$\text{Gain}(\text{Windy}) = E(\text{Rain}) - I(\text{Windy})$$

$$= 0.9709 - 0$$

$$= 0.9709$$

Since gain of windy with rain is max^m among other attributes. So next node is windy for rain.



Hence, the best decision tree //

4.5 Supervised Learning Algorithm: Decision Tree and Support Vector Machine

Support Vector Machine

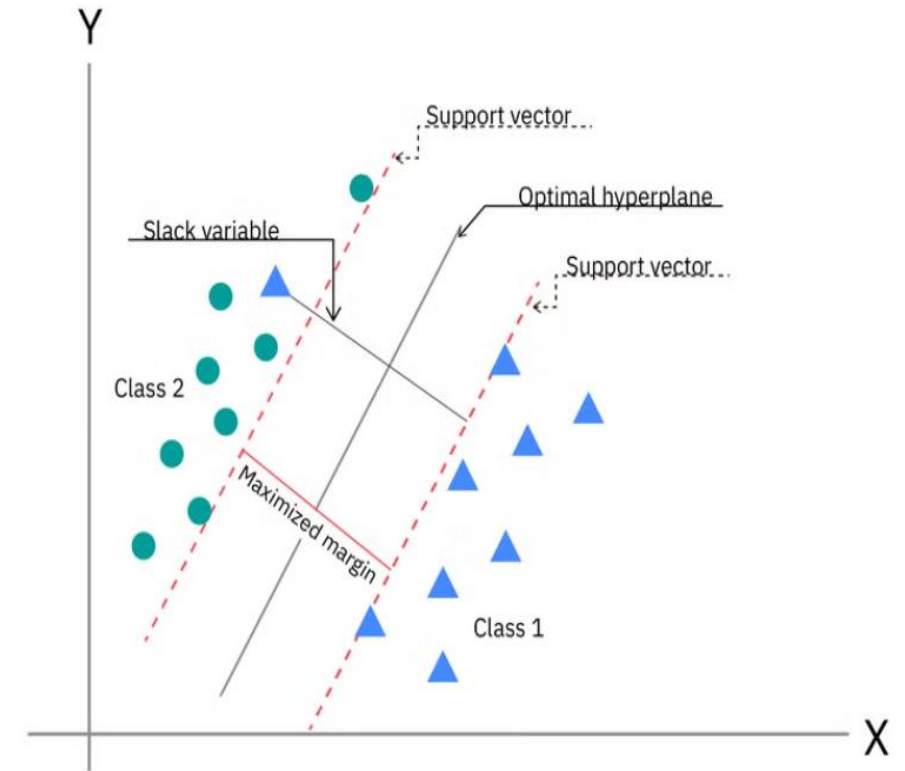
- A Support Vector Machine (SVM) is a popular supervised machine learning algorithm used for classification, regression, and outlier detection.
- It works by finding the optimal "hyperplane" (a decision boundary) that maximizes the distance between different data classes in an N-dimensional space.
- It was introduced by Vladimir Vapnik in the 1990s.
- Unlike Decision Trees , SVM tries to find the best possible decision boundary that separates different classes.

4.5 Supervised Learning Algorithm: Decision Tree and Support Vector Machine

Core Concepts

- Hyperplane

- A hyperplane is a decision boundary that separates the data points of different classes in the feature space.
- For two-dimensional data, it is a line: $w_1x_1 + w_2x_2 + b = 0$ where; w = weight vector & b = bias.
- While it is a plane in a n -dimensional space.
- Let us suppose we want to classify animals into cats (+1) and dogs (-1) based on weight & height then a hyperplane could be: $3 * \text{Weight} + 2 * \text{Height} - 50 = 0$.



4.5 Supervised Learning Algorithm: Decision Tree and Support Vector Machine

- Margin

- The margin is the distance between the hyperplane and the nearest data points of the either class.
- SVM tries to maximize this margin to improve classification confidence employing Maximum Margin Hyperplane.

- Support Vectors

- These are the data points that are closest to the hyperplane or the exact data points that lie directly on margin lines.
- These points influence the orientation & position of the hyperplane.

4.5 Supervised Learning Algorithm: Decision Tree and Support Vector Machine

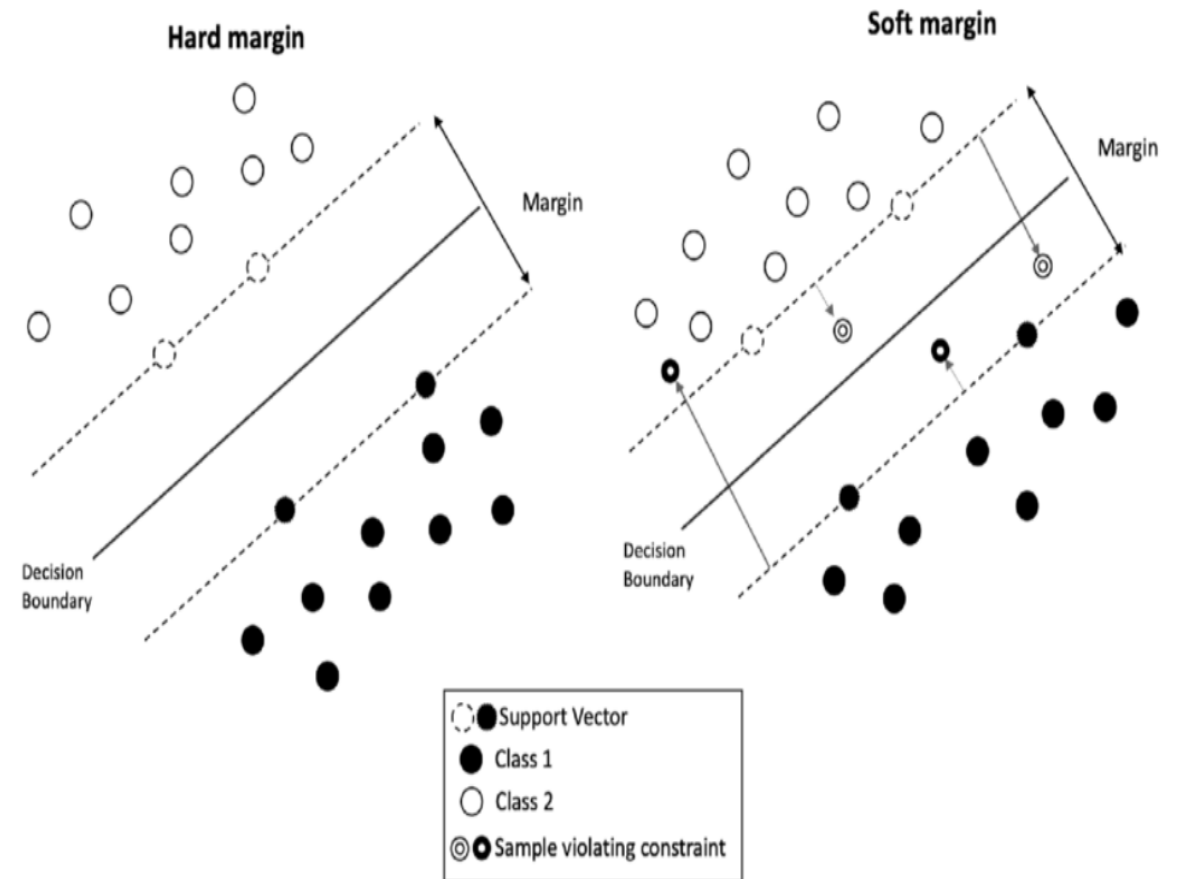
- Hard Margin

- Assumes data is perfectly separable by the hyperplane. All points should lie outside the margin.

- It has zero tolerance for misclassifications and fails if even a single noisy point cannot be cleanly separated.

- Soft Margin

- Allows some misclassification or margin violations for non-linearly separable data.



4.5 Supervised Learning Algorithm: Decision Tree and Support Vector Machine

- Kernel Function

➤ A function that transforms data into a higher dimensional space to make it linearly separable.

Types of Kernels:

1. Linear Kernel=Suitable for linearly separable data
2. Polynomial Kernel=For curved boundaries
3. Radial Basis Function Kernel=Captures complex relationships

4.5 Supervised Learning Algorithm: Decision Tree and Support Vector Machine

- Mathematical Foundation of Support Vector Machine:
- If we are given training data: $\{(x_i, y_i)\}_{i=1}^n, x_i \in \mathbb{R}^d, y_i \in (-1, +1)$.
- We want to find a hyperplane: $w \cdot x + b = 0$ where $w \in \mathbb{R}^d$ is the weight vector (normal to the hyperplane) and b is the bias.
- The classifier is then: $f(x) = \text{sign}(w \cdot x + b)$ where

$$\text{sign}(z) = \begin{cases} +1, & \text{if } z > 0 \\ 0, & \text{if } z = 0 \\ -1, & \text{if } z < 0 \end{cases}$$

where $z = w \cdot x + b$.

- The expression: $w \cdot x + b$ calculates which side of the hyperplane a point lies on.
- If $w \cdot x + b > 0$, the point is on the positive side of the hyperplane, so SVM predicts Class +1.
- If $w \cdot x + b < 0$, the point is on the negative side, so SVM predicts Class -1.
- If $w \cdot x + b = 0$, the point lies exactly on the decision boundary (hyperplane).

4.5 Supervised Learning Algorithm: Decision Tree and Support Vector Machine

- Algorithm

Step 1: Draw all the data points on plane.

Step 2: Find the support vectors.

Step 3: Add the bias (b) value in vectors to convert 2D to 3D form.

Step 4: Consider parameters for every support vectors like α_1 , α_2 , etc.

Step 5: Form the linear equations and solve then to find parameters value.

Step 6: Find the final equation for the hyperplane.

Step 7: Express in general form $w_1x_1 + w_2x_2 + b = 0$ with obtained w & b to find the decision boundary.

Step 8: Calculate the margin.

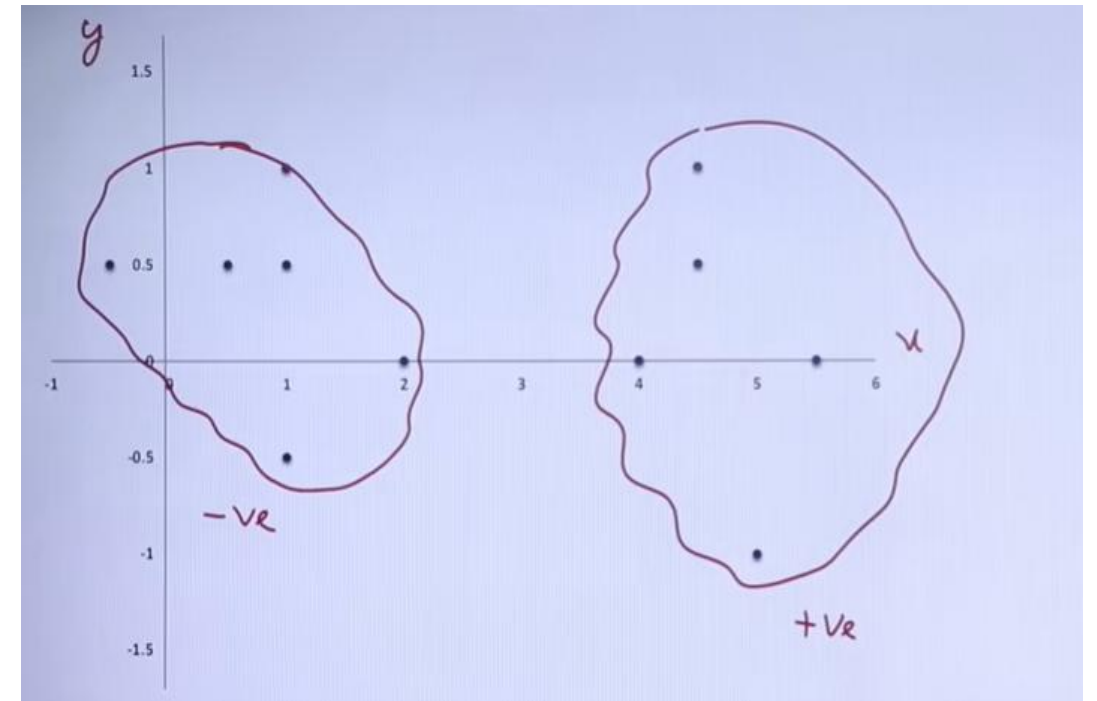
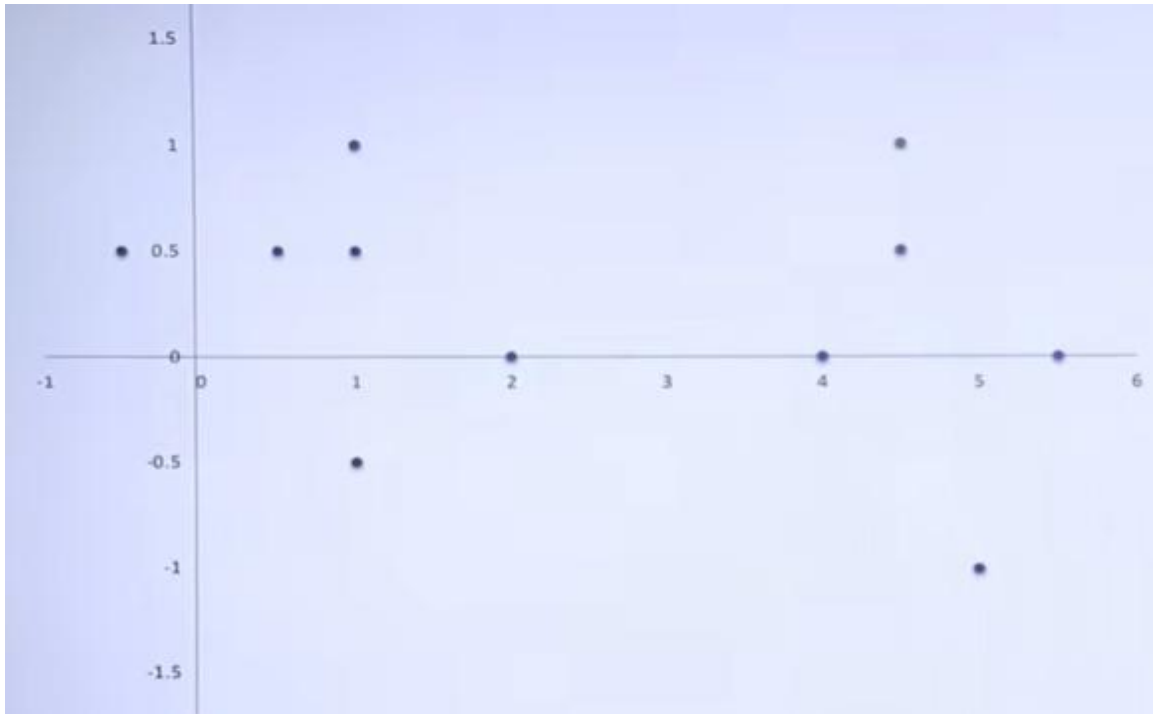
Step 9: Classify new data points.

Support Vector Machine: Example 1

- Example 1: Consider the following set of data points given, find the optimal hyperplane for the data points.

$(1, 0.5), (1, 1), (1, -0.5), (-0.5, 0.5), (0.5, 0.5), (2, 0), (4, 0), (4.5, 1), (4.5, 0.5), (5, -1), (5.5, 0)$.

➤ Step 1: Draw all the points on the plane.



Step 2: Find Support Vectors

Support Vectors: $(2 \times 1) + 0 \times 0 + 1 \times 6 = 6$

$$S_1 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad S_2 = \begin{pmatrix} 4 \\ 0 \end{pmatrix}$$

Step 3: Add bias values to vectors.

$$S_1 = \begin{pmatrix} 2 \\ 0 \\ 1 \end{pmatrix}, \quad S_2 = \begin{pmatrix} 4 \\ 0 \\ 1 \end{pmatrix}$$

Step 4: Consider parameters for every vectors:

α_1 for S_1 & α_2 for S_2

Step 5: Form linear equations & then solve to find parameters value.

$$\alpha_1 \cdot S_1 \cdot S_1 + \alpha_2 \cdot S_2 \cdot S_1 = -1$$
$$\text{or, } \alpha_1 \cdot \begin{pmatrix} 2 \\ 0 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ 0 \\ 1 \end{pmatrix} + \alpha_2 \cdot \begin{pmatrix} 4 \\ 0 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ 0 \\ 1 \end{pmatrix} = -1$$

$$\text{or, } \alpha_1 \cdot (2 \times 2 + 0 \times 0 + 1 \times 1) + \alpha_2 \cdot (4 \times 2 + 0 \times 0 + 1 \times 1) = -1$$

$$\text{or, } 5\alpha_1 + 9\alpha_2 = -1 \quad \text{--- (i)}$$

$$\alpha_1 \cdot S_1 \cdot S_2 + \alpha_2 \cdot S_2 \cdot S_2 = +1$$
$$\text{or, } \alpha_1 \cdot \begin{pmatrix} 2 \\ 0 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 4 \\ 0 \\ 1 \end{pmatrix} + \alpha_2 \cdot \begin{pmatrix} 4 \\ 0 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 4 \\ 0 \\ 1 \end{pmatrix} = 1$$

$$\text{or, } \alpha_1 (2 \times 4 + 0 \times 0 + 1 \times 1) + \alpha_2 (4 \times 4 + 0 \times 0 + 1 \times 1) = 1$$

$$\text{or, } 9\alpha_1 + 17\alpha_2 = 1 \quad \text{--- (ii)}$$

$$\text{Now, from eqn (i), } \alpha_1 = \frac{-1 - 9\alpha_2}{5}$$

Substituting value of α_1 in eqn (ii) we get.

$$9 \left(\frac{-1 - 9\alpha_2}{5} \right) + 17\alpha_2 = 1$$

$$\therefore \alpha_2 = 3.5 \quad \& \quad \alpha_1 = -6.5$$

Mo Tu We Th Fr Sa Su

Step 6: Find equation for hyperplane.

$$\bar{w} = \sum_{i=1}^n \alpha_i S_i$$

or, $\bar{w} = \alpha_1 S_1 + \alpha_2 S_2$

or, $\bar{w} = -6.5 \times \begin{pmatrix} 2 \\ 0 \\ 1 \end{pmatrix} + 3.5 \begin{pmatrix} 4 \\ 0 \\ 1 \end{pmatrix}$

or, $\bar{w} = \begin{pmatrix} -13 \\ 0 \\ -6.5 \end{pmatrix} + \begin{pmatrix} 14 \\ 0 \\ 3.5 \end{pmatrix}$

$\therefore \bar{w} = \begin{pmatrix} 1 \\ 0 \\ -3 \end{pmatrix}$

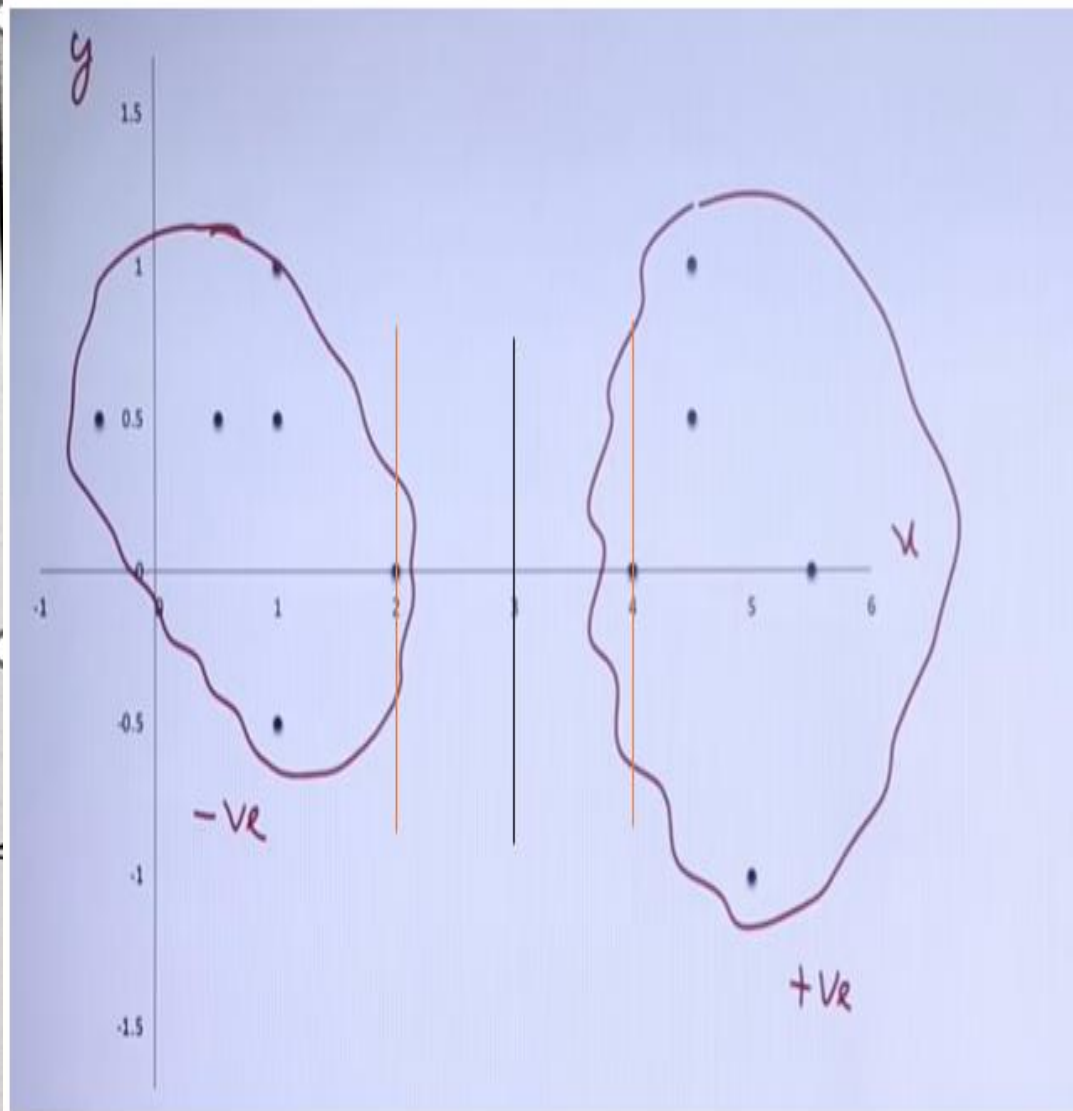
here $b = -3$ now eqn becomes $b + 3 = 0$;

$(1, 0)$ suggests the orientation of the hyperplane

If $(1, 0) \rightarrow$ hyperplane will be vertical.

If $(0, 1) \rightarrow$ hyperplane will be horizontal.

If $(1, 1) \rightarrow$ hyperplane will be drawn at an angle of 45° .



Now, general equation $w_1x_1 + w_2x_2 + b = 0$ with $w = (1, 0)$ & $b = -3$ we get,

- Decision function: $f(x) = 1(x_1) + 0(x_2) - 3$
or, $f(x) = x_1 - 3$

Now, classify each point given in the question as:

(1, 0.5): $f(x) = x_1 - 3 = 1 - 3 = -2$, -ve class

(1, 1): $f(x) = x_1 - 3 = 1 - 3 = -2$, -ve class

(1, -0.5): $f(x) = x_1 - 3 = 1 - 3 = -2$, -ve class

(-0.5, 0.5): $f(x) = x_1 - 3 = -0.5 - 3 = -3.5$, -ve class

(0.5, 0.5): $f(x) = x_1 - 3 = 0.5 - 3 = -2.5$, -ve class

(2, 0): $f(x) = x_1 - 3 = 0 - 3 = -3$, -ve class

(4, 0): $f(x) = x_1 - 3 = 4 - 3 = 1$, +ve class

(4.5, 1): $f(x) = x_1 - 3 = 4.5 - 3 = 1.5$, +ve class

(4.5, 0.5): $f(x) = x_1 - 3 = 4.5 - 3 = 1.5$, +ve class

(5, -1): $f(x) = x_1 - 3 = 5 - 3 = 2$, +ve class

(5.5, 0): $f(x) = x_1 - 3 = 5.5 - 3 = 2.5$, +ve class

Margin Calculation

Positive Support Vector = (4, 0) & Negative Support Vector = (2, 0).

Positive support vectors: $x=4$

Negative support vectors: $x=2$

Decision boundary: $x=3$

Distance from boundary to each support vector: 1

Total margin: 2

- The geometric margin of an SVM hyperplane is given by:

$$\text{Margin} = \frac{2}{\|w\|}$$

For our classifier:

$$w = (1, 0)$$

The Euclidean norm is

$$\|w\| = \sqrt{0^2 + 1^2} = 1$$

Therefore,

$$\text{Margin} = \frac{2}{1} = 2$$

This agrees with the geometric observation: the support vectors lie one unit above and one unit below the hyperplane, giving a total margin of 2 units.

- Test for New Data Points:

1. (5.5, 1)

$$f(x) = x_1 - 3$$

$$= 5.5 - 3$$

$$= 2.5, \text{ +ve class}$$

2. (3, 0)

$$f(x) = x_1 - 3$$

$$= 3 - 3$$

$$= 0, \text{ on the hyperplane}$$

3. (2, -1)

$$f(x) = x_1 - 3$$

$$= 2 - 3$$

$$= -1, \text{ -ve class}$$

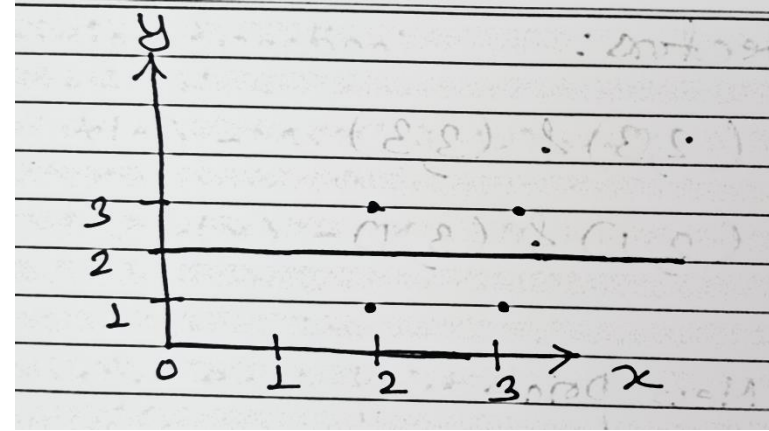
Support Vector Machine: Example 2

- Suppose we have the following training data.

Sample	X1	X2	Class
A	2	3	+1
B	3	3	+1
C	2	1	-1
D	3	1	-1

Step 1: Plot the points:

- Positive class: $(2,3)$, $(3,3)$
- Negative class: $(2,1)$, $(3,1)$
- The classes are separated horizontally.



Step 2: Find Hyperplane:

- Positive class lies around $y = 3$
- Negative class lies around $y = 1$
- Middle line $y = 2$
- Hence, Decision Boundary: $x_2 = 2$

Step 3: Express in Standard Form:

- General equation: $w_1x_1 + w_2x_2 + b = 0$
- Since, $x_2 = 2$ rewrite as $x_2 - 2 = 0$
- Therefore, $w = (0,1)$ & $b = -2$
- Decision function: $f(x) = 0(x_1) + 1(x_2) - 2$ or $f(x) = x_2 - 2$

Step 4: Classify Each Point:

- Point A (2,3): $f(x) = 3 - 2 = 1$, Positive & Correct
- Point B (3,3): $3 - 2 = 1$, Positive & Correct
- Point C (2,1): $1 - 2 = -1$, Negative & Correct
- Point D (3,1): $1 - 2 = -1$, Negative & Correct

Step 5: Margin Calculation

- Positive support vectors: $y=3$
- Negative support vectors: $y=1$
- Decision boundary: $y=2$
- Distance from boundary to each support vector 1. Therefore, Total margin = 2.

Step 6: Classify a New Point

- $P(4, 2.8)$: Decision function = $f(x) = 2.8 - 2 = 0.8$, Positive
- Hence, Class = +1
- $P(5, 1.3)$: $f(x) = 1.3 - 2 = -0.7$, Negative
- Hence, Class = -1

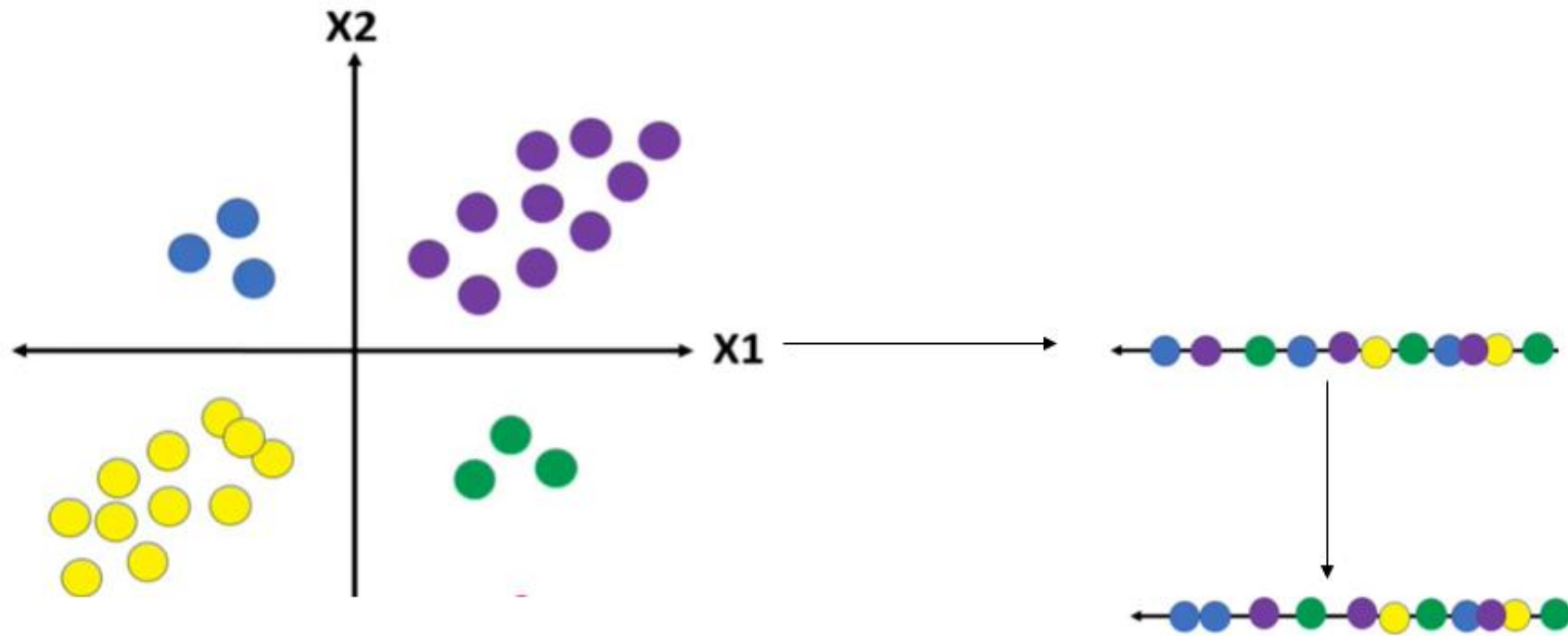
4.6 Unsupervised Learning Algorithm: t-SNE

- t-SNE (t-Distributed Stochastic Neighbor Embedding) is a nonlinear dimensionality reduction algorithm primarily used for visualization.
- It converts High-dimensional data into 2D or 3D representations while keeping similar points close together.
- t-SNE tries to preserve local neighborhood structure rather than global variance.
- Instead of preserving exact distances, t-SNE preserves probability of being neighbors.
- If two points are close in high-dimensional space, they should also have a high probability of being close after low dimensional projection.

4.6 Unsupervised Learning Algorithm: t-SNE

- The t-SNE algorithm executes in two main steps:
- High-Dimensional Probability Map
 - It calculates the distance between data points in the original high-dimensional space and converts these distances into conditional probabilities that represent similarities.
 - Points that are close together are assigned a high probability, while distant points are assigned a very low probability.
- Low-Dimensional Projection
 - It maps these points to a lower-dimensional space (e.g., a 2D scatter plot) and calculates a similar probability distribution.
 - To solve the "crowding problem" (where points in multi-dimensional space get squished together when reduced to 2D), t-SNE uses a Student's t-distribution in the low-dimensional space, which allows distinct clusters to form rather than clumping in the center.
 - Finally, it uses gradient descent to minimize the difference (Kullback-Leibler divergence) between the two probability maps.

4.6 Unsupervised Learning Algorithm: t-SNE



4.6 Unsupervised Learning Algorithm: t-SNE

- Key Parameter: Perplexity
- The most important hyperparameter in t-SNE is perplexity, which dictates how many neighbors each data point considers when evaluating similarities.
- Low perplexity: Focuses heavily on the fine, local details and tiny clusters.
- High perplexity: Considers a larger neighborhood, which helps capture more of the global structure and broader arrangements.
- Typically, perplexity values range between 5 and 50.

4.6 Unsupervised Learning Algorithm: t-SNE

- Steps Involved:

Step 1: Calculate High-Dimensional Probabilities ($p_{j|i}$)

- For any point $\mathbf{x}_i$, we model the probability that it chooses $\mathbf{x}_j$ as its neighbor using a Gaussian distribution:

$$p_{j|i} = \frac{\exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_k\|^2}{2\sigma_i^2}\right)}, \quad p_{i|i} = 0$$

- where σ_i is chosen dynamically based on Perplexity (a smooth measure of the effective number of neighbors).
- To handle outliers, we convert conditional probabilities into joint probabilities and symmetrize them:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

Step 2: Calculate Low-Dimensional Probabilities (q_{ij})

- Now, we map the points to a low-dimensional space (typically 2D or 3D) as $\mathbf{y}_i$ and $\mathbf{y}_j$.
- We use a Student-t distribution with 1 degree of freedom (Cauchy distribution):

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_k \sum_{l \neq k} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}, \quad q_{ii} = 0$$

Step 3: Compute KL Divergence (Cost Function)

- We want q_{ij} to match p_{ij} as closely as possible. We measure the difference between these probability distributions using the Kullback-Leibler (KL) Divergence:

$$\mathcal{L} = \text{KL}(P \parallel Q) = \sum_i \sum_{j \neq i} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right)$$

Step 4: Minimize Cost Function using Gradient Descent

- Compute the gradient of the KL-divergence with respect to every coordinate of every point:

$$\frac{\partial KL}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij})(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1}$$

- Step 5: Update y_i using a simple gradient descent step.

$$y_1^{(new)} = y_1^{(old)} - \eta \frac{\partial KL}{\partial y_1}$$

4.6 Unsupervised Learning Algorithm: t-SNE (Example)

- Three students obtained the following marks in Mathematics and Physics. Use t-SNE algorithm and project the given data into 1D. Consider $\sigma_i=10$ and learning rate =10.

Student	Mathematics	Physics
A	50	55
B	52	53
C	90	92

Step 1: Compute High-Dimensional Similarity Probabilities:

(A) Calculate Euclidean distance between two points as:

$$\begin{aligned}\text{Distance bet}^n A \& B: d_{AB} &= \sqrt{(52-50)^2 + (53-55)^2} \\ &= \sqrt{8} \\ &= 2.828\end{aligned}$$

$$\begin{aligned}\text{Distance bet}^n A \& C: d_{AC} &= \sqrt{(90-50)^2 + (92-55)^2} \\ &= \sqrt{2969} \\ &= 54.488\end{aligned}$$

$$\begin{aligned}\text{Distance bet}^n B \& C: d_{BC} &= \sqrt{(90-52)^2 + (92-53)^2} \\ &= \sqrt{2965} \\ &= 54.452\end{aligned}$$

Mo Tu We Th Fr Sa Su

(B) Draw Distance Matrix as:

	A	B	C
A	0	2.828	54.488
B	2.828	0	54.452
C	54.488	54.452	0

(C) Compute Conditional Probabilities

For this $\sigma = 10 \therefore \sigma^2 = 100$
we can simply formula as:

$$P_j | i = \frac{\exp(-d^2/200)}{\sum \exp(-d^2/200)}$$

(1) Computing Probabilities for Point A i.e. Student A has neighbor as B & C.

$$\begin{aligned}\text{Similarity bet}^n A \& B: d &= 2.828 \\ \therefore d^2 &\approx 8 \\ \therefore \exp\left(-\frac{8}{200}\right) &= e^{-0.04}\end{aligned}$$

$$(0.8-0.8) + \dots (0.8-0.8) = 0.96079$$

$$\begin{aligned}\text{Similarity bet}^n A \& C: d &= 54.488 \\ d^2 &\approx 2969 \\ \therefore \exp\left(-\frac{2969}{200}\right) &= e^{-14.845} \\ &= 3.57 \times 10^{-7}\end{aligned}$$

$$\text{Denominator} = 0.96079 + 0.000000357$$

$$= 0.960790357$$

Probability of B given A:

$$P(B|A) = \frac{0.96079}{0.960790357}$$

$$= 0.99999963$$

Probability of C given A:

$$P(C|A) = \frac{0.000000357}{0.960790357}$$

$$= 0.00000037$$

$$\therefore 0.99999963 + 0.00000037 = 1$$

We can say A finds B as its neighbor.

(ii) Computing Probabilities for Point B with neighbors A & C.

$$\text{Similarity bet}^n A \& B = 0.96079$$

$$\text{Similarity bet}^n B \& C = d = 54.452$$

$$d^2 = 2965$$

$$\therefore e^{-2965/200} = 3.64 \times 10^{-7}$$

$$\text{Denominator} = 0.96079 + 0.000000364$$

$$= 0.960790364$$

Probability of A given B:

$$P(A|B) = \frac{0.96079}{0.960790364} = 0.99999962$$

Probability of C given B:

$$P(C|B) = \frac{0.000000364}{0.960790364} = 0.00000038$$

again sum of two probabilities is 1.

$\therefore$ From B's perspective, B also considers A as its neighbor.

(iii) Computing Probabilities for C with neighbors A & B.

$$\text{Similarity bet}^n (A \& C) = 3.57 \times 10^{-7}$$

$$\text{Similarity bet}^n (B \& C) = 3.64 \times 10^{-7}$$

$$\text{Denominator} = 3.57 \times 10^{-7} + 3.64 \times 10^{-7}$$

$$= 7.21 \times 10^{-7}$$

Probability of A given C:

$$P(A|C) = \frac{3.57}{7.21} = 0.495$$

Probability of B given C:

$$P(B|C) = \frac{3.64}{7.21} = 0.505$$

Date: / /



∴ From C's perspective A & B are almost equally far away.

∴ A & B forms a cluster however C remains detached in another cluster.

(iv) Draw Conditional Probability Matrix.

	A	B	C
A	0	0.99999963	0.00000037
B	0.99999962	0	0.00000038
C	0.495	0.505	0

Here, $P_{C|A} \neq P_{A|B}$, however

⊗ t-SNE requires a single symmetric probability for every pair.

Ⓓ Compute Symmetric Probability:

$$P_{ij} = \frac{P_{ji} + P_{ii}}{2N}$$

$$\text{here } N=3 \therefore 2N=6$$

$$P_{AB} = \frac{0.99999963 + 0.99999962}{6}$$

$$= 0.33333321$$

Mo Tu We Th Fr Sa Su

$$P_{AC} = \frac{0.00000037 + 0.495}{6}$$

$$= 0.08250006$$

$$P_{BC} = \frac{0.00000038 + 0.505}{2}$$

$$= 0.08416673$$

Our total probability should be $\sum_{i \neq j} P_{ij} = 1$

Since, $P_{ij} = P_{ji}$ we have:

$$= 2(0.33333321 + 0.08250006 + 0.08416673)$$

$$= 2(0.49999999)$$

$$= 0.99999998$$

$$\approx 1$$

Ⓔ Draw Symmetric Probability Matrix

P_{ij}	A	B	C
A	0	0.33333321	0.08250006
B	0.33333321	0	0.08416673
C	0.08250006	0.08416673	0

∴ A-B pair has highest probability.
Hence, A & B should be neighbors in the final map.

Step 2: Low Dimensional Projection

Ⓐ Random Initialization:
Let current 1D Co-ordinates be:
 $y_1 = 0$, $y_2 = 1$, $y_3 = 2$
(A) (B) (C)

Ⓑ Compute distance in 1D:
 $d = \sqrt{(y_1 - y_2)^2} = \sqrt{(0 - 1)^2} = \sqrt{1} = 1$
 $= \sqrt{(y_1 - y_3)^2} = \sqrt{(0 - 2)^2} = \sqrt{4} = 2$
 $= \sqrt{(y_2 - y_3)^2} = \sqrt{(1 - 2)^2} = \sqrt{1} = 1$

Ⓒ Compute low Dimensional Probabilities

Ⓓ Compute unnormalized similarity as:

for (1,2) $q_{ij}^* = \frac{1}{1 + \|y_i - y_j\|^2} = \frac{1}{1 + 1} = \frac{1}{2} = 0.5$

For (1,3): $q_{13}^* = \frac{1}{1 + \|y_1 - y_3\|^2} = \frac{1}{1 + 4} = \frac{1}{5} = 0.2$

For (2,3): $q_{23}^* = \frac{1}{1 + \|y_2 - y_3\|^2} = \frac{1}{1 + 1} = \frac{1}{2} = 0.5$

Ⓙ Compute the normalization constant.

$$Z = \sum_{1 \leq k \neq l} q_{kl}^*$$

Denominator sum $(Z) = (0.5 + 0.2 + 0.5) \times 2$
 $= 1.2 \times 2 = 2.4$

Ⓜ Compute low dimensional probabilities as:

$$Q_{ij} = \frac{q_{ij}^*}{Z}$$

$\therefore Q_{12} = 0.5 / 2.4 = 0.208$

$\therefore Q_{13} = 0.2 / 2.4 = 0.083$

$\therefore Q_{23} = 0.5 / 2.4 = 0.208$

④ Compare P & Q:

Pair	high-dimensional	Low-dimensional
A-B	0.3333	0.208
B-C	0.0842	0.208
A-C	0.0825	0.083

Here as per our high-dimensional probabilities A-B was neighbors however low dimensional projection shows A & B have drifted apart while B & C has come together. Hence, using optimization A & B should move closer while B & C should move farther apart.

⑤ Compute the KL Divergence (Cost Function):

$$KL = (P || Q) = \sum_{i \neq j} P_{ij} \log \left(\frac{P_{ij}}{Q_{ij}} \right)$$

$$\text{Pair (A-B)} = 0.3333 \log \left(\frac{0.3333}{0.208} \right) = 0.1567$$

$$\text{Pair (B-C)} = 0.0842 \log \left(\frac{0.0842}{0.208} \right) = -0.0763$$

$$\text{Pair (A-C)} = 0.0825 \log \left(\frac{0.0825}{0.083} \right) = -0.00079$$

Since each pair is symmetric:

$$\begin{aligned} KL(P || Q) &= 2 \times (0.1567 - 0.0763 - 0.00079) \\ &= 0.1592, \text{ this value should be minimum} \end{aligned}$$

∴ Now, optimization should be done to reduce this value using gradient descent.

⑥ Compute Gradient Descent:

$$\frac{\partial KL}{\partial y_i} = \sum_{j \neq i} (P_{ij} - Q_{ij}) (y_i - y_j) (1 + \|y_i - y_j\|^2)^{-1}$$

① For Student i=A: j are B & C

For j=B

$$P_{12} = Q_{12}$$

$$(P_{AB} - Q_{AB}) = 0.3333 - 0.208 = 0.1250$$

$$(A-B) = 0 - 1 = -1$$

$$\frac{1}{1 + (A-B)^2} = \frac{1}{1 + 1^2} = \frac{1}{2} = 0.5$$

$$\therefore \text{Product} = 0.1250 \times -1 \times 0.5 = -0.0625$$

For $j = C$:

$$\therefore P_{AC} - Q_{AC} = 0.0825 - 0.0833 = -0.0008$$

$$y_A - y_C = 0 - 2 = -2.0$$

$$\frac{1}{1 + \|y_A - y_C\|^2} = \frac{1}{1 + (-2)^2} = \frac{1}{5} = 0.2$$

$$\therefore \text{Product} = -0.0008 * -2 * 0.2$$

$$= 0.00032$$

$$\therefore \frac{\partial K_L}{\partial y_i} = 1 * (-0.0625 + 0.00032)$$

$$\frac{\partial K_L}{\partial y_i} = -0.2487$$

(F) Update value of y_i i.e. Student A's position:

$$y_i(\text{new}) = y_i(\text{old}) - \eta * \frac{\partial K_L}{\partial y_i}$$

$$= 0 - 10 * -0.2487$$

$$= 2.487$$

$\therefore$ Student A moves from 0 to 2.487.

4.7 Semi supervised and Reinforcement Learning

- Semi Supervised Learning

- Semi-supervised learning is a machine learning approach that trains a model using a small amount of labeled data, and a large amount of unlabeled data.
- It combines ideas from both supervised and unsupervised learning.
- The goal is to use a small amount of labeled data (X_l, y_l) and a large amount of unlabeled data (X_u) to build a better classifier than what we could build with only the labeled data.
- Suppose we have 20 labeled images of cats and dogs & 5,000 unlabeled images. Instead of ignoring the 5,000 images, Semi Supervised Learning uses them to learn the structure of the data, resulting in better classification.

4.7 Semi supervised and Reinforcement Learning

- Assumptions of Semi Supervised Learning

- For Semi Supervised Learning to work, we must rely on one or more of these assumptions:

- Continuity/Smoothness: Points close to each other are likely to share a label. This is a general machine learning assumption.

- Cluster: Data tends to form discrete clusters, and points in the same cluster share a label.

- Manifold: Data lies on a low-dimensional manifold. We can learn this manifold from the unlabeled data and then project the labeled data onto it to classify.

- If these assumptions are violated, Semi Supervised Learning can actually hurt performance. If the unlabeled data is full of outliers or belongs to different classes that overlap heavily, adding it can mislead the model.

Semi Supervised Learning: Example(Self-Training)

- Self-training is one of the simplest and most widely used Semi Supervised Learning algorithms.

Algorithm

- Train a supervised classifier f on the labeled data L .
- Use f to predict labels for the unlabeled data U .
- Select the most confident predictions from U (e.g., those with probability > 0.9). Add these data points and their *pseudo-labels* to the labeled set L .
- Retrain f on the expanded L .
- Repeat steps 2-4 until convergence or a stopping criterion is met.
- This algorithm learns a decision boundary. The unlabeled points near the boundary will have low confidence and are ignored.
- The points far from the boundary in high-density areas will have high confidence and are added, reinforcing the model's understanding of the dense regions and pushing the boundary even further away from them.

Semi Supervised Learning: Example(Self-Training)

- Example: Suppose you have small set of labelled data to classify the images of animals as Dog or Cat and also you have set of unlabeled data as:

- Labelled Dataset:

Image	Label
Cat Image	Cat
Dog Image	Dog

- Unlabeled Dataset:

Image	Label
Cat Image (Image A)	?
Dog Image (Image B)	?
Dog Image (Image C)	?
Cat Image (Image D)	?

Semi Supervised Learning: Example(Self-Training)

- Step 1: Train an initial classifier using labeled data.
- Step 2: Predict labels for unlabeled samples as:

Image	Predicted Label	Confidence
Image A	Cat	99%
Image B	Dog	98%
Image C	Dog	50%
Image D	Cat	97%

- Only high-confidence predictions are selected i.e. Image A, B and D are selected.
- Step 5: Add confidently labeled samples to the training set. Retrain the model and repeat the process.

4.7 Semi supervised and Reinforcement Learning

- Reinforcement Learning:
 - Reinforcement Learning (RL) is a machine learning technique in which an agent learns to behave by taking actions in an environment to achieve some goal.
 - In this learning method, agent learn through trial-error method by receiving some rewards for good move while receiving penalties for bad moves.

4.7 Semi supervised and Reinforcement Learning

- Key Components of Reinforcement Learning

Agent = Everything that interacts with the environment.

Action = All the possible moves the agent can make.

State = A situation the agent finds itself in within the environment.

Reward = A feedback signal. After each action, the agent receives a reward (or penalty), which is a numerical score to learn what is good and bad.

Policy = A strategy that agent employs to determine its next action based on the current state.

Value Function = It estimates what the total amount of reward an agent can expect to accumulate over the future, starting from a specific state.

Action Value = This is similar to value except it takes extra parameter which is current action.

Reinforcement Learning Concepts

- **Reward Maximization:** It states that a Reinforcement Learning agent must be trained in such a way that it takes the best action so that the reward is maximum.
- **Exploitation:** Exploitation is about using the already known exploited information to maximize the rewards.
- **Exploration:** Exploration is about exploring and capturing more information about the environment.



A → B
+20
-10
Path 1 - +60
Path 2 - +80



A → B
+20
-10
Path 1 - +60
Path 2 - +80
Path 3 - +100

Exploitation to Exploration

Reinforcement Learning: Q-Learning

- Q-Learning is a model-free, off-policy reinforcement learning algorithm that find the next best action, given a current state.
- It chooses this action at random and aims to maximize the reward.
- Model-free means the agent doesn't need to know the transition probabilities or the reward function . It learns directly from experience.
- Off-policy means agent can learn using different methods which is outside the current policy to get the optimal result.

Reinforcement Learning: Q-Learning

Key Terms in Q-Learning

- State = represents the current position of agent in an environment.
- Action = step taken by the agent when it is in a particular state.
- Rewards = for every action, agent will get a positive or negative reward.
- Episodes = when an agent ends up in a terminating state and can't take a new action.
- Q-values = used to determine how good an action 'A' is, taken at a particular state, 'S' and is denoted as $Q(A, S)$
- Temporal Difference = a formula used to find the Q-value by using the value of current state and action & previous state and action.
- Bellman's Equation = equation used to determine the value of a particular state and deduce how good it is to be in or take that state. The optimal state will give us the highest optimal value.

$$Q(s, a) = R(s, a) + \gamma \max_{a'} Q(s', a') \text{ Where:}$$

- $Q(s, a)$ is the Q-value for a given state-action pair.
- $R(s, a)$ is the immediate reward for taking action a in state s.
- γ is the discount factor, representing the importance of future rewards.
- $\max_{a'} Q(s', a')$ is the maximum Q-value for the next state s' and all possible actions.

Q-Learning-Example

- Suppose we have 5 rooms in a building connected by doors as shown in the figure. We'll number each from 0 to 4. The outside of the building can be thought of as one big room (5). Notice that doors 1 and 4 lead into the building i.e. room 5 (outside). We represent the building as a graph where each room will be considered as a node or state and each door is considered as action/link. The goal room is 5. The doors that lead immediately to the goal have an instant reward of 100. Other doors not directly connected to the target room have zero reward. Each arrow contains an instant reward value, as shown in the figure below.

- To summarize:

States: 0,1,2,3,4,5

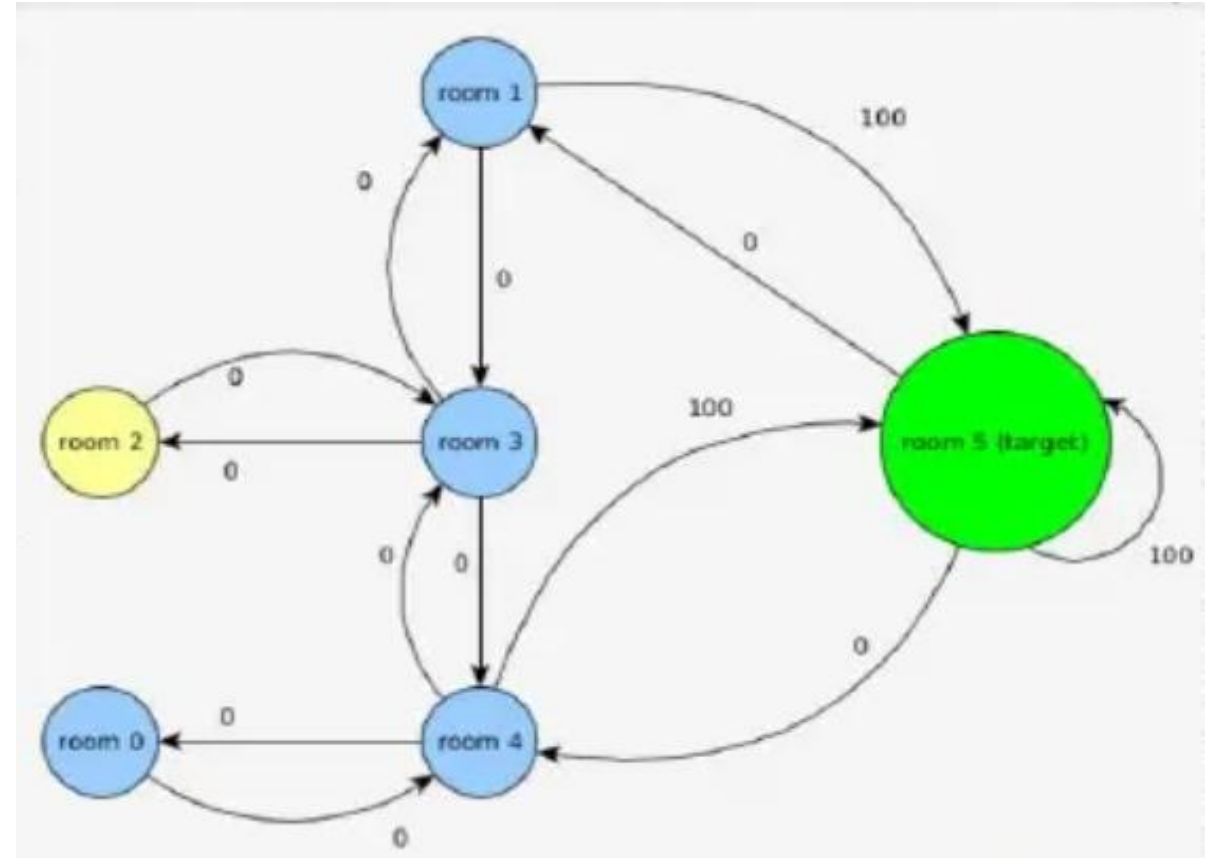
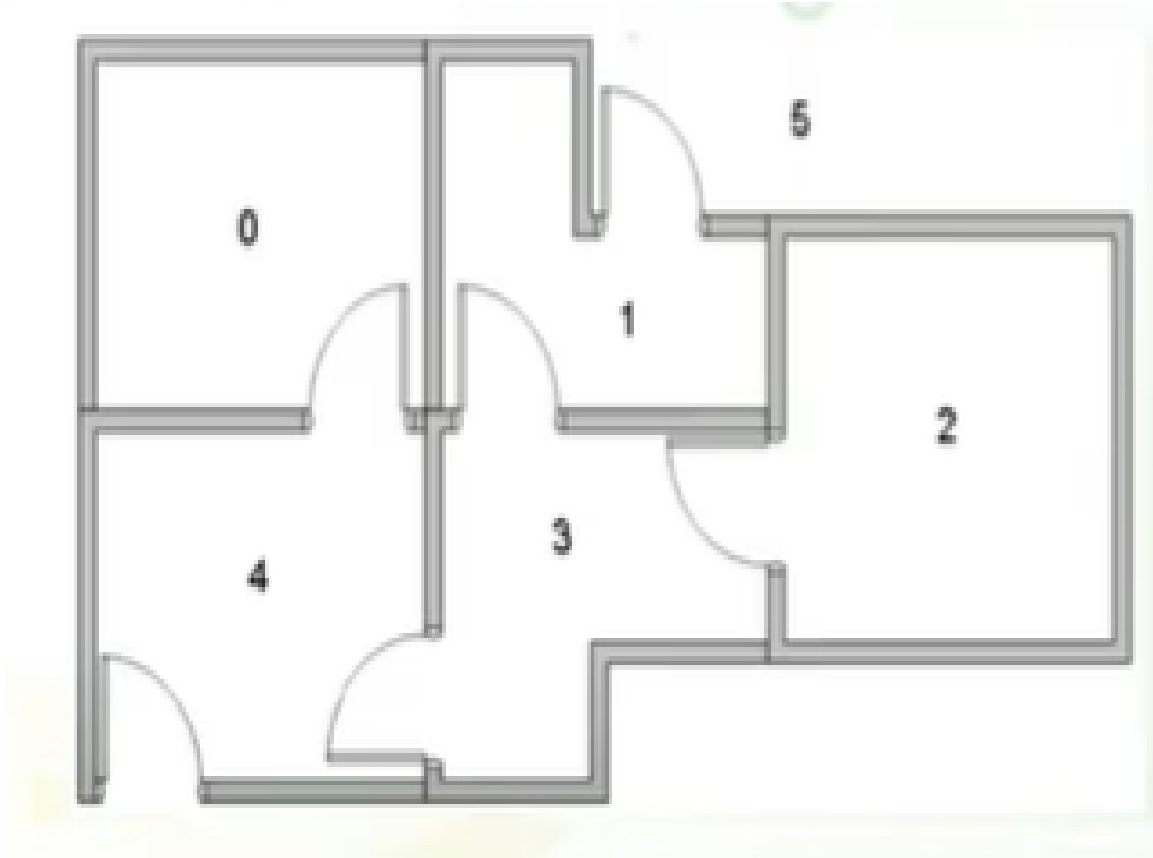
Actions: 0,1,2,3,4,5

Rewards: 0, 100

Goal State: 5

$\gamma = 0.8$

Q-Learning: Example



- Step 1: Draw the Reward table as:

$$R = \begin{array}{c|cccccc} & \text{Action} \\ \text{State} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & -1 & -1 & -1 & -1 & 0 & -1 \\ 1 & -1 & -1 & -1 & 0 & -1 & 100 \\ 2 & -1 & -1 & -1 & 0 & -1 & -1 \\ 3 & -1 & 0 & 0 & -1 & 0 & -1 \\ 4 & 0 & -1 & -1 & 0 & -1 & 100 \\ 5 & -1 & 0 & -1 & -1 & 0 & 100 \end{array}$$

- Step 2: Initialize all Q-Values to 0.

$$Q = \begin{array}{c|cccccc} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

- Step 3: Perform Q-value update for multiple Episodes. For this let our initial state be room = 1.

Episode 1

As we start from state "room 1" (second row) there are only the actions 3(reward 0) or 5(reward 100) do be done, imagine that we choose randomly the action 5.

		Action					
State		0	1	2	3	4	5
$R=$	0	-1	-1	-1	-1	0	-1
	1	-1	-1	-1	0	-1	100
	2	-1	-1	-1	0	-1	-1
	3	-1	0	0	-1	0	-1
	4	0	-1	-1	0	-1	100
	5	-1	0	-1	-1	0	100

On this new (next state 5) there are 3 possible actions: 1 , 4 or 5, with their rewards 0,0,100. Basically is all positive values from row "5", and we're just interested on the one with biggest value. We need to select the biggest Q value with those possible actions by selecting $Q(5,1)$, $Q(5,4)$, $Q(5,5)$, then using a "max" function. But remember that at this state the Q table is still filled with zeros.

$$Q(1, 5) = R(1, 5) + 0.8 \cdot \max([Q(5, 1), Q(5, 4), Q(5, 5)])$$

$$\therefore Q(1, 5) = 100 + 0.8(0)$$

As the new state is 5 and this state is the goal state, we finish our episode. Now at the end of this episode the Q matrix will be:

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

Episode 2

On this new state we randomly selected the state "room 3", by looking the R matrix we have 3 possible actions on this state, also now by chance we chose the action 1

$$R = \begin{matrix} & \begin{matrix} \text{Action} \\ 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{matrix}$$

By selecting the action "1" as our next state will have now the following possible actions

		Action					
State		0	1	2	3	4	5
$R =$	0	-1	-1	-1	-1	0	-1
	1	-1	-1	-1	0	-1	100
	2	-1	-1	-1	0	-1	-1
	3	-1	0	0	-1	0	-1
	4	0	-1	-1	0	-1	100
	5	-1	0	-1	-1	0	100

		0	1	2	3	4	5
$Q =$	0	0	0	0	0	0	0
	1	0	0	0	0	0	100
	2	0	0	0	0	0	0
	3	0	80	0	0	0	0
	4	0	0	0	0	0	0
	5	0	0	0	0	0	0

Now let's update our Q table:

$$Q(3, 1) = R(3, 1) + 0.8 \cdot \max([Q(1, 3), Q(1, 5)])$$

$$\therefore Q(3, 1) = 0 + 0.8(100) = 80$$

As our new current state 1 is not the goal state, we continue the process. Now by chance from the possible actions of state 1, we choose the action 5. From the action 5 we have the possible actions: 1,4,5 [Q(5,1), Q(5,4), Q(5,5)] unfortunately we did not compute yet these values and our Q matrix remains unchanged.

Episode 100000

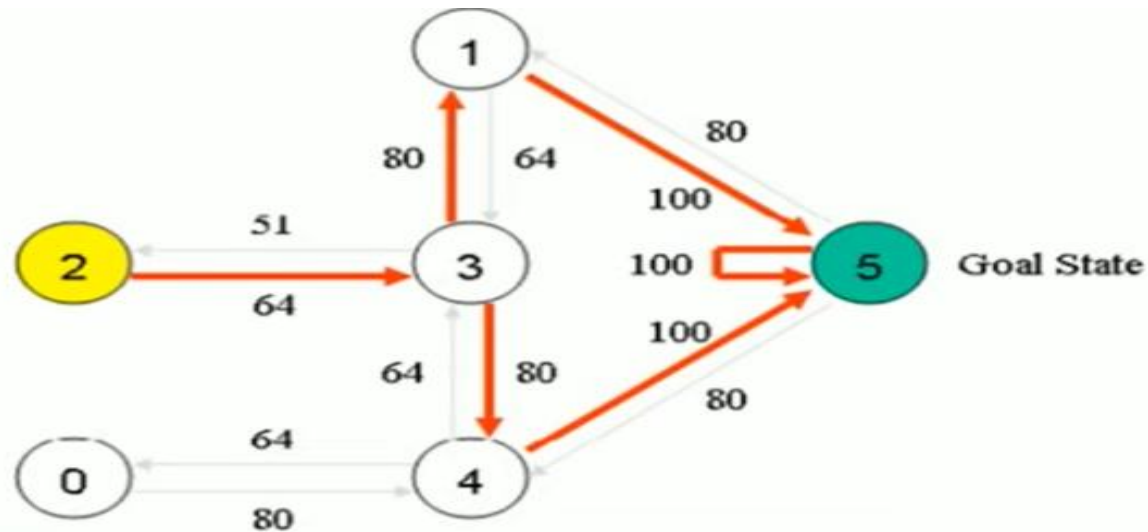
After a lot of episodes our Q matrix can be considered to have converged, in this case Q will be like this:

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 400 & 0 \\ 0 & 0 & 0 & 320 & 0 & 500 \\ 0 & 0 & 0 & 320 & 0 & 0 \\ 0 & 400 & 256 & 0 & 400 & 0 \\ 320 & 0 & 0 & 320 & 0 & 500 \\ 0 & 400 & 0 & 0 & 400 & 500 \end{bmatrix} \end{matrix}$$

if we divide all of the nonzero elements by it's greatest value (on this case 500) we normalize the Q table (Optional):

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{matrix}$$

On receiving this updated Q-table, tracing the best sequences of states is as simple as following the links with the highest values at each state.



4.8 Model Evaluation: Energy-based indicators

- For very long, we evaluated AI models only by accuracy. If Model A had 90% accuracy and Model B had 91%, we chose Model B, regardless of size.
- The Paradigm Shift: We have hit the "Hardware Wall." The compute required to train state-of-the-art models doubles roughly every 3.4 months.
- The Three Pillars of Cost:
 - Financial Cost: Training GPT-3 cost an estimated \$4.6 million in cloud compute alone.
 - Environmental Cost: Training a single large transformer model emits roughly 280,000 kg of CO₂ (that's the lifetime emissions of 5 average cars).
 - Latency Cost: If your model takes 2 seconds to infer, it's useless for autonomous braking in a car or real-time voice translation.
 - Today's Goal is to learn how to measure, compare, and optimize these "Energy-Based Indicators" so we can build models that are not just smart, but also lean and efficient.

Energy-Based Indicators

- The major indicators are:
 - Training Time (or Training Energy)
 - Model Size
 - Inference Time (or Inference Energy)
- These indicators evaluate how efficiently a model uses computational resources.

Training Time or Training Energy

- This indicator measures the total computational effort required to take a randomly initialized model and converge to a satisfactory minimum.
- How We Measure It
 - A. Time (Wall-clock time)
 - The easiest metric, but highly dependent on hardware. Measured in:
 - Seconds/Minutes (for small models)
 - Days/Weeks (for foundation models)
 - B. Energy (Joules / kWh)
 - The actual electrical energy consumed. This depends on:
 - Power Draw of the GPU/CPU = (Watts) × Training Time (Hours).
 - Cooling overhead in the data center (often 1.5 x the compute energy).

Training Time or Training Energy

- Training time is the total amount of time required to train a machine learning model.
- It includes time for loading the dataset, forward propagation, loss calculation, backpropagation and updating model parameters.
- Formula: Training Time = Training End Time – Training Start Time
- Example: Training starts at 9:00 AM & Training ends at 11:30 AM then Training Time = 11:30 – 9:00. Therefore, Training Time = 2.5 hours.
- Factors affecting training time are Dataset size, Number of epochs, Batch size, Hardware (CPU/GPU), Number of parameters & Optimization algorithm.
- Advantages of Lower Training Time:
 - Faster experimentation
 - Lower computational cost
 - Lower electricity consumption
 - Faster deployment

Training Time or Training Energy

- Training energy is the electrical energy consumed while training the model. Large AI models require powerful hardware, which consumes significant electricity.
- Formula: Training Energy = Power $\times$ Training Time where; Power is measured in Watts (W), Training Time is measured in Hours (h) & Energy is measured in Watt-hours (Wh).
- Example: GPU Power = 250 W, Training Time = 4 hours then Training Energy = 250×4 . Therefore, Training Energy = 1000 Wh = 1 kWh
- Importance of Low Training Energy Consumption are:
 - Lower electricity bills
 - Reduced carbon emissions
 - More sustainable AI

Model Size

- Model size refers to the storage footprint of the trained model. It dictates whether your model can fit into RAM, GPU memory, or a smartphone.
- It is usually measured in KB, MB or GB.
- Formula: $\text{Model Size} = \text{Number of Parameters} \times \text{Bytes per Parameter}$
- Example: Number of Parameters = 2,000,000, Bytes per Parameter = 4, Model Size = $2,000,000 \times 4 = 8,000,000$ Bytes $\approx$ 8 MB.
- The Hidden Costs of Model Size:
 - Communication Overhead: In distributed training (across many GPUs), large models require massive bandwidth to synchronize gradients. This introduces latency and network energy costs.
 - Serving Infrastructure: If your model is 175 GB, you can't run it on a single consumer GPU. You need multiple high-end A100s (costing ~\$10,000 each) just to load it into memory.
 - Mobile Deployment: For on-device AI (like Siri or Google Assistant), the model must be under 500 MB. If it's bigger, it has to run in the cloud, incurring network delays and cloud energy costs.

Model Size

- Why Small Models Are Better?
- Small models:
 - Load quickly
 - Require less storage
 - Consume less memory
 - Are suitable for mobile phones and IoT devices
- Evaluation Insight: Model size is a direct proxy for operational cost. "Parameter Efficiency" is now a primary evaluation metric. If Model A achieves 90% accuracy with 10M parameters, and Model B achieves 91% with 100M parameters, Model A is often the superior engineering choice despite being slightly less accurate.

Inference Time or Inference Energy

- This is arguably the most critical metric for real-world applications. While a model is trained once, it is inferred (run) millions or billions of times over its lifetime.
- Inference time is the time required by a trained model to make one prediction. Unlike training, inference occurs during actual use.
- Formula: Inference Time = Prediction End Time – Prediction Start Time
- Example: Prediction starts at 10.000 seconds & Prediction finishes at 10.015 seconds then Inference Time = 10.015 – 10.000.
- Inference Time = 0.015 seconds = 15 milliseconds.
- Low inference time is essential for self-driving cars, medical diagnosis, face recognition, robotics & real-time surveillance.

Inference Time or Inference Energy

- Inference energy is the electrical energy consumed while making one prediction.
- Although one prediction consumes little energy, billions of predictions every day consume a large amount of electricity.
- Formula: Inference Energy = Power $\times$ Inference Time
- Example: CPU Power = 30 W, Inference Time = 0.5 seconds
- Convert time into hours:
- $0.5 \div 3600 = 0.000139$ hours
- Inference Energy = 30×0.000139
- Inference Energy = 0.00417 Wh

Inference Time or Inference Energy

- The Metrics of Inference

- Latency: The time it takes to process a single input (measured in milliseconds). Crucial for real-time systems (autonomous driving, high-frequency trading).
- Throughput: The number of inputs processed per second (measured in inferences per second). Crucial for batch systems (analyzing millions of user photos overnight).
- Energy per Inference: Joules required per single forward pass. The most accurate measure for battery-constrained devices (smartphones, IoT sensors).

Inference Time or Inference Energy

- Why Inference is a Bottleneck?

Inference only requires the Forward Pass (no backpropagation). So, why is it so expensive?

- Memory Bandwidth is the Killer: For large models, the bottleneck isn't computation; it's moving the weights from the GPU's high-bandwidth memory into the compute cores. This memory access consumes massive energy. In fact, for most transformer models, over 90% of the inference energy is spent on moving data, not on math operations.
- Auto-Regressive Generation: For LLMs like ChatGPT, inference isn't just one forward pass. To generate a 100-word response, the model must run 100 forward passes (generating one token at a time). This drastically multiplies latency and energy.

Numerical Comparison

- Suppose two models have the following characteristics.

Metric	Model A	Model B
Accuracy	97%	96%
Training Time	10 hours	4 hours
GPU Power	250 W	200 W
Model Size	700 MB	100 MB
Inference Time	150 ms	20 ms

- Training Energy

Model A

Training Energy = $250 \times 10 = 2500 \text{ Wh} = 2.5 \text{ kWh}$

Model B

Training Energy = $200 \times 4 = 800 \text{ Wh} = 0.8 \text{ kWh}$

Here, although Model A has slightly higher accuracy, Model B trains faster, consumes less electricity, occupies less storage, predicts much faster. Therefore, Model B is more suitable for mobile and embedded devices.

References

- Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). Mathematics for machine learning. Cambridge University Press.
- E. Rich and Knight, *Artificial Intelligence*, McGraw Hill, 1991.
- D. W. Patterson, *Artificial Intelligence and Expert Systems*, Prentice Hall, 2001.